

SRCDIFF: SYNTACTIC DIFFERENCING TO
SUPPORT SOFTWARE MAINTENANCE AND EVOLUTION

A dissertation submitted
to Kent State University in partial
fulfillment of the requirements for the
degree of Doctor of Philosophy

by

Michael John Decker

August 19, 2017

Dissertation written by

Michael John Decker

B.S., The University of Akron, USA 2010

M.S., The University of Akron, USA, 2012

Ph.D., Kent State University, USA, 2017

Approved by

Dr. Jonathan I. Maletic

Chair, Doctoral Dissertation Committee

Dr. Michael L. Collard

Members, Doctoral Dissertation Committee

Dr. Feodor F. Dragan

Dr. Kambiz Ghazinour

Dr. John J. Portman

Dr. Jeffrey A. Ciesla

Accepted by

Dr. Javed I. Khan

Chair, Department of Computer Science

Dr. James L. Blank

Dean, College of Arts and Sciences

TABLE OF CONTENTS

LIST OF FIGURES	VII
LIST OF TABLES	XII
DEDICATION	XVIII
ACKNOWLEDGEMENTS	XIX
CHAPTER 1 INTRODUCTION.....	1
1.1 Research Questions	6
1.2 Research Contributions	7
1.3 Organization	8
CHAPTER 2 BACKGROUND AND RELATED WORK	9
2.1 Textual Differencing	10
2.2 Syntactic Differencing.....	13
2.3 Semantic Differencing.....	16
2.4 Track-changes Approaches	17
2.5 File Comparison Software.....	17
2.6 Other Differencing Approaches	18
2.7 Version Control Systems.....	18
2.8 Software Merging.....	19
2.9 Source-code Representations	22
2.10 Improving Upon the Literature	22
CHAPTER 3 SRCML INFRASTRUCTURE	25
3.1 srcML Format.....	26

3.2	srcML	28
3.3	To srcML Format	28
3.4	From srcML Format	30
3.4.1	libsreml.....	32
3.5	XML & Models	32
3.6	Tools	33
3.7	Support	33
CHAPTER 4 SYNTACTIC DIFFERENCING VIA SRCDIFF		34
4.1	srcDiff Format	34
4.2	srcDiff Algorithm	37
4.3	srcDiff Rules	44
4.3.1	Match Rules	47
4.3.2	Convertibility Rules	56
4.3.3	Nesting Rules	58
4.4	Complexity	62
CHAPTER 5 SRCDIFF TOOL		65
5.1	Implementation.....	65
5.2	Interface	69
CHAPTER 6 PRELIMINARY STUDY		74
6.1	Hypotheses	77
6.2	Results	78
6.3	Threats to Validity.....	83

CHAPTER 7 EVALUATION.....	86
7.1 Data Collection and Preparation	89
7.2 Measures.....	91
7.3 Results and Discussion.....	92
7.4 Timing	96
7.5 Threats to Validity.....	97
CHAPTER 8 INVESTIGATING METHOD STEREOTYPE HISTORY	100
8.1 Method Stereotypes.....	100
8.2 Evolution of a Method's Stereotype.....	103
8.3 Data Collection.....	106
8.4 Results and Discussion.....	111
8.4.1 Empirical Results	111
8.4.2 Manual Investigation.....	117
8.5 Threats to Validity.....	120
8.6 Conclusion.....	123
CHAPTER 9 INVESTIGATING CODE REWRITE/REPLACEMENT AND CODE	
CONVERSION	124
9.1 Rewrite and Replacement.....	125
9.2 Conversion	127
9.3 Data Collection.....	130
9.4 Results and Discussion.....	135
9.4.1 Replace Results and Discussion.....	136

9.4.2	Convert Results and Discussion.....	146
9.5	Threats to Validity.....	152
9.6	Conclusion.....	154
CHAPTER 10 CONCLUSION.....		155
APPENDIX A CODE FOR SRCDIFF TOOL INTEGRATION INTO GIT.....		158
APPENDIX B COMPLETE SRCDIFF TOOL OPTIONS		159
APPENDIX C SOURCE-CODE CHANGES USED IN PRELIMINARY STUDY		161
APPENDIX D SOURCE-CODE CHANGES USED IN THE EVALUATION		191
REFERENCES.....		302

LIST OF FIGURES

Figure 1.1. Example change showing the deficiency of textual differencing approaches on source code. The top-left is the original source code. A { was added at the end of the first line, and a new line with a } was added at the end, resulting in the code on the top-right. The resulting textual difference (line difference) is shown on the bottom. A deleted line is shown with a preceding – and an insert with a preceding +. Background colors have been added to highlight the change with red for delete and green for insert. There are two problems with the line difference. First, the addition of the { is much smaller than a line, yet the whole line is marked as deleted and inserted. Secondly, the logical change is the addition of the block as a whole { } where several separate modifications are indicated here..... 3

Figure 1.2. A simple example of a line-diff where two associated lines are not logically grouped together. In this example, the method call to `anOrder.basePrice()` is extracted and replaced with a new variable `basePrice`. Even though, it is the same return statement, another edit lies between the inserted and deleted lines..... 4

Figure 2.1. The output of GNU diff on a method `setImage` (simplified to fit) from KOffice revisions 1026809-1026810. On the top is the original and modified function. The middle is the side-by-side view, and the bottom is the unified view. 12

Figure 3.1. Diagram of the srcML Infrastructure. The foundation is the srcML Format. On top of this is the srcML tool, which converts source code to and from the srcML Format. From this core functionality, XML technologies and additional models can

be leveraged to develop tools to support exploration, analysis, and manipulation of large software systems. The infrastructure is supported (right) via a multi-university development team.....	26
Figure 3.2. Example source code function and its markup in srcML. Syntax information such as a section of code being a function or an expression statement is encoded by wrapping the appropriate source code with XML tags. The process is lossless. All the original text, including whitespace and comments is preserved.....	27
Figure 3.3. Example srcML archive. A separate <code>unit</code> tag is added as a root tag around the file <code>unit</code> tags.....	28
Figure 4.1. Example source-code change. The top-left contains the original source code and the top-right contains the modified source code. The original and modified code contained within the srcDiff format is given. Deletions are highlighted in red and with a line-through mark. Insertions are highlighted with green . All the original source code is placed in bold.....	36
Figure 4.2. An example of the changed methods from Figure 4.1 in two different human readable formats produced by srcDiff. The top is a side-by-side view of the change, where the original method is on the left with deleted code marked with a red background and the modified method is on the right with inserted code marked with a green background. The bottom is a unified view where both methods are merged into one with deleted code marked with a red background and inserted code marked with a green background.	37

Figure 4.3. The srcDiff algorithm. A preorder traversal is performed on both the original and modified trees. The input is the original and modified children of a node. A sequence-differencing method is used to categorize the children as common, unique to one version, or changed. Common, inserted, and deleted children are output directly. Changed children are further analyzed in `determineMatchings` and `determineNestings` for the following situations: a child in the original is a previous version of a child in the modified, a child and its subtree in one version subsumes children in the other version, and a child is unique to one version. The functions `determineMatchings` and `determineNestings` use a set of rules to identify these situations 42

Figure 4.4. Example of the srcDiff process. a) The original and modified version of a declaration statement with their tree representations below each. b) With `decl_stmt` as common root, run a sequence diff on their child sequences (blue). Compares are between entire child subtrees. c) Output of sequential diff. red is delete edit, green is insert edit, remaining are common. `<specifier>` subtree is an insert and `<init>`'s are a change. Common and insert/delete require no further action. The change needs to be analyzed to see if *Match*, *Nested*, or *Inserted/Deleted*..... 43

Figure 4.5. Textual similarity rules. The rules use similarity, dissimilarity and size measures to test if two elements are similar enough textually. The measures, with exception to rule 2, are calculated after stripping out punctuation characters and

other tokens that negatively affect performance. Rule 2 uses all tokens to test for an exact match. It is not applied to <i>Nesting</i> rules.....	55
Figure 5.1. A simplified high-level view of modules in <code>srcdiff</code>	65
Figure 5.2. Examples of <code>srcDiff</code> output provided by the <code>view</code> module. Top shows the original and modified source code, middle is the default theme and default level of syntax highlighting. Bottom is the Monokai Theme with full syntax highlighting. Both outputs are taken from <code>srcDiff</code> 's <code>view</code> module to HTML.	69
Figure 6.1. Answers to preferability question (Q5) as a stacked bar chart. Blue is the percentage selected as most preferable, green second preferable, and red least preferable	83
Figure 9.1. Example code replacement. The body of the <code>for</code> -statement is completely deleted and replaced with a different body.....	125
Figure 9.2. Simplified example of <code>srcDiff</code> markup of code replacement. Red is delete and green is insert. A <code>diff:delete</code> tag and <code>diff:insert</code> tag are issued consecutively (one after the other) and they both are given the type <code>replace</code> . Here an expression-statement, <code>if</code> -statement, and <code>while</code> -statement are replaced with a declaration-statement and a different expression-statement.....	127
Figure 9.3. Example code conversion. The original code is on the top-left, modified top-right, and <code>srcDiff</code> on the bottom. In the example, the <code>for</code> -statement was converted into an equivalent <code>while</code> -statement. The body of the <code>for</code> -statement is largely unchanged.	128

Figure 9.4. Output of GumTree on a simple code conversion. Red is delete, yellow-green insert, and blue is move. A while-statement is converted to a for-statement. GumTree marks the while-statement deleted and for-statement inserted and a series of moves between the two.	129
Figure 9.5. Example markup of a code conversion in srcDiff. Red is delete and green is insert. A <code>diff:delete</code> is wrapped around the original syntactic element (and keyword if present) with the type <code>convert</code> , and a <code>diff:insert</code> is wrapped around the modified syntactic element (and keyword if present) with type <code>convert</code> . The contents are then marked as common, inserted, and deleted as necessary.....	130
Figure 9.6. The total number of revisions plotted against the number of replaces. X-axis is number of revisions and y-axis is the number of replaces. The result is fairly linear.	138
Figure 9.7. The total number of revisions plotted against the number of conversions. X-axis is number of revisions and y-axis is the number of conversions. The result is fairly linear.	149

LIST OF TABLES

Table 4.1. <i>Match</i> rules and the associated syntactic categories (srcML elements).	48
Table 4.2. Convertibility Rules organized by type. Each type has the associated syntactic categories.....	56
Table 6.1. List of null and alternate hypothesis tested for questions: Q1-Q4.....	78
Table 6.2. Results of Two-tailed Paired T-Tests on Q1 and Q2. Left-most column is the sample, and top-most is the questions. For each question, the mean of GNU <code>diff</code> and <code>srcDiff</code> views are given along with the t-test p-value and finally if we can reject the null hypothesis	79
Table 6.3. Results of Two-tailed Paired T-Tests on Q3-Q4. Left-most column is the sample, and top-most is the questions. For each question, the mean of GNU <code>diff</code> and <code>srcDiff</code> views are given along with the t-test p-value and finally if we can reject the null hypothesis	80
Table 6.4. Effect size (Cohen's d) for each sample and for Q1 and Q2. In general, there was a small effect for Accuracy Changed (Q1) and almost no effect for Accuracy Common (Q2).....	80
Table 6.5. Effect size (Cohen's d) for each sample and for Q3 and Q4. In general, there was a medium to large effect on Accuracy (Q3) and Understandability (Q4).	81
Table 7.1. Evaluation survey questions.	92

Table 7.2. Results for each of the samples and post-questionnaire (Overall). All reported measures are medians with GitHub (GH), Mergerly (M), srcDiff (SD), and GumTree (GT). Last line shows a count of the number of samples (and Overall) for which the approaches achieved the best result (including ties) out of all the approaches. For Changed, smaller number is better, others larger is better.	94
Table 7.3. Results for each of the samples and post-questionnaire (Overall). All reported measures are medians with GitHub (GH), Mergerly (M), srcDiff (SD), and GumTree (GT). Last line shows a count of the number of samples (and Overall) for which the approaches achieved the best result (including ties) out of all the approaches. Larger is better.....	95
Table 8.1. Method stereotype taxonomy. The far-left column is the broad stereotype categories, the middle column the individual stereotypes, and the right column is a description of each stereotype	101
Table 8.2. Categories of important stereotype transitions. The category-column lists the categories of stereotype changes. The group-column groups similar categories, the description-column is a description of the category and what belong as part of that category, and the notes-column contains notes about the transitions.....	104
Table 8.3. Method stereotype transition categories (continued).....	105
Table 8.4. List of selected software systems by alphabetical order. Left column is system, middle is number of revisions with modified C++ files, and right is the repository location. Unqualified locations are from GitHub: https://github.com/{Location}.git	107

Table 8.5. Selected systems (continued).....	108
Table 8.6. Top ten most common stereotype transitions. from-column is original version stereotype, to-column is the modified version stereotype, count is the number of times it occurred, and % is the percentage of occurrence out of the all transitions.	112
Table 8.7. Top ten stereotype type transitions where the stereotype changed. from- column is the original version stereotype, to-column is modified version stereotype, count-column is the number of occurrences, and %-column is the percentage it occurred out of all transitions.	113
Table 8.8. Top ten purely deleted stereotypes. Left-column is the stereotypes deleted, the middle-column is the number of times those stereotypes were the only change in method stereotype, and the right-column is the percent out of all method stereotypes that changed.	114
Table 8.9. Top ten purely inserted stereotypes. Left-column is the stereotypes inserted, the middle-column is the number of times those stereotypes were the only change in method stereotype, and the right-column is the percent out of all method stereotypes that changed.	115
Table 8.10. Top ten completely changed stereotypes. The from-column is the original version stereotype, the to-column is the modified version stereotype, count-column is the number of occurrences and %-column is the percentage of occurrence out of all method stereotypes that changed.	116
Table 8.11. Top ten replaced stereotypes. The from-column is what was replaced, the to- column what it was replaced with, count-column is the number of occurrences, and	

%-column is the percentage of occurrence out of all method stereotypes that changed.....	116
Table 8.12. Summary of manual investigation. The category-column contains method stereotype change categories. Count is the number of times that category appeared. The suspicious-column contains the count and percent of the number of times that category had a change that should be investigated more deeply, and the benign-column is the count and percentage of the times that category was of no concern.	119
Table 9.1. Code conversion supported by srcDiff. Left is category and right are the syntactic elements that can be converted to each other.	130
Table 9.2. List of selected software systems by alphabetical order. Left column is system, middle is number of revisions with modified C++ files, and right is the repository location. Unqualified locations are from GitHub: https://github.com/{Location}.git	132
Table 9.3. Selected systems (continued).....	133
Table 9.4. Number of replaces per system (alphabetically). First column is system, the second is the number of replaces, and the average per revision.....	137
Table 9.5. Number of replaces (continued).	138
Table 9.6. Deleted elements in a replace. Left column is what element is deleted, the middle column is how many instances were deleted in all replaces, and the third column is the number each is replaced per revision.	140

Table 9.7. Inserted elements in a replace. Left column is what element is deleted, the middle column is how many instances were deleted in all replaces, and the third column is the number each is replaced per revision.....	141
Table 9.8. Number times an element was replaced with itself. The left column is the element, the middle column is number of occurrence, and the right column is the number (on average) that occurred in each revision.....	142
Table 9.9. Normalized deleted elements in a replace. The left-column is the deleted element. The right-column (normalized per revision) is calculated as: for each system, each element is normalized by the number of revisions for that system and the median across all systems is taken.....	143
Table 9.10. Normalized inserted elements in a replace. The left-column is the inserted element. The right-column (normalized per revision) is calculated as: for each system, each element is normalized by the number of revisions for that system and the median across all systems is taken.....	144
Table 9.11. Normalized times an element was replaced with itself. The left-column is the element. The right-column (normalized per revision) is calculated as: for each system, each element is normalized by the number of revisions for that system and the median across all systems is taken.....	145
Table 9.12. Total number of conversions for each system across all revisions. Left-column is system, middle is the number of converts, and the right column is the number of converts divided by revision (as percent). Last row contains total overall systems.	148

Table 9.13. Total conversions (continued).	149
Table 9.14. Frequency of each time of conversion. The left column is the number of conversions, the middle column is the raw number of how many times the conversion occurred between all systems, and the right column is the percentage that conversion occurred out of all types.....	150
Table 9.15. Percentage of each type of conversion normalized for each system by the total number of conversions.	151

DEDICATION

To the one True God, creator of Heaven and of Earth.

Soli Deo gloria

To my mother, sister, father, and the rest of my family who cared for and supported me in so many ways throughout my life.

ACKNOWLEDGEMENTS

I owe a great many people for helping me through this dissertation. I acknowledge only a few of them here. First, my Master's advisor, Michael L. Collard, and my Ph.D. advisor, Jonathan I. Maletic, who I owe very much for molding me into the researcher I am now. For the people who helped with various parts of my research such as Heather M. Guarnera who double checked my algorithm complexity, Natalia Dragan who identified the interesting method-stereotype changes, and Christian D. Newman who helped with confirming the method-stereotype manual analysis. I am also indebted to the members of the Software Development Laboratory (SDML) for their friendship, advice, and everything else. All of you are very dear to me. I hope to continue working with all of you long into the future.

I also would like to thank my family, friends, and members of Zion Evangelical Lutheran Church in Valley City for supporting me, as well as, my cat, Shampoo, for help keeping me sane.

Michael John Decker

July 10, 2017, Kent, Ohio

CHAPTER 1

Introduction

Developing and maintaining software is an iterative process of change. That is, software developers constantly improve and modify (add to and delete from) a code base a little at a time. In a system modified by a solitary developer, understanding these changes is generally straightforward. However, when scaled to real-world, large-scale systems (>500,000 lines of code) that involve large teams of individuals located in different geographical locations, understanding what exactly changed in a code base is often problematic and requires substantial effort by developers. It is essential for developers to quickly and easily comprehend changes made to source code. This is critical when understanding the impact of changes, merging changes from multiple developers, conducting code reviews and reviews of commit requests, and tracking down errors arising from changes.

Textual differencing tools such as GNU diff have been in wide use by developers for many years. They provide an extremely efficient means to determine the difference between two versions of a file (or set of files). These tools typically take a line-based view and produce the set of lines that have been inserted and/or deleted between the two versions (i.e., a delta). Textual differencing (diff) is the de facto standard for computing and presenting changes of source code. It is widely incorporated into Integrated Development Environments (e.g., Microsoft Visual Studio, Apple's Xcode) and version-control systems

(e.g., Git, Subversion). Line-based differencing approaches are very flexible (i.e., can be applied to any text file). They are also highly efficient (i.e., scale well to large files/projects). One of the best known textual differencing algorithm's (e.g., Myer's algorithm [Myers 1986]) has a run time complexity of $O(ND)$ where N is the sum of the lengths of both files (i.e., the number of lines) and D is the total number of edits (i.e., number of inserted/deleted lines). When the file is not greatly modified between commits (as is normal with commits), this, in practice, approaches a linear time algorithm. The efficiency and flexibility of textual differencing comes at a price. As they only treat the files as text, they understand nothing about the structure or syntax of the source code being differenced. As such, it is often difficult to understand the output in the context of the language syntax and structure.

In practice, source-code changes are typically more fine-grained than a line (e.g., an identifier is changed) or cross multiple lines (e.g., an entire loop or block is deleted). Moreover, a single logical modification may span multiple non-contiguous lines, such as adding a block (i.e., `{ }`) to an else-clause that previously did not have one. Figure 1.1 shows an example. The top-left shows an if-statement that was modified by adding a `{` at the end of the first line, and a `}` in a new line at the end. The top-right is the resulting source code. The bottom shows the resulting textual (line) difference. Deleted lines are preceded with a `-` and inserted lines with a `+`. Background colors have been added with red for delete and green for insert. There are two problems with the markup. First, the addition of the `{` is much smaller than a line, yet as the textual difference can only report line differences, the whole line is marked as deleted and inserted. Secondly, one logical

operation was performed on the code, that is, a block { } was inserted. However, the line difference shows several separate (seemingly unrelated) deletes and inserts that the developer has to cognitively associate.

Another problematic issue is that modifications to a line are represented as a deletion and then an insertion. The related –delete and +insert can occur many lines apart in the delta, making it very difficult to understand the (simple) change. Figure 1.2 shows a simple example. A method call to `basePrice` is extracted out and replaced with a newly created variable. However, the two logically related return-statements are separated with another edit. Again, the developer has to expand further cognitive effort to associate the two.

Original	Modified
<code>if(itr != list.end()) list.remove(itr);</code>	<code>if(itr != list.end()) { list.remove(itr); }</code>
Textual Difference	
<code>-if(itr != list.end())</code> <code>-if(itr != list.end()) {</code> <code>- list.remove(itr);</code> <code>-}</code> <code>-</code>	

Figure 1.1. Example change showing the deficiency of textual differencing approaches on source code. The top-left is the original source code. A { was added at the end of the first line, and a new line with a } was added at the end, resulting in the code on the top-right. The resulting textual difference (line difference) is shown on the bottom. A deleted line is shown with a preceding – and an insert with a preceding +. Background colors have been added to highlight the change with red for delete and green for insert. There are two problems with the line difference. First, the addition of the { is much smaller than a line, yet the whole line is marked as deleted and inserted. Secondly, the logical change is the addition of the block as a whole { } where several separate modifications are indicated here.

differencing tools have not been widely accepted by developers. More about syntactic differencing will be discussed in CHAPTER 4 along with the approach proposed in this dissertation.

Another approach is to save each keystroke and edit action of the developer(s) while the change is taking place, such as track-changes in a word processor. This information along with syntactic knowledge can produce a delta that reflects the actual syntactic change that occurred. However, this information is generally not available unless a specialized environment is used [Robbes, Lanza 2008]. Furthermore, this information is completely unavailable in the case of two non-consecutive versions of a file or in the case of multiple versions from different sources (multiple developers editing the same file).

The goal of the work presented here is to produce a more understandable and readable delta than what is produced by current line-based and syntactic-differencing approaches. My overarching hypothesis is that on its own, shortest edit script is a poor heuristic for syntactic differencing if the goal is to generate understandable deltas. To investigate this, we developed the tool srcDiff. It produces differences in the srcDiff format [Maletic, Collard 2004], which is an extension to the srcML (www.srcML.org) format [Collard, Decker, Maletic 2013]. srcDiff is a syntactic-differencing approach that works on the AST of source code as represented by srcML. Thus, it respects syntactic boundaries and supports both coarse and fine-grained changes. Unlike pure syntactic-differencing approaches, srcDiff does not use a standard tree-based differencing approach. Instead, it is a hybrid approach that utilizes more efficient sequence differencing, as provided by Myers' $O(ND)$ Shortest Edit Script algorithm [Myers 1986], to map changes onto the tree

structure. Thus, it combines the efficiency of line differencing with localized tree comparison.

In order to produce a more understandable delta, we use a set of empirically derived rules for selecting the most appropriate syntactic category (granularity) for a given change and to determine the scope of a set of changes within the code. The rules model how programmers make changes to code thus the resulting deltas better reflect their expectations in the context of changes.

To evaluate the approach, a comparison study against a state-of-the-art syntactic-differencing approach and two line-based differencing tools is conducted as an online within-participant study of about 70 subjects. The results give evidence that srcDiff produces more accurate and understandable deltas.

1.1 Research Questions

The study addresses the following research questions when comparing the different differencing approaches.

- Does srcDiff produce changes that are easier to read and more understandable than the other approaches?
- Is srcDiff preferred over the other approaches?
- Does srcDiff result in a delta that is more accurate than the other approaches?
- Is srcDiff scalable and practical to use in a software development environment?

In addition, this dissertation addresses the following research questions via application of the srcDiff differencing approach.

- How stable are a method’s stereotype?
- What consequences are there to a method’s stereotype changing?
- How commonly do developers replace and rewrite code?
- What are the consequences of replace and rewriting code?
- How often do developers convert code from one type to another?
- What are the consequence of converting code?

1.2 Research Contributions

To the best of the author’s knowledge, no work has presented a syntax-based differencer that completely preserves the underlying source code and thus can show changes to whitespace, comments, preprocessor, and code, works on syntactically incomplete and incorrect code, nor has any work been done for source-code differencing that uses rules based on the source-code syntax as presented here. In addition, to the best of the author’s knowledge, no previous work has done a full user study on viewing source-code changes. An exception is GumTree [Falleri et al. 2014], which performed a manual investigation with the authors and is related to the manual investigation used to develop srcDiff’s rules, and Dotzler and Philippsen [Dotzler, Philippsen 2016] that performed a questionnaire with few participants (eight participants) that are not all computer scientists and only using optimal tree-differencers. The following is a summary of the contributions:

- Development of differencing rules based on knowledge of how developers change code and perceive changes

- Integration of these rules into an efficient syntactic-differencing tool
- First user study on viewing and evaluating differencing approaches with a large number of participants
- Development of a differencing approach that produces more readable, understandable, and accurate deltas as compared to other approaches
- Investigation into the stability of method stereotypes and their consequences via a detailed analysis of changed methods
- Investigation into the prevalence of code conversion and code rewrite/replacement and the consequences

1.3 Organization

The dissertation is organized as follows. Background and related work is presented in CHAPTER 2. The infrastructure supporting the srcDiff approach is presented in CHAPTER 3. CHAPTER 4 introduces the srcDiff approach and describes the algorithm along with the rules we developed. The srcDiff tool is discussed in CHAPTER 5. A preliminary study is described in CHAPTER 6 and the evaluation is presented in CHAPTER 7. In CHAPTER 8 and CHAPTER 9, two applications of srcDiff to software engineering analysis tasks are presented. CHAPTER 8 investigates changes in method stereotypes and CHAPTER 9 investigates code replacement and code conversion. The conclusion of the dissertation is presented in CHAPTER 10.

CHAPTER 2

Background and Related Work

This section presents some basic terminology about differencing in relation to source code.

Differencing is the process of taking any two versions of code and computing the changes between them. For ease of explanation, we will discuss these in terms of successive versions. That is, there is an original version of the source code, and the source code is changed in some manner to produce a modified version.

The individual changes to source code are known as edits. Here, we define an edit as an indivisible operation that transforms the source code. There are two basic types:

- delete – an edit that removes something from the source code.
- insert – an edit that adds something to the source code.

Several more composite edit types also exist which can be described in terms of these basic edit types.

- substitution – an edit where source code in the original is replaced with code in the modified. That is, source code is deleted (a delete edit) in one location, and source-code is inserted (an insert edit) at the same location.
- move – an edit where source code is moved from one location to another. That is, source code in the original version is deleted (delete edit) and inserted (insert edit) at a different location.

In addition, there is a concept of a change or replace. A change or replace is one or more delete edits and one or more insert edits at the same location. This is very similar to the description of a substitution edit, however, a change is not an edit, but a description about a collection of edits. That is, an algorithm may compute substitution edits, while another that does not, reports the same operation as a delete edit and an insert edit (i.e., a change/replace). In this document, we will use “substitution” when we mean the edit version, while we will use replace or change to mean the other.

Depending on how the source code is represented, a single edit of code will take on different forms. That is, when textual line differencing is applied, source code is treated as a sequence of lines and a single edit would be to one line of source code such as one deleted line or one inserted line. When syntactical differencing is performed, the source code is represented as a tree, and the edits are to nodes of the tree such as a deleted node or inserted node. We now talk about textual, syntactical, and other types of differencing and the relevant research on each. We will then discuss topics related to source-code differencing and the other topics used as part of this dissertation. This includes: version-control systems, software merging, and source-code representations.

2.1 Textual Differencing

In textual differencing, the code being differenced is represented as a sequence (e.g., lines, words, or characters). Computing the difference between the original and modified is calculated by finding the minimum number of edits to transform the one sequence into the other (i.e., shortest edit script). There are several different algorithms for computing the shortest edit script, with variations performing better under various

conditions. For example, one of the most popular algorithms is Myers' algorithm [Myers 1986] which can compute the shortest edit script in $O(ND)$ time, where N is the sum of the lengths of both the original and modified sequences and D is the shortest number of edits. Thus, it is highly efficient (approaches a linear algorithm) the more similar the two sequences. This is ideal for source code, as much of the code remains unchanged between versions. The normal algorithm utilizes $O(N^2)$ space, but applying a Hirschberg linear refinement can turn it into $O(N)$ space [Myers 1986]. The popular GNU diff utilizes Myers' algorithm with the Hirschberg linear refinement in combination with heuristics [Free Software Foundation 2010]. Figure 2.1 shows two sample outputs of GNU diff on the function `setImage` (simplified to fit) from KOffice (revisions 1026809-1026810). The original version of the function (top-left) and modified version of the function (top-right) are also provided. The middle is a side-by-side view of the changes between the two functions. For GNU diff's side-by-side view, the original and modified function are placed next to each other and the changes are indicated via characters in the center. A `<` means the line on the left was deleted (none in example), and `>` means the line on the right was inserted. A `|` means the line on the left changed to the line on the right (i.e., the line on the left was deleted and the line on the right was inserted). On the bottom is a GNU diff's unified view. In a unified view, the original and modified functions are merged and shown together. Deleted lines are preceded by a `-` and insert edits are preceded by a `+`.

Original	Modified
<pre>void Settings::setImage(KisImageSP image) { m_widget->m_filter->setImage(image); }</pre>	<pre>void Settings::setImage(KisImageSP image) { if (m_options) { m_options->m_filter->setImage(image); } }</pre>
GNU diff - Side-by-side	
<pre>void Settings::setImage(KisImageSP image) { m_widget->m_filter->setImage(image); }</pre>	<pre>void Settings::setImage(KisImageSP image) { if (m_options) { > m_options->m_filter->setImage(image); > } }</pre>
GNU diff - Unified	
<pre>-- old.cpp 2017-04-15 12:15:21.000000000 -0400 +++ new.cpp 2017-04-15 12:15:42.000000000 -0400 @@ -1,3 +1,5 @@ void Settings::setImage(KisImageSP image) { - m_widget->m_filter->setImage(image); + if (m_options) { + m_options->m_filter->setImage(image); + } }</pre>	

Figure 2.1. The output of GNU diff on a method `setImage` (simplified to fit) from KOffice revisions 1026809-1026810. On the top is the original and modified function. The middle is the side-by-side view, and the bottom is the unified view.

Additional shortest edit script algorithms include the following. Wagner and Fischer provides a $O(N^2)$ time and space algorithm [Wagner, Fischer 1974] and a subsequent refinement by Hirschberg solved the problem with linear space [Hirschberg 1975]. An approach by Masek and Paterson allows for a $O(N^2 \log \log N / \log N)$ approach for arbitrary alphabets and a $O(N^2 / \log N)$ for finite alphabets [Masek, Paterson 1980]. Let L be the length of the longest common subsequence between two sequences, then Hirschberg provided two additional algorithms taking $O(NL + N \lg N)$ and $O(DL \log N)$ respectively [Hirschberg 1977]. Finally, if we let R be the total number of ordered pairs of positions for which the two input sequence match, then, an algorithm by Hunt and Szymanski has a time complexity of $O((R + N) \log N)$ [Hunt, Szymanski 1977]. As mentioned in the previous paragraph, since code is typically modified in small increments (D is small), Myers' algorithm will have a better run-time performance.

Ldiff is an extension to GNU diff that adds enhancements to identify fragment changes and moves. Its output is similar to GNU diff [Canfora, Cerulo, Penta 2009]. Both GNU diff and Ldiff can be used in conjunction with a diff viewer such as TkDiff [2011], which will highlight within-line changes. spiff produces results similar to GNU diff, but highlights tokens within a line that changed. Google-diff-match-patch computes changes on a character basis and colors the changed tokens. As all these tools are textual, the differences they report do not always make sense syntactically. The approach proposed in this study is compared to several popular textual difference tools that support within line changes (GitHub’s differencing view [GitHub 2016] and Mergely [Peabody 2016]). The results show that the approach presented in this study outperforms textual differencing approaches along all investigated criteria.

2.2 Syntactic Differencing

Syntactical differencing approaches utilize the Abstract Syntax Tree (AST), a tree representation of code where nodes in the tree contain information about the syntax and structure of the language. The problem of computing the difference between two versions of source code is then finding the difference between the two trees. Typical, approaches compute the shortest edit script (smallest number of edits to nodes that will transform the one tree into the other) between the two trees. When considering arbitrary move edits (code was deleted in one location in the tree and inserted in another location), the problem is NP-Hard [Bille 2005]. Thus, approximation techniques are utilized. General algorithms for the tree-edit distance problem include the work of Chawathe et al. [Chawathe, Rajaraman, Garcia-Molina, Widom 1996], Zhang and Shasha [Zhang, Shasha 1989], and

Pawlik and Augsten [Pawlik, Augsten 2011]. Although, these algorithms are not directly used for syntactical differencing, they have been used as either the basis or as part of syntactic-differencing approaches.

In [Fluri, Wursch, Pinzger, Gall 2007], Fluri et al presents ChangeDistiller, a tree-differencer, which computes an approximate minimal edit script (via computing similarities) that is used to classify and report (output) the tree-edit operations according to their taxonomy [Fluri, Gall 2006]. The algorithm is based off of Chawathe et al.’s algorithm [Chawathe, Rajaraman, Garcia-Molina, Widom 1996]. ChangeDistiller is designed to work on a simplified tree where leaf nodes are statements. GumTree [Falleri et al. 2014] is another tree-differencing approach that approximates a shortest edit script. GumTree compared itself to ChangeDistiller both on the ChangeDistiller tree and a fuller-tree and was shown to produce a shorter edit script in most to nearly all cases. In addition, GumTree was compared to RTED (i.e., Pawlik and Augsten algorithm) [Pawlik, Augsten 2011], a tree-differencer that produces an optimal edit-distance when moves are not considered, and was found to produce a shorter edit script in a majority of the cases. Pawlik and Augsten algorithm [Pawlik, Augsten 2011] is also used internal to GumTree. GumTree supports several outputs including a human readable side-by-side view by overlaying changes onto the source-code. In the evaluation in this paper, we compare the presented approach (i.e., srcDiff) with GumTree, and show that the presented approach is superior on all investigated criteria. In [Dotzler, Philippsen 2016], GumTree, ChangeDistiller, and other differences are improved to produce even shorter edit scripts. They also present MTDIFF, which is based on ChangeDistiller, but has better move detection. The work of

Dotzler and Philippsen [Dotzler, Philippsen 2016] was not yet published at the time of our evaluation or original write up. A goal of this dissertation is to show that obtaining a shortest edit script fails as a good metric for computing the difference between two versions of source code. So, as their goal is to create an even shorter edit script, this would only exasperate the problems identified in this dissertation.

diff/TS [Hashimoto, Mori 2008] is a syntactic-differencing approach that like GumTree aims to approximate an optimal tree-edit distance. It utilizes Zhang and Shasha's algorithm [Zhang, Shasha 1989]. It is also one of the few syntactic-differencing approaches that provides a human-readable view. It does this by overlying edit operations onto the source-code text. As GumTree is more recent, and it compared itself to other well-known source-code differencers, GumTree is chosen. As diff/TS also approximates an optimal tree edit distance, the results of our study generalize.

JSync [Nguyen et al. 2012] is a clone management tool for evolving software that incorporates an approximate optimal tree edit algorithm of their own design and LTDIFF [Hunt, Tichy 2002] simulates tree differencing for use with their software merger. As part of both approaches (as well as a very early version of srcDiff [Maletic, Collard 2004]), they first use textual differencing and map/align the changes to text with an AST to come up with a tree difference. Both approaches could be used to report a human readable difference. However, this was not their original intent and since, the approaches are doing some form of approximation of optimal tree-differencing the results of this dissertation still apply.

Smart Differencer [Semantic-Designs 1995-2012], based on Baxter's Design Maintenance System (DMS) [Baxter, Pidgeon, Mehlich 2004], is a heavy-weight compiler-based approach to source-code parsing and syntactic differencing of source code which describes the changes in terms of abstract editing operations and gives the code fragments affected. It reports changes to preprocessor and comments, but not to whitespace. Smart Differencer is proprietary software.

The most recent version of srcDiff appeared in a Master's thesis [Decker 2012] and is similar only in name to [Maletic, Collard 2004]. Additionally, the rules presented here are a significant extension to the thesis.

2.3 Semantic Differencing

Semantic differencing is a form of source-code differencing that considers the semantics of source code and reports changes in terms of what changed in behavior. That is, source code may be modified textually and syntactically, however, the original code and modified code may be semantically equivalent. In such a case, a differencer supporting full semantic detection would report no change. In practice, however, semantic differencing is an undecidable problem [Berzins 1986; Jackson, Ladd 1994], so current semantic differencers are limited to some extent. The following are semantic-differencing approaches.

Dex is a syntax and semantic source code difference tool that uses graph differencing on Abstract Semantic Graphs (ASG) created utilizing patches between two systems. The output is a properties file describing the changes in a patch [Raghavan, Rohana, Podgurski, Augustine 2004]. JDiff is a diff tool for Java that understands object-

oriented features and outputs a pair of nodes sets and their status (modified/unchanged) [Apiwattanapong, Orso, Harold 2004; Apiwattanapong, Orso, Harrold 2007]. Due to what these tools provide as output, they were not directly comparable with srcDiff, and so were not included as part of the evaluation.

Semantic Diff performs a difference using the change of dependence relations between inputs and outputs of a procedure and results in a textual description [Jackson, Ladd 1994]. Semantic Diff presents changes in a separate manner than srcDiff, and can be used in conjunction with the approach in this dissertation

2.4 Track-changes Approaches

Omori and Maruyama [Omori, Maruyama 2008] propose an IDE that records edit operations as they occur. Similarly, SpyWare is a change-aware development toolset that captures changes as they occur and can provides many views of source-code change, including a session inspector that allows changes in a session to be replayed [Robbes, Lanza 2008]. Dias [Dias 2015] gives another example of an approach that listens and records developer actions taken in an IDE. The main difference is these approaches record changes as they are made, while srcDiff, is able to recover changes without having a specially designed development environment.

2.5 File Comparison Software

A multitude of file comparison software exists. A list of these can be found at [Wikipedia® 2017]. Examples include TkDiff [2011], Meld [Willadsen 2011-2012], diff

and diff 3 [Free Software Foundation 2010; Miller, Myers 1985], Kompare [Keel, Snyder 2013], and Eclipse (compare) [Foundation 2017].

2.6 Other Differencing Approaches

There is a considerable number of other differencing approaches that are less related to the work presented here. Since, srcDiff works on an XML (eXtensible Markup Language) [W3C 2008] representation of source-code, the most relevant are XML-differencing techniques [DeltaXML 2000-2016; IBM 1998; INRIA ; Wang, DeWitt, Cai 2003]. XML-differencing techniques are designed for general XML documents, that is, unlike srcDiff, they have no knowledge of the underlying source code or AST and its significance for differencing.

LSdiff infers systematic structural differences as logic rules [Loh, Kim 2010]. Since, this approach reports differences in a separate manner than what is presented here, it can be used in conjunction with a tool like srcDiff.

Additionally, there are separate bodies of work that difference non-source code such as UML [Zhenchang, Eleni 2005] and Verilog hardware descriptions [Duley, Spandikow, Kim 2012].

2.7 Version Control Systems

Tichy defines Software Configuration Management (SCM) as the discipline of managing the evolution of large and complex software systems [Tichy 1988]. An integral part of this is version control systems. In their book, Pro Git, Chacon and Straub define Version Control Systems (VCS) or version control as systems that records changes to a file

or set of files over time so that you can recall specific versions later [Chacon, Straub 2014]. Examples of Version Control Systems are: Git [Inc.], Subversion (SVN) [Foundation 2011], Mercurial, Microsoft's Team Foundation Server [Microsoft 2017], Perforce [Perforce 2017], and Concurrent Versions System (CVS) [mdb 2015]. They are integral to the software development process as they enable and support the collaboration of software developers on software projects. As part of VCS, differencing tools are usually included. For example, SVN incorporates a textual line diff, and Git includes several textual diffs: line, word, and a separate algorithm named "patient diff" [Cohen 2010]. In addition, VCS can incorporate differencers as part of their storage mechanism. That is, by storing deltas (changes) relative to a base version, a VCS can save a substantial amount of disk space [Hunt, Vo, Tichy 1998].

2.8 Software Merging

Highly related to the field of software differencing is software merging. For a more complete discussion on software merging than discussed here, see Mens [Mens 2002]. As software development is a significantly collaborate effort, developers work on separate bodies of code in parallel. Periodically, these parallel lines of development need to be merged into one. Software merging is the process of integrating together parallel lines of development of the same software. As the parallel versions are typically kept in the same Version Control Systems (VCS), they usually incorporate mechanisms for merging. In addition, merging software requires knowing what is the same and what is different between the two parallel versions. As such, software differencing is an integral part of

software merging. To simplify discussion, we will discuss software merging in relation to merging two parallel lines of development.

There are two types of merging techniques: two-way merging and three-way merging. In two-way merging, two files are merged by comparing the differences between the two parallel versions. In three-way merging, a third file, a common ancestor, is used to clarify edits (i.e., what was truly inserted/deleted). Due to the additional power of three-way merges, they are used almost exclusively [Mens 2002].

Analogous to differencing techniques, there are also textual, syntactic, and semantic merging techniques. As with differencing, textual merging treats the software as text (e.g., lines), syntactic merging incorporates elements of the syntax of the language, and semantic merging uses behavior elements as part of the merge. With the progression from simple textual merging to syntax merging through to semantic merging techniques, more conflicting edits between the parallel versions can be detected and less important conflicts such as to comments or whitespace can be ignored [Mens 2002].

GNU diff [Free Software Foundation 2010] provides support for textual two-way merging and GNU diff3 [Free Software Foundation 2010] provides support for three-way merging [Mens 2002]. In addition, VCS such as Git [Inc.], Subversion (SVN) [Foundation 2011], and Concurrent Versions System (CVS) [mdb 2015] provide or incorporate textual merge tools. Despite the simplicity of textual-merge techniques, according to a case study, they are able to automatically merge roughly 90% of files without conflict [Perry, Siy, Votta 2001]. The reason why textual merges fail in the 10% of cases is due to a lack of syntactic and semantic information [Mens 2002]. In addition, the study notes that in 1% of

all cases, the tools made an incorrect automatic merge, usually resulting in compiler errors [Perry, Siy, Votta 2001], which can be solved by adding syntactic information.

Syntactic merging takes the syntax into account, and are, therefore, more powerful. They either use Abstract Syntax Trees (AST) or graph-based representations [Mens 2002]. Grass's Cdiff is a two-way tree-based technique for C++ programs [Grass 1992]. Asklund [Asklund 1994], Westfechtel [Westfechtel 1991], and Wu [Wuu 1994] are examples of three-way tree-based approaches. The work of Mens [Mens 1999] and Rho and Wu [Rho, Wu 1998] use graph-based approaches.

Semantic merging adds semantic information to arrive at a better merge. Often, graph-based approaches are used, as an AST does not contain enough context information for a semantic merge [Mens 1999]. An exception to this is Westfechtel's approach [Westfechtel 1991] which augments the AST with some context information. Hunt and Tichy's approach (ELAM: Extensible Language-Aware Merging) [Hunt, Tichy 2002], although not quite a semantic merger, is a tree-based three-way merger that utilizes some semantic information. Mens [Mens 1999] is an example of a graph-based approach.

To avoid unexpected behavior in the merged result, more complex semantic-merging approaches add additional formalisms [Mens 1999]. Berzins uses denotational semantics [Berzins 1994] for merging, Horwitz et al. use both program slicing and program dependency graphs [Horwitz, Prins, Reps 1989], Yang et al. use what they call limited slicing and program-representation graphs [Yang, Horwitz, Reps 1992], and Binkley et al. generalize Horwitz et al.'s approach to languages with procedure calls with interprocedural slicing [Binkley, Horwitz, Reps 1995].

2.9 Source-code Representations

The work of this dissertation builds off of and adds to the srcML Infrastructure [Collard, Decker, Maletic 2011; Collard, Decker, Maletic 2013]. The most closely related work to srcML is JavaML [Badros 2000]. Although JavaML provides an alternative representation of Java source-code, it does not preserve the source code, a significant advantage to using srcML. Examples of other XML representation are GXL [Holt, Winter, Schürr 2000], CppML [Mammas, Kontogiannis 2000], Aterms [van den Brand, Sellink, Verhoef 1998], GCC-XML [Kitware 2002-2012] (which has been succeeded by CastXML [Kitware 2013-2015]), and Harmonia [Boshernitsan, Graham 2000]. In these, the AST is encoded in XML and not the actual source code.

2.10 Improving Upon the Literature

A deep review of the literature identified the following. Textual-differencing approaches are by far the most commonly used and available, but they have problems as they lack information about the syntax of the language. Semantic differencing is attractive as it can be used to ignore semantically equivalent changes, however, the approaches are computationally expensive, and the general problem is undecidable [Berzins 1986; Jackson, Ladd 1994]. Track-changes approaches are capable of capturing exactly what edit events occur, but their use is restrictive and they cannot be applied in a general case. Syntactic-differencing approaches, therefore, have the greatest potential.

As syntactic-differencing approaches include the program syntax and structure, the resulting edits can conform to that syntax and structure. However, current syntax-

differencing approaches have several shortcomings, which inhibit them from seeing widespread use.

- Rely on optimal tree-edit distance as metric
- Under-utilization of program syntax
- Discard comments, whitespace, and/or preprocessor statements
- Lack a human readable view of source-code

First, optimal tree-edit distance is used to compute the difference. That is, approaches aim at shortening the tree-edit script. This dissertation will show in CHAPTER 4 and in the evaluation (CHAPTER 7) that this metric is flawed. A shorter edit script does not lead to a better or more accurate difference.

Secondly, the syntax of the language is not used to the full extent. Nodes in the tree are labeled with the syntax (i.e., a node is an if-statement) and an algorithm to do tree differencing is applied. Rarely is the encoded syntax or semantics of the syntax used beyond the matching of labels by the algorithm.

Third, many approaches discard whitespace, comments, etc. in the tree and so are unable to produce the difference of these elements. Tied to this is the fourth shortcoming. Since, this whitespace, comments, etc. are not recorded, they cannot go from the AST to the original source-code to produce a human-readable difference. Those that do resort to other means such as recording character positions as part of the nodes when the AST is formed so that they can overlay changes onto the source code, which has limitations.

The approach presented in this dissertation, srcDiff, addresses these shortcomings by:

- Purposefully deviating from an optimal tree-edit distance to result in a better difference.
- Utilize a set of rules based on the program syntax and domain knowledge on how developer's change and perceive code change.
- Preserving whitespace, comments, and preprocessor statements.
- Providing human readable views of source-code

CHAPTER 3

srcML Infrastructure

The srcML Infrastructure is used extensively as a part of this dissertation. As such, the following is an in-depth discussion of the infrastructure. The srcML Infrastructure is a format and a set of tools and technologies to support the exploration, analysis, and manipulation of large software projects [Collard, Decker, Maletic 2011; Collard, Decker, Maletic 2013; Kagdi 2003]. That is, it supports the maintenance and evolution of real software. Figure 3.1 is a diagram (from srcML.org) of the srcML Infrastructure. The srcML Infrastructure is comprised of the following parts: srcML format, srcML, XML, Models, Tools, and Support. The srcML format is an XML (eXtensible Markup Language) representation of source-code that forms the foundation of the infrastructure. The tool `srcml` converts source code and software systems to and from the srcML format. XML technologies and additional models are used to develop tools supporting software evolution and maintenance (i.e. the exploration, analysis, and manipulation of source-code). Tying every together is the Support of a multi-university development team. For the remainder of this chapter, the parts of the srcML Infrastructure will be discussed in detail.

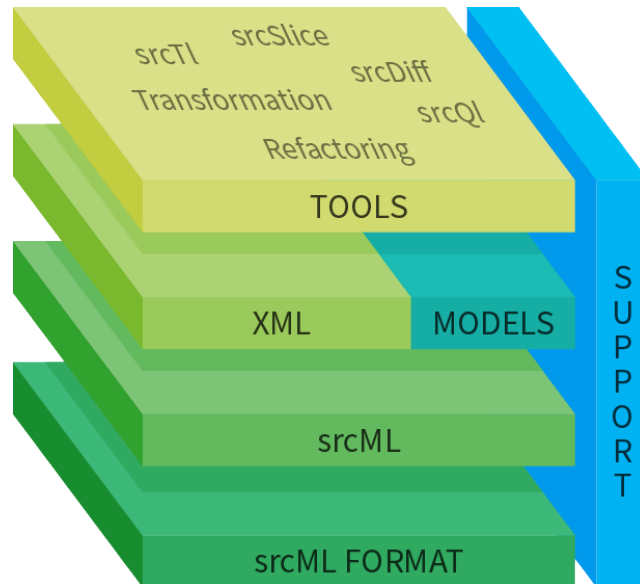


Figure 3.1. Diagram of the srcML Infrastructure. The foundation is the srcML Format. On top of this is the srcML tool, which converts source code to and from the srcML Format. From this core functionality, XML technologies and additional models can be leveraged to develop tools to support exploration, analysis, and manipulation of large software systems. The infrastructure is supported (right) via a multi-university development team.

3.1 srcML Format

The srcML format is an XML representation of source code. In the srcML format, all the original source code text (i.e., code, preprocessor statements, and whitespace) is marked up (i.e., wrapped) with XML tags (e.g., start tag: `<expr>`, end tag: `</expr>`) that represent information about the syntax and structure of the source code. This information is known as an Abstract Syntax Tree (AST). Figure 3.2 provides an example function (top) and its markup in the srcML format (bottom). For comparison between the source code and srcML format representations, the original text is highlighted in bold in the srcML format representation. At the top, is the root tag `unit`, which stands for compilation unit (i.e., file) and is common to all srcML documents. In addition to being

the root document, it also encodes metadata (as XML attributes) about the file and how it was created such as what version of srcML produced the file (`revision="0.9.5"`) and the input file's name (`filename="original.cpp"`). Internal to the `unit` tag information about the program syntax is marked up. For example, the function is marked with a `function` tag and the expression statement in the body of the function is wrapped with an `expr_stmt` tag.

A significant advantage to using the srcML format is that all the original source code is preserved. This is important because any resulting transformation of the code that is intended to be returned to a developer will be rejected unless the programmer's view (i.e., whitespace, comments, etc.) of the source code is maintained [Cordy 2003; Van De Vanter 2002]. Since the srcML format is lossless, removing all the XML tags, along with some unescaping, will reproduce the original source-code.

Source Code
<pre>void Settings::setImage(KisImageSP image) { m_widget->m_filter->setImage(image); }</pre>
srcML
<pre><unit xmlns="http://www.srcML.org/srcML/src" xmlns:cpp="http://www.srcML.org/srcML/cpp" \ revision="0.9.5" language="C++" filename="original.cpp"> <function><type><name>void</name></type> \ <name><name>Settings</name><operator>::</operator><name>setImage</name></name>\ <parameter_list>(<parameter><decl><type><name>KisImageSP</name></type> <name>image</name>\ </decl></parameter>)</parameter_list> <block>{ <expr_stmt><expr><call><name><name>m_widget</name><operator>-&gt;</operator>\ <name>m_filter</name><operator>-&gt;</operator><name>setImage</name></name><argument_list>(\ <argument><expr><name>image</name></expr></argument></argument_list></call></expr></expr_stmt> }</block></function></unit></pre>

Figure 3.2. Example source code function and its markup in srcML. Syntax information such as a section of code being a function or an expression statement is encoded by wrapping the appropriate source code with XML tags. The process is lossless. All the original text, including whitespace and comments is preserved.

Figure 3.2 shows a single file in the srcML format. The srcML format additionally supports the storage of entire software projects (i.e., a collection of files) in a single

document. This is known as a srcML archive. Figure 3.3 shows an example of a srcML archive. The example consists of two source code files: `sum.hpp` and `sum.cpp` in the srcML format. A separate `unit` tag is added around both file `unit` tags to comprise a srcML archive. Each file `unit` tag contains the original file's path and name (e.g., `filename="sum/sum.hpp"`).

srcML
<pre> <unit xmlns="http://www.srcML.org/srcML/src" revision="0.9.5"> <unit xmlns:cpp="http://www.srcML.org/srcML/cpp" revision="0.9.5" language="C++" \ filename="sum/sum.hpp"> </unit> <unit xmlns:cpp="http://www.srcML.org/srcML/cpp" revision="0.9.5" language="C++" \ filename="sum/sum.cpp"> </unit> </unit> </pre>

Figure 3.3. Example srcML archive. A separate `unit` tag is added as a root tag around the file `unit` tags

3.2 srcML

The tool `srcml` handles the conversion of source code text into the srcML format and the conversion of code in the srcML format back into source-code text.

3.3 To srcML Format

`srcml` uses ANTLR (ANother Tool for Language Recognition) [ANTLR 2014] to parse source code and produce the srcML format. Currently, `srcml` supports the conversion of C, C++, C#, and Java into the srcML format. In addition, Objective-C is unofficially supported and a mechanism for plugging-in any arbitrary language grammar is under development. The conversion process is remarkably fast. On an iMac running El Capitan with 2.7 GHz Intel Core i5 processor and 8 GB DDR3 RAM, `srcml` can achieve

speeds of ~92KLOC¹ per sec. This means, the entire Calligra [Calligra] project (an Office suite) consisting of about 1,100KLOC can be converted into a srcML archive in less than 12.5 seconds. The remainder of this section gives examples on how to use `srcml` to convert source-code into the srcML format.

The following will take a source-code file `main.cpp` convert it into the srcML format and print it to the terminal. The tool uses language extensions to determine what language is contained in the file and how to parse the file. For example, `main.cpp` has the extension `cpp`, which corresponds to a C++ file.

```
srcml main.cpp
```

To save the output to a file the `-o` option is used. Here the source code file `main.cpp` is converted to the srcML format and written to the file `main.cpp.xml`.

```
srcml main.cpp -o main.cpp.xml
```

A srcML archive can be produced by giving a collection of files, a folder, or a compressed/archive file. This first example takes the two files `sum.hpp` and `sum.cpp`, converts each separately to the srcML format and writes them out into `sum.xml` as a srcML archive.

¹ KLOC – thousand lines of code.

```
srcml sum/sum.hpp sum/sum.cpp -o sum.xml
```

Here, a folder (and all its subfolders) containing source-code is traversed. Each source code file is converted into the srcML format and written out to the file `project.xml` as a srcML archive. Language extensions are used to determine what is a source-code file and how to parse each file. The folder may contain multiple languages.

```
srcml project/ -o project.xml
```

When a compressed and or archive file is passed, `srcml` uses `libarchive` [Kientzle 2003] to un-archive and uncompress the file, walk through all the files contained in the archive and convert each source-code file to the srcML format. In this example, the `tar` archive and `xz` compressed project is processed and all source code files written to `project.xml` as a srcML archive.

```
srcml project.tar.xz -o project.xml
```

3.4 From srcML Format

`srcml` uses the Simple API for XML (SAX) [SAX 2001] parser from `libxml2` [Veillard 2004b] to convert code in the srcML format back into the original source code, generating files and folders as necessary. The process is extremely efficient. On an iMac running El Capitan with 2.7 GHz Intel Core i5 processor and 8 GB DDR3 RAM, `srcml`

can achieve a conversion rate of over ~580KLOC per second when converting from the srcML format back to source-code. This means, the entire Calligra project in srcML, can be converted back to source including generating all the files and folders in under 2 seconds. The remainder of this section gives examples on how to use `srcml` to convert code in the srcML format back into the original source code.

The following takes the srcML format (non-archive) file `main.cpp.xml` strips out the XML tags and prints the source-code to the terminal.

```
srcml main.cpp.xml
```

The following also strips out the XML tags from the srcML format (non-archive) file `main.cpp.xml`, but uses the `-o` option to write the contents to the file `main.cpp`.

```
srcml main.cpp.xml -o main.cpp
```

The following used the `--to-dir` option to take a srcML archive and convert each file to the original source-code and write it out into the folder `project`. The original folder structure is reconstructed in the `project` folder via the `filename` attribute on the individual `unit` tags.

```
srcml --to-dir project/ project.xml
```

3.4.1 libsrcml

In addition to the `srcml` tool, the srcML Infrastructure provides the library `libsrcml`. `libsrcml` provides the core functionality such as recognition of programming language and conversion to/from the srcML format. The `srcml` tool is then a front-end (i.e, command-line interface) to the library `libsrcml`. The current implementation of `srcDiff` (CHAPTER 5) uses `libsrcml` directly to allow for file-by-file differencing and to avoid the overhead of running an additional application. The `srcml` tool and the commands to convert to the srcML format functioned as a template for the development of the `srcdiff` tool. Although, the `srcdiff` tool replaces the to-srcML format functionality with to-srcDiff format, the from-srcML format functionality of the `srcml` is still utilized as part of the `srcDiff` tool chain.

3.5 XML & Models

As the srcML format is an instance of XML [W3C 2008], any XML technology can be applied directly to srcML. For instance, several methods are typically used for the parsing and processing of XML documents such as: Document Object Model (DOM) [W3C 1997], Text Reader [Veillard 2004a], and Simple API for XML (SAX) [SAX 2001]. Searching and querying of XML document is supported, as an example, through XPath [W3C 1999a] and XQuery [W3C 2000], and the transformation of srcML is support, as an example, by XSLT (eXtensible Stylesheet Language Transformations) [W3C 1999b]. The current implementation of `srcDiff` uses the Text Reader approach from `libxml2` to read, and parse the srcML format so that the approach can be applied. In addition, SAX parsing

is used for the parsing of documents in the srcDiff format (such as for generating human-readable output).

The srcML format is extendable. There is no current DTD (document type definition) for the srcML format. That is, there is no specification that states what is or is not valid srcML format. As such, additional models and information can be embedded with or on top of the general srcML format. The srcDiff format is an example. It extends the srcML format with additional tags and attributes. This will be discussed in detail in CHAPTER 4.

3.6 Tools

The srcML Infrastructure is commonly used both in research, academia, and in industry. As such, a number of tools have been produced on top of and integrated into the infrastructure. The research undertaken for this dissertation utilized tools that are a part of the srcML infrastructure and produced other tools that are now incorporated into the infrastructure.

3.7 Support

The development of the srcML Infrastructure is supported via multiple developers at multiple universities. In addition, tutorials and other documentation are freely available at srcML.org. The multi-university developers also provide quick feedback to questions and other information pertaining to using and building on the srcML Infrastructure. This support is invaluable for the research and tool development of this dissertation.

CHAPTER 4

Syntactic Differencing via srcDiff

We developed an efficient syntactic differencing approach based on the srcML [Collard, Decker, Maletic 2013] infrastructure (www.srcML.org). This includes a syntactic-difference output format [Maletic, Collard 2004] based on srcML and differencing tool called srcDiff. The work of this chapter and CHAPTER 7 is accepted with revision to the Journal of Software: Evolution and Process [Decker, Collard, Volkert, Maletic 2017].

4.1 srcDiff Format

The srcDiff format [Maletic, Collard 2004] extends the srcML format with the addition of four XML tags (`diff:common`, `diff:delete`, `diff:insert`, and `diff:ws`) to contain original and modified source code (i.e., any two versions) marked as the delta or the set of changes to the original source code (base version). Figure 4.1 gives an example of the format, as produced by the srcDiff tool, showing both the original and modified source code (simplified) of the function `setImage` from KOffice revisions 1026809-1026810. The changes include the statement in the function `setImage` being wrapped with an if-statement and the data member `m_widget` being renamed to `m_options`. The srcDiff subsequently has a `diff:insert` tag around the if-statement and a `diff:common` tag around the contents. The text of the renamed identifier (marked in srcML with a `name` tag) has a `diff:delete` tag around the old text and a

`diff:insert` around the new text. In addition, the tags have an attribute type with the value `replace` to signify that the code was replaced (e.g., rename). If a modification is to an attribute of a `srcML` element, these values are versioned with `|`, as in the attribute `filename` on the `unit` tag. Deleted/inserted whitespace is marked with `diff:ws` for easy processing/analysis. The `srcDiff` format completely preserves the original and modified `srcML` and therefore completely preserves both the original and modified source code (e.g., code, whitespace, comments, preprocessor). That is, the `srcDiff` format is a multi-version single document format that allows both the original and modified `srcML`/source-code to be extracted. As such, `srcDiff` supports both source-code change analysis, as well as, an efficient means of producing human-readable deltas. Examples of the human-readable format for the changes in Figure 4.1 are in Figure 4.2. The top is known as a side-by-side view, where the original code is presented on the left and modified code on the right. Changes are marked up individually on the original and modified code. For `srcDiff`, the original method is on the left with deleted code marked with a **red** background and the modified method is on the right with inserted code marked with a **green** background. The bottom is a unified view. In a unified view, the original and modified code are merged together and the changes marked on the single merged code. For `srcDiff`, the original and modified method are merged into one with deleted code marked with a **red** background and inserted code marked with a **green** background. As a note, since other syntactic-differencing approaches do not maintain/preserve whitespace, comments and preprocessor statements, they cannot easily (if at all) produce a unified view.

Other syntactic-differencing approaches achieve a side-by-side view by overlaying change information on the text files [Falleri et al. 2014; Hashimoto, Mori 2008].

Typically, syntactic-differencing methods support additional edits (e.g., update node value, move, etc.). Because srcDiff marks up text directly (e.g., renamed identifier in Figure 4.1), it does not need a separate edit for an update. In srcDiff, moves are marked as a delete (moved from) and insert (moved to) tags with an attribute move and a unique identifier.

The srcDiff tool is designed to efficiently produce the srcDiff format from any two versions of a source-code document, i.e., two files, directories, or repository versions (Subversion or Git). In contrast to other syntactic differencers, the code does not need to be syntactically complete, and changes to whitespace and comments are marked up. srcDiff also attempts to produce results on syntactically incorrect code.

Original	Modified
<pre>void Settings::setImage(KisImageSP image) { m_widget->m_filter->setImage(image); }</pre>	<pre>void Settings::setImage(KisImageSP image) { if (m_options) { m_options->m_filter->setImage(image); } }</pre>
srcDiff	
<pre><unit xmlns="http://www.srcML.org/srcML/src" xmlns:cpp="http://www.srcML.org/srcML/cpp" xmlns:diff="http://www.srcML.org/srcDiff" revision="0.9.5" language="C++" filename="original.cpp modified.cpp"> <function><type><name>void</name></type> \ <name><name>Settings</name><operator>::</operator><name>setImage</name></name><parameter_list>(\ <parameter><decl><type><name>KisImageSP</name></type> <name>image</name></decl></parameter>)</parameter_list> <block>{ <diff:insert><diff:ws> </diff:ws><diff:ws><diff:ws> </diff:ws><condition>(<expr><name>m_options</name></expr>) \ </condition><then><diff:ws> </diff:ws><block>{<diff:ws> </diff:ws><diff:common> <expr_stmt><expr><call><name><diff:delete type="replace">m_widget</diff:delete> \ <diff:insert type="replace">m_options</diff:insert></name><operator>-&gt;</operator><name>m_filter</name><operator> \ </operator><name>setImage</name></name><argument_list>(<argument><expr><name>image</name></expr></argument>) \ </argument_list></call></expr>;</expr_stmt> </diff:common><diff:ws> </diff:ws></block></then></if><diff:ws> </diff:ws></diff:insert>}</block></function></unit></pre>	

Figure 4.1. Example source-code change. The top-left contains the original source code and the top-right contains the modified source code. The original and modified code contained within the srcDiff format is given. Deletions are highlighted in red and with a line-through mark. Insertions are highlighted with green. All the original source code is placed in bold.

The tool is very scalable as it handles 1,000 commits/versions (all changed files) in under 5 minutes (running on a standard desktop computer).

Side-by-side View	
Original	Modified
<pre>void Settings::setImage(KisImageSP image) { m_widget->m_filter->setImage(image); }</pre>	<pre>void Settings::setImage(KisImageSP image) { if (m_options) { m_options->m_filter->setImage(image); } }</pre>
Unified View	
<pre>void Settings::setImage(KisImageSP image) { if (m_options) { m_widget m_options->m_filter->setImage(image); } }</pre>	

Figure 4.2. An example of the changed methods from Figure 4.1 in two different human readable formats produced by srcDiff. The top is a side-by-side view of the change, where the original method is on the left with deleted code marked with a **red** background and the modified method is on the right with inserted code marked with a **green** background. The bottom is a unified view where both methods are merged into one with deleted code marked with a **red** background and inserted code marked with a **green** background.

4.2 srcDiff Algorithm

At a high level, srcDiff does a preorder traversal on both the original and modified Abstract Syntax Trees (AST), in srcML, to iteratively refine differences until the desired granularity is reached. A preorder traversal is required so that the resulting difference, in srcDiff format, completely preserves the original and modified code. Figure 4.3 presents the differencing algorithm for srcDiff. The algorithm applies a sequence-differencing algorithm (i.e., Myers' [Myers 1986]) to the original and modified children (including subtrees) of a node (see line 2). The result of the sequence differencing is a list of edits

where each edit is one or more consecutive children of the same edit type. The edit types are:

- *common* - children identical in both versions,
- *delete* - children removed from the original AST,
- *insert* - children inserted into the modified AST, and
- *change* - children from the original AST replaced with children from the modified AST².

Common and unique children are output directly (lines 5-9), however, changed children need to be further analyzed to determine which actions to take (lines 10-26). There are three possible situations:

- *Match* - the child in the modified is a new version of the original;
- *Nested* - one child and its subtree in one version subsumes children in the other version;
- *Deleted* and/or *Inserted* - a child is unique to one of the versions.

Figure 4.4 illustrates how srcDiff uses sequential differencing. Part a) gives the original and modified versions of a declaration statement, as well as, their tree representations. For the tree representation, only the declaration statement subtree is shown. That is, the root is the `decl_stmt` and the declaration statement's children are:

² A change can also be described as a list of deleted children and a list of inserted children that occur at the same relative location.

type (`type`) and its subtree, identifier name (`name`) and its subtree, and initialization (`init`) and its subtree. The original declaration statement (source code) was modified in two ways: the addition of the `extern` keyword and the replace of the numeric literal 42 with the constant `ANSWER`. As such, the tree representation is modified in two ways: a new child of `decl_stmt` (`specifier` and its subtree) is added and the `literal` (descendent of `init`) is replaced with the constant `ANSWER` (i.e., `name`). `srcDiff` takes the children of a common node (`decl_stmt`), treats each as a sequence, as shown in Figure 4.4 part b), and runs a sequential-diff algorithm (i.e., Myers' [Myers 1986]). The result of this for Figure 4.4 is part c). The `specifier` and its subtree is reported as an insert edit. In addition, the `init` (and its subtree) is reported as a delete edit in the original tree and an insert edit in the modified tree. As they occur in the same relative position in the sequence, the `init` insert and delete edits get aggregated into a change edit. The remaining children are then denoted as common. The insert of `specifier` and common children require no further action by the `srcDiff` algorithm. However, the changed children (i.e., `init`), are analyzed further to determine if they were *Inserted/Deleted* (i.e., original `init` was deleted and the modified `init` inserted), *Nested* (one `init` belongs inside the other), or a *Match* (i.e., deleted `init` is a previous version of the inserted).

The determination of which actions to apply are as follows. First, the changed children are checked to determine if any of the children can be matched (i.e., using the function `determineMatchings`). If so, then finer-grained differencing is performed via a recursive call to the `srcDiff` algorithm on their children. The `determineMatchings` process uses a dynamic-programming approach that minimizes

the number of unmatched children while utilizing a similarity metric as a tie breaker. What can be matched is determined via a set of *differencing rules* (discussed in detail in the following section). The result of `determineMatchings` is a sequence of items that is either consecutive original and modified children with no corresponding matches (i.e., unmatched) or a pair of matched children.

Next, unmatched children are checked (via `determineNestings` at line 16) to determine if any of the children in the original or modified AST can be nested within the other. The process works by iteratively examining each original child to determine if the modified children are nested within. This process is duplicated for the modified children to see if any of the original children are nested within. Similar to `determineMatchings`, a set of *differencing rules* is used to define the nestings. If only the original or modified has a nested child, that nesting is chosen. If both situations produce a possible nesting and the nestings have no common children, the nesting with the first occurring children is selected. When the original and/or modified nestings share children, the nesting that covers the largest number of children is selected. This process continues until no new nesting is found. The result of `determineNestings` is a list that partitions all the children into groups of consecutive un-nested children or children pertaining to a nesting. For each nesting, the deleted/inserted root is output and the `srcDiff` algorithm recursively called on the children.

In the case that a child can be both matched and nested, `determineMatchings` uses the *differencing rules* to determine if nesting produces a better result. If so, the match is rejected and the children will be left as unmatched for `determineNestings`. In

addition, `determineNestings` is greedy, selecting the first possible child that can be nested. In practice, we found this to work well, except in the case when there is a single child deleted or inserted and nothing can be nested into it, but it can be nested. In this case, `srcDiff` does an exhaustive search to determine the best matching.

Lastly, is the situation of a delete or insert; this can be output directly (see line 22). As a note, `srcDiff` provides limited support for moves (see line 3). Children of the original that are identified as deleted or changed by the sequence differencer are scanned to see if there is an exact match among the modified children identified as inserted or changed. If found, the children are marked as moves and ignored by *`determineMatchings`* and *`determineNestings`*. Further support for moves can be added by analyzing the resulting `srcDiff`, however we leave this for future work. Next, we turn to discussing the differencing rules.


```

1  srcDiff(original, modified)
2      ListOfEdits = computeShortestEditScript(original, modified)
3      markMoves(ListOfEdits, original, modified)

4      FOR EACH edit IN ListOfEdits DO
5          IF common THEN
6              OUTPUT children as common, also merging whitespace
7          ELSEIF deleted OR inserted THEN
8              OUTPUT deleted or inserted children marked appropriately
9          ELSE //a change
10             matches = determineMatchings(edit.original, edit.modified)
11             FOR EACH i IN matches DO
12                 IF i is matched THEN
13                     OUTPUT i.root
14                     srcDiff(i.originalChildren, i.modifiedChildren)
15                 ELSE //Apply nesting rules to a sequence of unmatched children
16                     nestings = determineNesting(i.originalChildren,
17                                                i.modifiedChildren)
18                     FOR EACH j IN nestings DO
19                         IF j is nested THEN
20                             OUTPUT deleted or inserted root marked appropriately
21                             srcDiff(j.originalChildren, j.modifiedChildren)
22                         ELSE
23                             OUTPUT deleted and inserted children marked appropriately
24                         END IF
25                     END FOR
26                 END IF
27             END FOR
28         END FOR
29     END

```

Figure 4.3. The srcDiff algorithm. A preorder traversal is performed on both the original and modified trees. The input is the original and modified children of a node. A sequence-differencing method is used to categorize the children as common, unique to one version, or changed. Common, inserted, and deleted children are output directly. Changed children are further analyzed in `determineMatchings` and `determineNestings` for the following situations: a child in the original is a previous version of a child in the modified, a child and its subtree in one version subsumes children in the other version, and a child is unique to one version. The functions `determineMatchings` and `determineNestings` use a set of rules to identify these situations

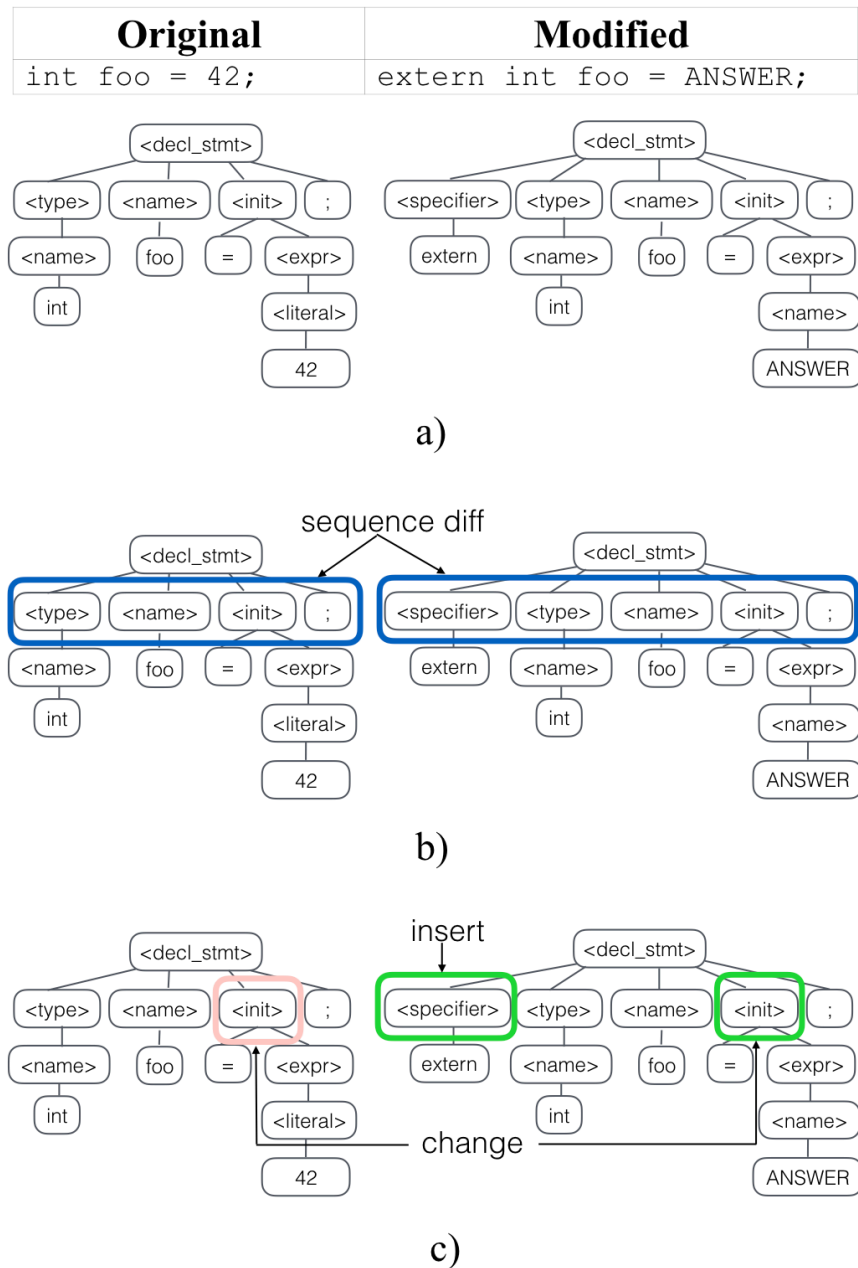


Figure 4.4. Example of the srcDiff process. a) The original and modified version of a declaration statement with their tree representations below each. b) With `decl_stmt` as common root, run a sequence diff on their child sequences (**blue**). Compares are between entire child subtrees. c) Output of sequential diff. **red** is delete edit, **green** is insert edit, remaining are common. `<specifier>` subtree is an insert and `<init>`'s are a change. Common and insert/delete require no further action. The change needs to be analyzed to see if *Match*, *Nested*, or *Inserted/Deleted*.

4.3 srcDiff Rules

Pure syntactic tree differencing often produces results that are difficult to understand. Take the following example code from hadoop (revision: eee0d4563c62647cfaaed6605ee713aaf69add78):

Original	Modified
<pre>public static boolean isStoragePolicyXAttr(XAttr xattr) { return xattr != null && xattr.getNamespace() == XAttrNS && xattr.getName().equals(STORAGE_POLICY_XATTR_NAME); }</pre>	<pre>public static String getStoragePolicyXAttrPrefixedName() { return XAttrHelper.getPrefixedName(XAttrNS, \ STORAGE_POLICY_XATTR_NAME); }</pre>

Textually, there is only a small amount of similarity between the two versions. Clearly, the method in the original version is completely replaced with a new method in the modified version that is, a delete and insert. Using a traditional syntactic differencing approach, it is more optimal (i.e., shorter edit script) to match any two given constructs (e.g., methods, if-statements, declaration-statements, etc.) because of common AST nodes that are shared. However, this approach fails miserably in this type of example and results in something very difficult to understand. The following is a side-by-side difference produced by GumTree [Falleri et al. 2014] a current state-of-the-art syntactic differencing tool. Deletes are marked in red, inserts in green, moves in blue. Updates are given in gold with common text in grey and deleted/inserted text in bold.

Original	Modified
<pre>public static boolean isStoragePolicyXAttr(XAttr xattr) { return xattr != null && xattr.getNamespace() == XAttrNS && xattr.getName().equals(STORAGE_POLICY_XATTR_NAME); }</pre>	<pre>public static String getStoragePolicyXAttrPrefixedName() { return XAttrHelper.getPrefixedName(XAttrNS, STORAGE_POLICY_XATTR_NAME); }</pre>

A large number of meaningless matches between syntactic elements (e.g., `XAttr` and `String`) are made to achieve more optimal results (i.e., shorter edit script). While

entire method replacement (as in this example) may not be very common, replacement of statements (or parts of statements) is a very common type of change. Anytime there is a coincidental similarity, pure syntactic differencing approaches will generate an inappropriate result.

Hence, we take a different approach here and use a set of customized rules instead of a single heuristic such as edit-script distance. We defined a set of rules that determine when it is acceptable to consider whether the change is a modification or a complete replacement of an existing statement. We derived the rules by manual and statistical analysis of over 1,000 revisions/version of changes to several open-source projects along with our combined development experience. Constant values within the rules are derived through statistical analysis and fine-tuning (via trial and error). These values can be further adjusted based on preference or additional empirical studies. In addition to the empirical analysis, we tested srcDiff on a suite of over 500 examples (simple to very complex) that includes changes from various open-source systems.

The differencing rules employed by srcDiff are organized into three categories:

- *Match* rules - identifies when two syntactic elements of the same type (e.g., two if statements) can be matched. For example, a developer adds additional statements to the body of an if-statement, the *Match* rules report that the if-statement in the original version can be matched with the corresponding if-statement in the modified version.
- *Convertibility* rules - identifies when one syntactic element can be converted into another syntactic element of differing type. For example, a

developer changes a for-statement into a while-statement. Here, *Convertibility* rules report that the for-statement in the original version can be converted into the while-statement in the modified version.

- *Nesting* rules - identifies when a syntactic element can be placed within another. For example, a developer adds an if-statement as a guard around an existing statement. In this case, *Nesting* rules report that the existing statement in the original version can be nested within the if-statement in the modified version.

Each of these rule categories is now discussed separately and illustrated in the same manner as Figure 4.1 and Figure 4.2 (Unified View), with deleted text in red with a line-through (**deleted**), inserted text in a green/dark grey (**inserted**), and the original text left unchanged in normal font. The following is the result of srcDiff on the previous method example (as side-by-side diff).

Original	Modified
<pre>public static boolean isStoragePolicyXAttr(XAttr xattr) { return xattr != null && xattr.getNameSpace() == XAttrNS && xattr.getName().equals(STORAGE_POLICY_XATTR_NAME); }</pre>	<pre>public static String getStoragePolicyXAttrPrefixedName() { return XAttrHelper.getPrefixedName(XAttrNS, \ STORAGE_POLICY_XATTR_NAME); }</pre>
srcDiff	
<pre>public static boolean isStoragePolicyXAttr(XAttr xattr) { return xattr != null && xattr.getNameSpace() == XAttrNS && xattr.getName().equals(STORAGE_POLICY_XATTR_NAME); }</pre>	<pre>public static String getStoragePolicyXAttrPrefixedName() { return XAttrHelper.getPrefixedName(XAttrNS, \ STORAGE_POLICY_XATTR_NAME); }</pre>

The function `determineMatchings` from the `srcDiff` algorithm in Figure 4.3 utilizes the *Match* and *Convertibility* rules along with a pre-check belonging to the *Nesting* rules to determine if nesting a child is better than matching. The function `determineNestings` only utilizes the *Nesting* rules.

The differencing rules, as a whole, incorporate domain knowledge of how a developer determines what changed in a piece of code. These rules can be generalized to most any syntactic-differencing approach. The rules encode knowledge that is common across a number of programming languages. We originally developed the rules in the context of C++ but our underlying infrastructure, srcML, also supports Java. Only a small amount of effort (less than a day) was spent extending the rules to work for Java. In particular, we needed to support a number of Java specific constructs that are not present in C++ (e.g., static block)

4.3.1 Match Rules

Match rules are applied to two of the same syntactic elements, such as two declaration statements or two if-statements to determine if they are a match. More explicitly, the Match rules are used to determine if a) two elements are similar enough to break down into constituent parts and perform additional fine-grained differencing, or b) if it is better to produce a delta showing a replacement of the one element with the other. The goal here is to generate a delta that is easily understood by the developer. Currently, the Match rules are divided into five different types of actions as shown in Table 4.1.

The *Always Match* rule includes syntactic categories that will always be broken down, no matter how different. There are two reasons for this. The first is to break down small nodes, which typically contain only text (e.g., names, operators, etc.). In the following example, the only item changed between the original and modified code is the variable name. Instead of marking the entire identifier as deleted and a separate identifier inserted (i.e., the node and its text is deleted and a new node with separate text inserted),

the subparts of the name are marked as deleted and inserted (i.e., it is the same node, but the text is changed) to indicate a rename.

Original	Modified
KisPenWidget *_m_optionsWidget;	KisPenWidget *_m_options;
srcDiff	
KisPenWidget *_m_optionsWidget	m_options;

Table 4.1. Match rules and the associated syntactic categories (srcML elements).

Rule	Syntactic Category
<i>Always Match</i>	name (single identifier), type, then, condition, control, init, default, comment, block, private, protected, public, parameter_list, argument_list, member_init_list, argument, range, literal, operator, modifier
<i>Any Similarity Match</i>	expr, expr_stmt
<i>Name Match</i>	call, decl, decl_stmt, parameter, function, function_decl, constructor, constructor_decl, destructor, destructor_decl, class, struct, union, enum
<i>Condition Match</i>	while, switch, for
<i>If-Statement Match</i>	if

The second reason is when it does not make sense for a syntactic element to be repeated (e.g., an argument list). In the following example, there is no similarity between the original argument list and the modified argument list, besides the parenthesis. Thus, without the *Always Match* rule, the argument list will not be broken down, leaving the original argument list as deleted and the modified argument list as inserted. However, with this rule, the argument list is broken down and the inserted argument can be correctly marked.

Original	Modified
update ();	update (true);
srcDiff	
update (true);	

The *Any Similarity Match* rule applies to expressions and expression statements, and is similar to *Always Match*, except if the two have no similarity at all or there is more deleted/inserted text than similar text. In both situations, a match does not occur. Otherwise, any amount of similarity constitutes a match. The elements in this rule are ubiquitous and can have very complex contents, which makes it valuable in many contexts not to perform finer-grained differencing thus obtaining more understandable deltas. In the following, there is no matching text between the two expressions, so the entire statement can be marked deleted and inserted.

Original	Modified
a + b;	c - d;
srcDiff	
a + b; c - d;	

In this next example, the expressions have some similarity without excessive dissimilarity. This rule, followed by the use of *Always Match*, has the operator marked as inserted and the identifier renamed.

Original	Modified
a + b;	a + -d;
srcDiff	
a + bd;	

The rule *Name Match* applies to elements which contain a name (e.g., functions and variable declarations). For *Name Match*, a matching name between both constitutes a match no matter what other differences exist. In its simplest form, a class in the original and modified code with the same name is considered the same, even if the contents are massively different. The following example contains a number of textual changes between

the original and modified code, such that the inserted declaration could be considered a replacement if not for the matching variable name.

Original	Modified
<code>QTextDocument * m_document;</code>	<code>QTextDocumentPtr m_document;</code>
srcDiff	
<code>QTextDocumentQTextDocumentPtr* m_document;</code>	

A small extension is needed for method calls of an object, e.g., `str.size()` and the call to `setImage` in Figure 4.1. If the variable is a rename, then we still want a match to occur. `srcDiff` currently considers it a match if any of the names match.

The *Condition Match* rule applies to loop statements and the switch statement and produces a match if the conditional is the same. For instance, the following while-statement has a large number of changes to its block. However, as it still has the same condition, it is considered to be the same statement

Original	Modified
<code>while(args[0]) { ++args; }</code>	<code>while(args[0]) { if(strcmp(args[0], OUT_FLAG) == 0) { output_file = args[1]; ++args; } ++args; }</code>
srcDiff	
<code>while(args[0]) { if(strcmp(args[0], OUT_FLAG) == 0) { output_file = args[1]; ++args; } ++args; }</code>	

In the case of a for-statement, the initialization and increment are considered in addition to the condition (i.e., the whole control group). In the following, the contents of

the block are completely different. However, since the control group is the same, it is treated as the same for-statement. In contrast, if any part of the control group does not match, then it will not pass this rule.

Original	Modified
<pre>for(i=n.begin(); i != n.end(); ++i) { std::cout << i->treeID() << '\n'; }</pre>	<pre>for(i=n.begin(); i != n.end(); ++i) { int id = i->treeID(); if(id == treeID) return *i; }</pre>
srcDiff	
<pre>for(i=n.begin(); i != n.end(); ++i) { std::cout << i->treeID() << '\n'; int id = i->treeID(); if(id == treeID) return *i; }</pre>	

A special, and more complex, rule is applied to if statements, the *If-Statement Match*. The simplest situation is when both if-statements have identical then-clauses. In this case, they are considered the same if-statement. The following is an example:

Original	Modified
<pre>if (p >= count()) break;</pre>	<pre>if (already_exist(chain, p)) break;</pre>
srcDiff	
<pre>if (p >= count()already_exist(chain, p)) break;</pre>	

The conditions of the two if-statements have very little similarity and the amount of change between the statements is greater than the similarity. However, since the then-clauses are identical, it is considered the same if-statement.

The other rules associated with an if-statement are related to when both conditions are identical. First, the rule produces a match if either of the following is true about the original and modified if-statements: 1) both contain a block-statement ({ }) in the then-clause; or 2) both do not have a block-statement in the then-clause. In the following, both

if-statements have the same condition, but very different statements in the block. Despite this, they should be considered the same if-statement.

Original	Modified
if(name == null) { return false; }	if(name == null) { throw new IllegalArgumentException("..."); }
srcDiff	
if(name == null) { return false; throw new IllegalArgumentException("..."); }	

Second, the rule produces a match when any of the following is true about the original and modified if-statements: 1) both contain an else-clause; 2) both do not contain an else-clause; or 3) only one has a block-statement, while the other if-statement does not have an else-clause. This complexity is needed in cases such as when an if-statement is taken from around code and an else-clause added. The following two examples illustrate the condition/block/else situation. In the first example, the modified if-statement has a block and the other does not. The if-statement appears to be a previous version of the other, so matching is expected.

Original	Modified
if(!paraStyles.contains(root.ID())) tree_root << root.ID();	if(!paraStyles.contains(root.ID())) { int id = root.ID(); if(id != tree_rootID) tree_root.insert(id); }
srcDiff	
if(!paraStyles.contains(root.ID())) tree_root << root.ID(); { int id = root.ID(); if(id != tree_rootID) tree_root.insert(id); }	

In the next example, the original if-statement has a block and the other does not. Here, they do not match so the second modified if-statement can be placed in the original if-statement. The main difference is the addition of an else-clause. With the additional else-clause logic, the correct delta is achieved.

Original	Modified
<pre>if (shape.has("anchor")) { string type = shape.get("anchor"); if(type != "page") { // several lines common code } }</pre>	<pre>string type; if (shape.has("anchor")) type = shape.get("anchor"); else type = "as-char"; if(type != "page") { // several lines common code }</pre>
srcDiff	
<pre>if (shape.has("anchor")) { string type = shape.get("anchor"); if (shape.has("anchor")) type = shape.get("anchor"); else type = "as-char"; if(type != "page") { // several lines common code } }</pre>	

This rule is derived from examining the change history of existing projects and fails when the extracted if-statement has no else-clause. However, if an else-clause is not needed, then it does not need extraction.

For code that does not pass one of the rules above, there are two fallback rules applied in succession: *syntactic similarity* and *textual similarity*. These are also used as part of *Convertibility* and *Nesting* rules. Both rules use similarity and dissimilarity metrics computed using Myers' Shortest Edit Script Algorithm [Myers 1986]. Unlike Levenshtein distance [Levenshtein 1966], substitutions are not a single operation, but a delete and an

insert. We used Myers' algorithm as substitutions are not required by the metrics. These two metrics are defined as:

```
similarity = min(#original - #deletions, #modified - #inserts)
dissimilarity = #deletions + #inserts
```

where *#original* and *#modified* is the number of textual tokens in the case of *textual similarity*, and the number of child elements in the case of *syntactic similarity*. The *#deletions* and *#inserts* are the number of inserted and deleted lexical tokens/child elements.

Syntactic similarity is computed on the child elements of two candidate elements after stripping out immediate child literals, modifiers, operators, empty argument lists, and text tokens (e.g., the *if* keyword when applied to an *if*-statement). If similarity is greater or equal than half the minimum number of children between both sets and the amount of dissimilarity is not greater than this minimum, then the two candidate elements are a match. Depending on how the AST is formed, the syntactic-similarity rule can subsume the *Condition Match* rule (i.e., a *for*, *while*, or *switch* consist only of control group/condition and block statements). In these instances, the *Condition Match* rule may be considered a quick pre-check to avoid a costlier rule. Additionally, if both elements have a block as the final child or an *if*-statement with only a *then*-clause, and the previous check fails, the same syntactic check is also applied on children of the block.

The final fallback rule is *textual similarity* (Figure 4.5). With exception of rule 2 (Figure 4.5), it is computed on the textual leaf tokens after stripping out non-operator

punctuating characters such as (,), {, }, [,], :, ;, and ,, and operators/modifiers part of a name (except : :). Rule 2 uses measures without stripping out any tokens.

```

minSize = min(#originalTokens, #modifiedTokens)
maxSize = max(#originalTokens, #modifiedTokens)

1      if ((similarity == minSize) && (dissimilarity < 2*minSize)) return true;
2      if (similarity_no_strip == min_size_no_strip)                return true;
3      if (dissimilarity > maxSize)                                  return false;
4      if ((minSize < 30) && (dissimilarity > 1.25*minSize))          return false;
5      if ((minSize ≥ 30) && (dissimilarity > 1.05*minSize))          return false;
6      if (minSize ≤ 2)                                              return similarity ≥ ½* minSize;
7      else if (minSize ≤ 3)                                          return similarity ≥ 2/3* minSize;
8      else if (minSize ≤ 30)                                         return similarity ≥ 7/10*minSize;
9      else                                                           return similarity ≥ ½ * minSize;

```

Figure 4.5. Textual similarity rules. The rules use similarity, dissimilarity and size measures to test if two elements are similar enough textually. The measures, with exception to rule 2, are calculated after stripping out punctuation characters and other tokens that negatively affect performance. Rule 2 uses all tokens to test for an exact match. It is not applied to *Nesting* rules.

The first two if-statements (rules 1 and 2) handle complete matches (i.e. one child is a complete subset). The first applies to all the rules, while the second does not apply to *Nesting* rules. The 3rd - 5th rules encode that a potential match should not have an excessive amount of dissimilarity. The remaining rules constitute the main textual-similarity tests. When the list is two or three tokens, a majority of them need to be the same. Finally, it was identified in our empirical analysis of changes that a relatively small structure (e.g., <30 tokens) needs a larger amount of similarity to be considered a match, while a larger structure can be a simple majority. That is, an if-statement of two or three lines (smaller number of tokens) requires a closer match than an if-statement of say 50 lines (large number of tokens).

4.3.2 Convertibility Rules

Convertibility rules occurs between two syntactic elements of different types and determines if one of the elements can be converted to the other, such as converting a for-statement to a while-statement. When identified, srcDiff marks the elements with delete and insert tags each with attribute type and value convert and performs fine-grained differencing on their children via recursively applying the srcDiff algorithm on the children. There are currently five types as listed in Table 4.2.

Table 4.2. Convertibility Rules organized by type. Each type has the associated syntactic categories.

Rule	Syntactic Category
<i>Class</i>	class, struct, union, enum
<i>Access Regions</i>	private, protected, public
<i>Logical</i>	if, while, for
<i>Else</i>	else, elseif
<i>Statement</i>	expr_stmt, decl_stmt, return

All the convertible rules use the *Match* fallback rules as a check to see if one can be converted to the other. *Class*, *Logical*, and *Statement* have additional conditions that will result in passing. For the *Class* rule, a conversion can occur if they both have the same name. The *Logical* rule passes if both have the same condition (similar to the *Condition Match*, except only the condition of a for is used). The following is an example of a for-statement converted to a while-statement.

Original	Modified
<pre>int itemCount = d.items.count(); for(int i = itemCount-1; i >= 0; --i) { Item &sbItem = d.items[i]; if (sbItems.widget() == widget) { // several common lines } }</pre>	<pre>int itemCount = d.items.count() - 1; while(i >= 0) { Item &sbItem = d.items[i]; if (sbItems.widget() == widget) { // several common lines } --i; }</pre>
srcDiff	
<pre>int itemCount = d.items.count() - 1; forwhile(int i = itemCount-1; (i >= 0); -i) { Item &sbItem = d.items[i]; if (sbItems.widget() == widget) { // several common lines } --i; }</pre>	

For *Statement*, the top-level expression in each is used to determine if they can be converted, i.e., if the expressions can be matched (initialization in declaration-statement, returned expression in return-statement, and the complete expression in expression-statement). To be converted, they both must have expressions, the expressions must pass the *Match* fallback rules, text similarity must be greater than half the size of the largest expression, and text dissimilarity must be less than half the size of the largest expression. In the case of matching a declaration statement to one of the other, then the size of the expression trees must also not be greater than four times each other. The following is a simple, but real change example.

Original	Modified
bool ok = openUrl(url);	openUrl(url);
srcDiff	
bool ok = openUrl(url);	

4.3.3 Nesting Rules

Nesting rules occur between syntactic elements with the goal to determine if one or more of the elements can be safely nested (placed) within another. That is, mark the one element as deleted/inserted and recursively apply the srcDiff algorithm on the nest-able elements and that element's children to determine if they can be matched or further nesting is required. For example, an if-statement is deleted, however, the statements in the if-statement's body are left unchanged. The nesting rules determine that the statements in the modified document can be nested within the deleted if-statement where they can later be marked as unchanged.

The *Nesting* rules are applied to unmatched elements, i.e., those that failed *Match/Convertibility* rules. However, *Nesting* rules are also checked during the *Match/Convertibility* process to see if an element can and will create a better difference by nesting it into the potential match/conversion or a subsequent element. If so, the match/conversion is rejected to allow for the nesting. An element is better nested with its best internally matching element when it has greater or equal text similarity and less than or equal text dissimilarity, or it has a better ratio of minimal text node size to text similarity and passes a slightly modified version of *Match* rules. This is described in more detail in following paragraphs.

We constructed rules for syntactic elements (statement and sub-statement) that can contain nested elements. For example, an if-statement can have other if-statements, blocks, while loops, etc. Unmatched elements are checked to see if they meet the conditions for nesting. Once identified, additional checks are made. First, the best internally matching

element (same element, both no greater than four times the size of the other, and highest text similarity) is found. In the case of a node having descendants with the same matching node name, only the ancestor is taken (a subsequent nesting will nest further if required). If no item is found, then that item will not be nested. Then, they are checked using rules similar to the *Match* rules except, then-clause, block, comment, literal, operator, modifier, expression, expression-statement, and name are no longer *Always Matched* or *Any Similarity Match*, but use the fallback rules. The remaining details are the same as the *Match* rules.

If the elements have the same type (i.e., two if-statements), an additional check is made (both when determining if better to nest or if can be nested). The best internally matching element must have greater or equal text similarity and less than or equal amount of text dissimilarity (i.e., it is more optimal to nest), or the smallest element (in terms of textual nodes) must be sufficiently large (≥ 50), the ratio of minimum size of text nodes to text similarity must be less than 90% when nested as compared to not being nested, and they must pass the previously discussed modified Match rules for nesting. The second condition catches false negatives. The best nesting may have slightly less total similarity (due to other changes) and still produce a more meaningful difference. However, to reduce false positives, it must have a sufficiently better ratio and is bounded to work for larger structures (we picked ≥ 50 after tuning) where ratios are more stable. The following is an example with the ≥ 50 relaxed due to space restrictions. In this example, the parent if-statement was deleted and its child declaration statement moved into the child if-statement. Because of the move of the declaration statement there is less similarity between the child

if-statement in the original and the matching modified if-statement. However, since the original child if-statement is smaller the ratio is better by a notable margin.

Original	Modified
<pre> if(root.ID()) { node left = root.left; if(child.ID() < root.ID()) { this->insert(left, child); } } </pre>	<pre> if(!root.ID()) return; if(child.ID() < root.ID()) { node left = root.left; this->insert(left, child); } </pre>
srcDiff	
<pre> if(!root.ID()) return;if(root.ID()) { node left = root.left; if(child.ID() < root.ID()) { node left = root.left; this->insert(left, child); } } </pre>	

Next, checks are made to see if a better nesting is possible. These are (for nesting into a deleted item): in the reverse direction (deleted element within the inserted), the inserted element in the next deleted element, and deleted element inside the next inserted element. The checks are similar for nesting into an inserted element. Lastly, there are some special checks for identifiers. First, if the parent of both the identifier and what it may potentially be nested into are expressions, then there must only be one inserted element and deleted element to check for nesting. Secondly, the identifiers (one being nested and the potential match when nested) either must have parents with the same element type or an identifier in an expression is to be nested within a function call identifier i.e., `foo -> foo.bar()`. In the latter case, prefixes of the identifier sequence and call name identifier sequence have to be sufficiently similar (at least half the longest identifier sequence length).

The following is an example where the modified code introduced an if-statement that wraps the statement from the original. In this example, the delete-statement is iteratively nested through recursive application of the srcDiff algorithm. The initial application places the delete-statement in the if-statement, a second application places it in the then-clause, and a final recursive application placed the delete-statement in the block.

Original	Modified
delete m_document;	if (m_frames.isEmpty()) { delete m_document; }
srcDiff	
if (m_frames.isEmpty()) { delete m_document; }	

The following is an example of a nest where the element being nested and the element being nested into are the same elements. Here, the original if statement is removed and the statements in the block are kept. Subsequently, the statements in the modified code are nested into the if statement in the original.

Original	Modified
if (shape.has("anchor")) { string type = shape.get("anchor"); if(type != "page") { Anchor *anchor = new Anchor(shape); anchor->loadShape(element); } }	string type = shape.get("anchor"); if(type != "page") { Anchor *anchor = new Anchor(shape); anchor->loadShape(element); }
srcDiff	
if(shape.has("anchor")) { string type = shape.get("anchor"); if(type != "page") { Anchor *anchor = new Anchor(shape); anchor->loadShape(element); } }	

4.4 Complexity

The following present the worst-case time complexity of the srcDiff algorithm. For illustration of its practical purposes, a timing comparison is performed as part of the evaluation in CHAPTER 7. Let N be the maximum number of children for a node, D be the maximum number of individual edits (inserts and deletes³) between children (which is always less than or equal to $2N$), C be the cost of comparing two child nodes⁴, and R be the cost of the srcDiff rule computation. Myers' algorithm (Line 2) then runs in $2CND$ or $O(CND)$ time. In Line 3, the D edits are checked for identical matches of the opposite edit type using a hash map. In the worst case, when everything hashes to the same value and using child node comparison to disambiguate, this is $O(CD^2)$ (which is less than CND). Line 4 iterates through the groupings of consecutive edits, consecutive common children, and changes, so at most $O(N)$. Common, deleted, and inserted children are output (lines 5-8), which over all recursive calls to the srcDiff algorithm and in combination with lines 13, 19, and 22, results in the output of all nodes. Let M be the maximum number of nodes between both trees, so output takes at most $2M$ or $O(M)$. Line 10 uses a quadratic dynamic programming algorithm. At worst all D edits will belong to a single change or $O(RD^2)$ (in

³ For D , deletes and inserts comprising a change are counted and not the change itself.

⁴ As a practical note, a hash is computed on demand for each node set minimizing the cost of successive comparisons. This was added as a refinement after the evaluation for both comparisons and markMoves (Line 3).

this case Line 10 will be $O(1)$). Line 16 (at worst when there are no matches) checks if each node can be nested in the other for $O(RD^2)$ (this only occurs when it finds nothing that can be nested). Combined, these will result in at worst D recursive calls. Finally, let K be the maximum depth of the srcML tree. Lines 10 and 16 will result in $O(D^K)$ total recursive calls. So, the runtime complexity of the srcDiff algorithm is $O(D^K(CND + RD^2) + M)$. In practice, K , N , and D are not typically large numbers. In addition, recursion is stopped (recursive subtrees pruned) when children are determined to be common, deleted, or inserted. Since, a large part of the srcDiff rules rely on children being similar enough, excessively dissimilar children will not result in a recursive call and thus, expected runtime is much less.

For the srcDiff rules (i.e, R), textual similarity and syntactic similarity are the costliest as they both employ Myers' algorithm on the text nodes and child nodes, respectively. The two times these are used most heavily (as part of the rules) is when checks are made in `determineMatchings` to see if it would be better to nest a child in one of the subsequent others of the other edit type and when searching for the best (highest similarity) descendant of a certain type. In the first case, textual similarity and syntactic similarity might be applied $O(D)$ times. In the latter, textual similarity will be applied once to each found descendant. Of note, the srcDiff rules are not run on every child part of an edit. *Match* rules will never be applied to two elements of separate type, so, R is constant for those children. Similarly, *Convertibility* rules and *Nesting* rules first check if an element can be converted to or nested in the other through simple lookups.

In practice, from various experiments on applying the approach to the complete history of software systems, the bottleneck is primarily textual similarity (and other Myer's shortest edit script usages). The only instance this was a problem was in the case of extremely large auto-generated files. For this purpose, approximation was added in place of Myer's algorithm when one of the sequences is greater than 2048.

CHAPTER 5

srcDiff Tool

In this chapter, we discuss the current implementation of the srcDiff tool as well as the interface for using the tool. The tool is called `srcdiff`.

5.1 Implementation

`srcdiff` is implemented in C++ and uses CMake [Kitware] as a build system. In total, `srcdiff` consists of over 16 KLOC distributed among the following main modules: `client`, `translator`, `srcml`, `shortest_edit_script`, and `view`. Figure 5.1 shows a simplified high-level UML Class diagram of the modules.

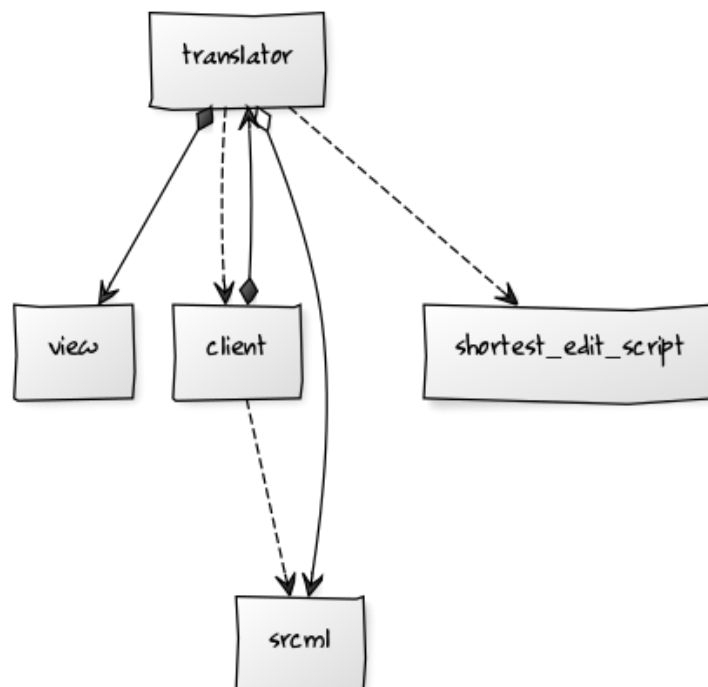


Figure 5.1. A simplified high-level view of modules in `srcdiff`.

The `client` module is responsible for handling user input (command-line interface) and managing and supporting various input sources. Currently, local filesystem, Git repositories, and limited support for Subversion repositories are supported as input sources. The parsing of the command-line arguments is handled through `boost`'s [Dawes, Abrahams, Rivera 1998-2007] `program_options` library [Prus 2002-2004]. As part of developing `srcdiff`, a fault was found and a patch submitted to `boost`'s `program_options` library where it was eventually included as part of the 1.59.0 release of `boost`. Once the options are parsed, the input source is processed and sequences of input file pairs are passed to the `translator` module.

The `translator` module is responsible for implementing the `srcDiff` approach and rules. That is, it computes the difference between the two supplied inputs. Once the `translator` module is invoked, each of the inputs convert themselves to `srcML` via the `srcml` module. The conversion of both original and modified input can be done in parallel via threads⁵. In addition to providing conversion of input to `srcML`, the `srcml` module is also responsible for traversing the `srcML` and generating the format used internally by `srcdiff`. The `srcml` module converts the input to `srcML` using `libsrcml` (CHAPTER 3). The `srcML` generated by `libsrcml` is then processed using `libxml2`'s Text Reader interface [Veillard 2004a] into a sequence of `srcML` nodes, one for each input.

⁵ Currently, some part of `srcdiff` or `libsrcml` is not thread safe. Once fixed, `srcdiff` can support `srcML` conversion in parallel with threads.

The `translator` module forms node sets (child sub-trees) on demand from these sequences. For example, each child of root is converted into a node set, which consists of all the nodes between the opening and closing srcML tags for that child. When required, such as when recursively calling the `srcDiff` algorithm on two matched children, node sets for those children's children are created. The current implementation of the node sets does not copy any srcML nodes, but consists of a list of position offsets into the srcML node sequences.

After the srcML nodes are generated and the first node sets created, `translator` module starts the `srcDiff` algorithm (Figure 4.3). The first part of the `srcDiff` algorithm, `computeShortestEditScript` (the execution of Myers' shortest edit script algorithm [Myers 1986]), is implemented via the `shortest_edit_script` module. The `shortest_edit_script` module implements a custom implementation of Myers' shortest edit script algorithm.

In [Myers 1986], Myers' gives several versions of the same algorithm supporting separate tradeoffs. The `shortest_edit_script` module implements two of these: the normal $O(ND)$ space complexity and the linear-space refinement version. The $O(ND)$ version executes more quickly, while the linear-space version scales to larger sequences. The `shortest_edit_script` module combines the two into a hybrid version. The linear-space version works by repeated execution with each execution identifying a middle that is always part of the final solution. That is, it splits the sequences into two subparts with each execution. The hybrid version selects the linear-space version for large sequences and switches to the $O(ND)$ version when the size becomes small enough. The

first call to the hybrid version uses a best guess based on the length of the sequences to decide which version to run, however, each call to the linear-space version reports the total-edit distance which can be used to accurately gauge the correct point to switch to the $O(ND)$ version.

Once the shortest-edit script is computed, the `translator` module applies the remainder of the `srcDiff` algorithm. During the execution of the `srcDiff`, once a node has been determined to be common, inserted, or deleted, it is output. That is, the final `srcDiff` is output incrementally through the execution of the algorithm. This is accomplished directly via `libsrcml`, which provides support for the incremental creation of a `srcML` (in this case `srcDiff`) document in memory. Once the `srcDiff` algorithm is finished, the complete `srcDiff` for the input pair is contained in memory.

Once the `srcDiff` for an input pair is created, the `srcDiff` is either appended to the `srcDiff` archive being created (when the desired output is the `srcDiff` format) or the `view` module is invoked to transform the `srcDiff` output into the desired view such as the human-readable unified or side-by-side views. After the evaluations were completed, the `view` module was extended to provide customizable output themes and various levels of syntax highlighting. Figure 5.2 shows sample output of the `view` module on the source-code from Figure 4.1 and Figure 4.2 (the source code is also reproduced in the top of Figure 5.2). The first view is `srcdiff`'s default theme and default level of syntax highlighting. The second view is `srcdiff`'s Monokai theme (based off of Sublime Text's [Ltd 2016] Monokai theme). Both of these outputs are produced as HTML by `srcDiff` and rendered

via a browser. The default theme is also supported as output to the terminal. In addition, the `view` module supports user-defined themes.

Original	Modified
<pre>void Settings::setImage(KisImageSP image) { m_widget->m_filter->setImage(image); }</pre>	<pre>void Settings::setImage(KisImageSP image) { if (m_options) { m_options->m_filter->setImage(image); } }</pre>
Default Theme and Default Syntax Highlighting	
<pre>@@ -1 +1 @@ void Settings::setImage(KisImageSP image) { if (m_options) { m_widgetm_options->m_filter->setImage(image); } }</pre>	
Monokai Theme and Full Syntax Highlighting	
<pre>@@ -1 +1 @@ void Settings::setImage(KisImageSP image) { if (m_options) { m_widgetm_options->m_filter->setImage(image); } }</pre>	

Figure 5.2. Examples of `srcDiff` output provided by the `view` module. Top shows the original and modified source code, middle is the default theme and default level of syntax highlighting. Bottom is the Monokai Theme with full syntax highlighting. Both outputs are taken from `srcDiff`'s `view` module to HTML.

After the view is generated, the process is repeated for each input pair contained in the input sources.

5.2 Interface

`srcdiff` is a command-line interface based off of `srcml`'s command-line interface. As such it accepts many of the same arguments. One of the main differences is

that it requires sequences of file pairs (or directory pairs) with one being the original file (or directory) and the other being the modified file (or directory). The following is the most basic way to run `srcdiff`. It takes the two files: `original.cpp` and `modified.cpp` and produces output to the terminal in the `srcDiff` format.

```
srcdiff original.cpp modified.cpp
```

The following does the same, except it places the output into the file `srcdiff.xml`.

```
srcdiff original.cpp modified.cpp -o srcdiff.xml
```

It can also take two directories and produces a `srcDiff` archive containing the differences of all the files in those directories. As a note, this requires matching names and directory structures to work as expected. Here all the code in the directories `original` and `modified` is differenced and placed in the `srcDiff` archive `srcdiff.xml`.

```
srcdiff original/ modified/ -o srcdiff.xml
```

`srcdiff` can also take a file containing a list of files to difference (i.e., a file list). The file list consists of file pairs. Each file pair is on a single line and of the form: `original_file_and_path|modified_file_and_path`. With this option, the files can be in separate locations (non-matching directory structures). Here is an example

where the files list, `file_list.txt`, is read and each file pair is differenced and placed in the srcDiff archive `srcdiff.xml`.

```
srcdiff --files-from file_list.txt -o srcdiff.xml
```

`srcdiff` supports separate input sources other than the local file system. It can take input from Git and Subversion repositories (Subversion implementation currently does not support authentication). The following are examples. Both take the URL of the repository, `url`, and the two revisions to compute the difference of: `original` and `modified`. With Git, `--git` is used, and with Subversion, `--svn` is used. The output is a srcDiff archive containing the delta of the two revisions. These options can also be combined with the `--files-from` option to only difference the specified files.

```
srcdiff --git url@original-modified -o srcdiff.xml
srcdiff --svn url@original-modified -o srcdiff.xml
```

Human-readable outputs are specified with `--unified` (or `-u`) for the unified view or `--side-by-side` (or `-y`) for the side-by-side view. Both can be configured to display a different number of lines of common context code. The unified view also supports additional context options: `all` for all code, `function` to display the entirety of a function with a change. The following will produce the unified view to the terminal for `original.cpp` and `modified.cpp` with a default of three lines of context.

```
srcdiff -u original.cpp modified.cpp
```

The following does the same with a hundred lines of context and all context respectively.

```
srcdiff -u100 original.cpp modified.cpp  
srcdiff -uall original.cpp modified.cpp
```

To output HTML instead the `--html` switches to output html for viewing in a browser.

```
srcdiff -u --html original.cpp modified.cpp
```

`srcdiff` can be set to not report changes to whitespace with the `--ignore-space` (`-w`) or the `--ignore-all-space` (`-W`) options and comments with `--ignore-comments` (`-c`) option. Option `-W` will not report any whitespace changes. However, this can break up consecutive inserts and deletes of the same type. So, the `-w` option does not break up consecutive code inserts and deletes by marking up whitespace in between. The following produces the unified view, but does not mark up whitespace and comments.

```
srcdiff -u -w -c original.cpp modified.cpp
```

Coloring themes are supported via the `--theme` option. The themes can be `default`, `monokai`, or a file containing a user-defined theme. In addition, the level of syntax highlighting can be specified with `--highlight`. Syntax highlighting can be set to `none`, `partial` (default), or `full`. The following will produce the unified view in HTML with the user specified theme in `theme.txt` and full syntax highlighting.

```
srcdiff -u --theme=theme.txt --highlight=full o.cpp m.cpp
```

As a command-line tool, `srcdiff` can be easily integrated with Version Control Systems such as Git. APPENDIX A contains the code for a wrapper to integrate `srcdiff` with a Git repository.

A complete list of all the options `srcdiff` currently supports is available in APPENDIX B.

CHAPTER 6

Preliminary Study

In order to evaluate the rules, we first performed a preliminary study comparing the output of srcDiff to the widely used GNU `diff`.

For the preliminary study, we choose to compare our tool with the standard version of GNU `diff` for several reasons. It is the de-facto standard of diff tools and it, or the underlying line-differencing algorithm, is used by millions of developers. A comparison with it is one obvious baseline. The full study compares srcDiff to additional textual approaches that support coloring of changes and within-line highlighting.

We did not compare srcDiff to a syntactic differencer as these approaches are not commonly used or adopted. Furthermore, very few provide capability of outputting the changes in a visual format (and thus comparable to srcDiff). GumTree is a more recent syntactic differencing that provides a visual output and thus can be compared to srcDiff. GumTree is also compared with srcDiff as part of the full study.

An online survey (utilizing Qualtrics) was conducted comparing the deltas produced by srcDiff to those produced by GNU `diff`. A PDF of the survey is included in a replication package (sdml.cs.kent.edu/srcDiff/dissertation/preliminary_study). The survey consisted of a pre-questionnaire, a sample problem to familiarize the participants with the format and questions, twelve change samples given in a random order, and an exit questionnaire.

The samples were collected from a corpus of over 1,000 bug-fix revisions on KOffice through a combination of manual inspection, mining for the most common syntactic changes, and changes conforming to Pan's Bug-fix pattern taxonomy [Pan, Kim, Whitehead 2009]. The samples contained both the lines that changed and context if necessary. From these methods, 77 different changes were collected. Of these, 12 complicated changes with a mix of insertions and deletions were selected to exercise the heuristics. Changes with only deletes or only inserts are generally easy to understand. The mixed cases we selected are fairly difficult to understand (and the focus of this study). The samples range from single lines to complete functions. The example given to participants and all samples are provided in APPENDIX C in the same order as presented here. In addition, the PDF of the survey in the replication package contains the samples, also in the same order as presented here.

For the survey, all tasks were timed, all questions were required to be answered for a given change before moving on to the next, and participants were not permitted to return to view a previous change and their answers.

For each change, the participants are presented with the original and modified source code, the GNU diff view of the source-code changes, and the srcDiff view of the source-code changes (all automatically generated from source code). For the GNU diff view, the participants were provided with complete unified context (unified context is the default mode used for version control systems such as Subversion and Git).

For srcDiff, a SAX (Simple API for XML) program was used to transform the XML output into a more easily read format (this study pre-dated the currently provided view

output). For simplicity this shall be denoted the srcDiff view. Code that was changed between the versions was given the background color red and marked with a line-through, inserted code was given the background color green, and code in common to both versions was left as is. In addition, to be able to see where newlines were inserted and deleted the Unicode carriage-return symbol was inserted before the newline when part of a change. Any amount of context (i.e., lines or entire function containing the change) can be given in the srcDiff view however, to be comparable to GNU diff all lines of context were provided. Both programs were run to treat whitespace changes as significant. The replication package contains all the questions presented in the study.

For each change, the following questions were asked on a five-point Likert scale (codified as 6-10). First, the questions were asked for the GNU diff and then, the identical questions were asked for the srcDiff view.

- **Q1:** *Some code marked as changed would be better marked as common.*
- **Q2:** *Some code marked as common would be better marked as changed.*
- **Q3:** *Does an excellent job of capturing the differences between the original and modified code.*
- **Q4:** *Presents differences in a manner that is easy to understand.*

Following Likert questions, participants were asked to rank the Original and Modified, GNU diff, and srcDiff view in the order in which they preferred each view (**Q5**). After answering the questions for the twelve changes, the participants were given an exit questionnaire that asked the same questions about the GNU diff and the srcDiff views overall. In addition, the participants were asked to give any additional comments.

Students in Kent State University's undergraduate Software Engineering course and The University of Akron's Graduate Software Engineering Methodology course were given the survey to take as a homework assignment. In addition, students from Youngstown University's Software Engineering course were given the option to take the survey for extra credit. In total, there were 131 participants and 97 completed the survey (74% completion).

6.1 Hypotheses

In order to evaluate srcDiff and the heuristics, we formulated the following hypotheses.

- **Hypothesis 1:** *srcDiff produces a view of source code that is more accurate than GNU diff.*
- **Hypothesis 2:** *srcDiff produces a view of source code changes that is easier to understand than GNU diff.*
- **Hypothesis 3:** *srcDiff provides a view of source code changes that is more preferable than GNU diff.*

In order to provide support for our hypotheses, we formulated the following null and alternate hypothesis based on **Q1-Q4** from the survey (Table 6.1).

Table 6.1. List of null and alternate hypothesis tested for questions: Q1-Q4.

Hypothesis 1	$H1_0$	There is no difference between GNU diff and srcDiff in marking code as changed that should be marked as common.
	$H1_a$	There is a difference between GNU diff and srcDiff in marking code as changed that should be marked as common.
	$H2_0$	There is no difference between GNU diff and srcDiff in marking code as common that should be marked as changed.
	$H2_a$	There is a difference between GNU diff and srcDiff in marking code as common that should be marked as changed.
	$H3_0$	There is no difference in how excellent a job GNU diff and srcDiff markup differences.
	$H3_a$	There is a difference in how excellent a job GNU diff and srcDiff markup differences.
Hypothesis 2	$H4_0$	There is no difference in how easy GNU diff and srcDiff views are to understand.
	$H4_a$	There is a difference in how easy GNU diff and srcDiff views are to understand.

For **Hypothesis 1**, a rejection of all three null hypotheses: **H1₀**, **H2₀**, **H3₀** with **H1₀** and **H2₀** having a smaller mean and **H3₀** having a larger mean would be ideal and would indicate that srcDiff is perceived to incorrectly mark less code (**H1** and **H2**) and do a more excellent job at marking up the code (**H3**). However, rejection of any hypothesis in a positive direction (and none in a negative direction) will provide support for **Hypothesis 1**. For **Hypothesis 2**, a rejection of the **H4₀** with a larger mean will provide support that srcDiff is perceived to be easier to understand.

6.2 Results

All data was analyzed using SPSS. Responses to **Q1-Q4** were tested for reliability using Cronbach's alpha. All had good (≥ 0.7) to excellent reliability (≥ 0.9). A two-tailed

paired T-test (applicable under Central Limit Theorem) with a 95% confidence interval was run between the GNU diff and srcDiff view responses independently for **Q1**, **Q2**, **Q3**, and **Q4** and for each sample as well as the exit questionnaire for a total of 52 tests (Table 6.2 and Table 6.3). Information about effect size (i.e., Cohen's d) is in Table 6.4 and Table 6.5.

Table 6.2. Results of Two-tailed Paired T-Tests on Q1 and Q2. Left-most column is the sample, and top-most is the questions. For each question, the mean of GNU diff and srcDiff views are given along with the t-test p-value and finally if we can reject the null hypothesis

Sample	Accuracy Changed (Q1)				Accuracy Common (Q2)			
	diff _μ	srcDiff _μ	p-value	Reject Null?	Diff _μ	srcDiff _μ	p-value	Reject Null?
1	7.88	7.34	0	Yes	7.66	7.59	0.52	No
2	7.9	7.25	0	Yes	7.65	7.41	0.023	Yes
3	7.9	7.7	0.11	No	7.54	7.53	0.901	No
4	7.97	7.58	0.01	Yes	7.56	7.56	1	No
5	7.92	7.46	0	Yes	7.59	7.58	0.913	No
6	8.28	7.73	0	Yes	7.74	7.73	0.92	No
7	7.53	7.45	0.56	No	7.63	7.56	0.517	No
8	8.06	7.7	0.02	Yes	7.65	7.74	0.461	No
9	8.15	7.82	0.01	Yes	7.68	7.74	0.577	No
10	7.79	7.57	0.13	No	7.6	7.62	0.857	No
11	7.8	7.53	0.03	Yes	7.65	7.72	0.461	No
12	7.91	7.34	0	Yes	7.56	7.57	0.913	No
Exit Questionnaire	8.28	7.54	0	Yes	7.7	7.71	0.924	No

Table 6.3. Results of Two-tailed Paired T-Tests on Q3-Q4. Left-most column is the sample, and top-most is the questions. For each question, the mean of GNU *diff* and *srcDiff* views are given along with the t-test p-value and finally if we can reject the null hypothesis

Sample	Accuracy (Q3)				Understandability (Q4)			
	<i>diff_μ</i>	<i>srcDiff_μ</i>	<i>p-value</i>	<i>Reject Null?</i>	<i>diff_μ</i>	<i>srcDiff_μ</i>	<i>p-value</i>	<i>Reject Null?</i>
1	8.38	9.01	0	Yes	8.35	9.05	0	Yes
2	8.58	9.16	0	Yes	8.48	9.19	0	Yes
3	8.49	9.19	0	Yes	8.54	9.26	0	Yes
4	8.37	9.05	0	Yes	8.27	9.16	0	Yes
5	8.41	9.11	0	Yes	8.43	9.22	0	Yes
6	8.35	8.99	0	Yes	8.29	8.92	0	Yes
7	8.42	9.01	0	Yes	8.3	9.02	0	Yes
8	8.26	8.94	0	Yes	8.3	8.93	0	Yes
9	8.02	8.99	0	Yes	7.85	9.09	0	Yes
10	8.35	8.89	0	Yes	8.3	8.91	0	Yes
11	8.16	9.06	0	Yes	8.05	9.2	0	Yes
12	8.49	9.06	0	Yes	8.54	9.11	0	Yes
Exit Questionnaire	8.49	9.26	0	Yes	8.32	9.27	0	Yes

Table 6.4. Effect size (Cohen's d) for each sample and for Q1 and Q2. In general, there was a small effect for Accuracy Changed (Q1) and almost no effect for Accuracy Common (Q2).

Sample	Accuracy Changed (Q1)		Accuracy Common (Q2)	
	<i>Cohen's d</i>	<i>Effect Size</i>	<i>Cohen's d</i>	<i>Effect Size</i>
1	0.487	Small	0.064	None
2	0.583	Medium	0.211	Small
3	0.168	None	0.009	None
4	0.354	Small	0.000	None
5	0.432	Small	0.009	None
6	0.485	Small	0.009	None
7	0.070	None	0.065	None
8	0.338	Small	-0.088	None
9	0.301	Small	-0.055	None
10	0.195	None	-0.019	None
11	0.253	Small	-0.066	None
12	0.531	Medium	-0.009	None
Exit Questionnaire	0.702	Medium	-0.009	None

Table 6.5. Effect size (Cohen's d) for each sample and for Q3 and Q4. In general, there was a medium to large effect on Accuracy (Q3) and Understandability (Q4).

Sample	Accuracy (Q3)		Understandability (Q4)	
	Cohen's d	Effect Size	Cohen's d	Effect Size
1	-0.627	Medium	-0.692	Medium
2	-0.594	Medium	-0.757	Medium
3	-0.722	Medium	-0.764	Medium
4	-0.720	Medium	-0.877	Large
5	-0.779	Medium	-0.907	Large
6	-0.656	Medium	-0.595	Medium
7	-0.632	Medium	-0.740	Medium
8	-0.669	Medium	-0.613	Medium
9	-0.970	Large	-1.226	Large
10	-0.519	Medium	-0.581	Medium
11	-0.888	Large	-1.193	Large
12	-0.610	Medium	-0.610	Medium
Exit Questionnaire	-0.881	Large	-1.004	Large

For **H1**, **Q1** has statistical significance in all but three cases allowing us to reject **H1₀** in favor of **H1_a** in most cases. *srcDiff* also has a smaller mean for **Q1**, indicating that *srcDiff* has less code incorrectly marked as changed. The effect size is small to medium. This allows us to conclude that in general, *srcDiff* view is perceived to produce source code with less common code marked as changed.

Only one null hypothesis can be rejected of **H2**. Therefore, no statistical inference can be made. However, **Q3** is statistically significant in all cases allowing us to reject **H3₀** (no difference in how excellent a job marking differences) in favor of **H3_a** (a difference in how excellent a job marking differences). *srcDiff* view also possesses a greater mean which indicates that the *srcDiff* view is perceived to do a better job of capturing changes. Effect size is medium to large. Thus, we can conclude that the *srcDiff* view does a more excellent job at marking source code changes.

Results for **H1-H3** provide support for **Hypothesis 1**. That is, srcDiff produces source-code changes with less common code marked as changed (**H1**), does not mark more changed as common than GNU diff (**H2**) and does a better job (**H3**). So, we conclude that *the srcDiff view is perceived to produce a more accurate view of source code changes than GNU diff*.

From the results of **Q4**, we can safely reject all **H4₀** in favor of **H4_a** with $\text{srcDiff}_\mu > \text{diff}_\mu$. The effect size is medium to large. From this we can conclude that srcDiff is perceived to provide an easier to understand view of source code changes. This directly provides support for **Hypothesis 2** and we can therefore claim that *the srcDiff view is perceived to provide an easier to understand view of source code changes than GNU diff*.

The results for **Q5** are shown in the stack bar chart in Figure 6.1. Blue shows the percentage each were selected the most preferable, green the second most preferable, and red the least preferable. The results show that on average over all the samples and exit questionnaire srcDiff was overwhelming chosen (67%) as the first choice and very infrequently chosen for either the second (16%) and last choice (17%). The original and modified was the second most frequently chosen first choice (18%), however, it was also the largest last choice (59%). Although GNU diff was the least popular first choice (14%), it was by far the most popular second choice (62%). As the results provide positive support for srcDiff in **Hypothesis 3**, we are able to conclude that the srcDiff view presents a view of source code changes that is more preferable than GNU diff.

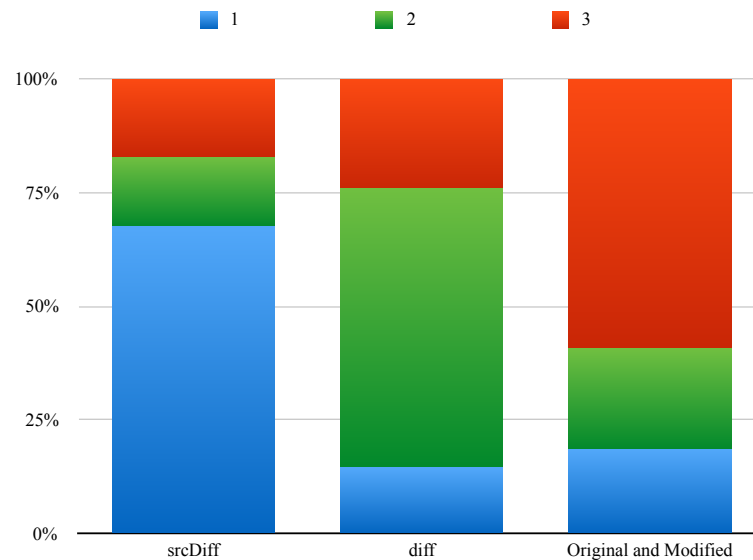


Figure 6.1. Answers to preferability question (Q5) as a stacked bar chart. Blue is the percentage selected as most preferable, green second preferable, and red least preferable

In conclusion, survey results provide positive results for srcDiff in all three hypotheses. That is, srcDiff view of source code is perceived as more accurate and easier to understand than GNU `diff` and it is more preferable than GNU `diff`.

6.3 Threats to Validity

The following threats to validity have been identified. As a survey was used, only twelve samples were chosen so that the survey could be conducted in a manageable amount of time necessary to obtain responses and quality answers [Deutkens, de Ruyter, Wetzels, Oosterveld 2004] [Galesic, Bosnjak 2009]. That is, adding questions would have made the survey too long and would have drastically lowered our response rate. While we could not cover all possible situations we selected changes to be representative as possible. Some participants completed the survey faster than possible (i.e., speeders). Adjusting for

speeders only benefited the statistical inference testing toward srcDiff slightly, so all participants were left in the study.

One threat to validity may be the experience of the participants. Only undergraduate and graduate students participated in the survey. Industry professionals may give different results. This threat is mitigated as some participants did have expert level programming experience, and many of the students have worked or are working in industry.

The rules are derived from examining real-world software projects and our experience as developers. Several situations could exist for which no current rule applies. We designate this for future work. However, for the rules presented in this paper, we are confident in their validity as shown by the evaluation.

We examined changes to KOffice in developing our rules. This could be a threat to over-fitting as the survey questions are from KOffice. However, our experience with changes in KOffice and other systems was used to develop the rules, while an in-depth analysis of KOffice was used to refine, add to, and evaluate them. For instance, GCC was used in the early development of the *Nesting* rules. As such the rules represent changes that typically occur across systems. The samples for the survey were constructed from a pool using a taxonomy, mining for the most common syntactic changes along with manual inspection and then they were chosen to be representative of the heuristics. One of the samples (9) used in the study involves a heuristic that we specifically derived from examination of KOffice. We did not see this type of change in other systems (but did not do an extensive search for other examples). Results for the t-test on this are no better than

the remaining questions and removing affects on **Q5** provides small changes ($>.6\%$) to the reported percentages in favor of srcDiff.

Comparison to a different tool may yield different results. The full evaluation compares srcDiff to additional tools.

Lastly, changes were gathered for only bug-fix revisions. All though these might not be representative of all changes, we believe that these are among the more complicated/non-trivial changes and therefore more than adequate.

CHAPTER 7

Evaluation

After running the preliminary study, a second study was designed (building upon the experience gained in the first) to address possible threats to validity with the first study. The following is a summary of what is changed.

- Comparison against multiple types of differencing tools including a syntactical differencer and textual differencers supporting within-line change highlighting.
- Questions were reworded to be less ambiguous and to accommodate the expanded study.
- Questions for accuracy, preference, and understandability were changed to constant-sum questions to allow for ranking of approaches in each criteria and to ascertain the degree of which they out rank each other.
- Comparison is in Java as the syntactical differencer does not support C++.
- Changes were collected from multiple projects uninvolved with rule creation.
- Changes were collected partially randomly from all change history with non-randomness used to try and diversify the selected changes and increase generalizability of results.

We now discuss the evaluation in depth.

In this study, srcDiff and its rules are compared to GumTree [Falleri et al. 2014], GitHub’s difference view [GitHub 2016] and Mergely [Peabody 2016].

For this study, we formulate three research questions.

- **RQ1:** *Does srcDiff produce changes that are easier to read and understand than the other approaches?*
- **RQ2:** *Is srcDiff preferred over the other approaches?*
- **RQ3:** *Does srcDiff result in a delta that is more accurate than the other approaches?*

GumTree is a syntactic-differencing approach. As computing an optimal edit script while considering moves is known to be NP-hard, GumTree uses heuristics to approximate the optimal-edit script. For details of the algorithm please see [Falleri et al. 2014]. Like most syntactic-differencing approaches, GumTree is unable to provide changes to comments and whitespace. GumTree can output a side-by-side view of changes by overlying the AST edits onto the source code. Several other non-visual representations of the edits are also provided. In [Falleri et al. 2014], GumTree is compared to several other differencers. The results show that GumTree produces a more optimal-edit script. In addition, the authors performed a manual investigation to compare the visual output of GumTree to that of Mergely. Once again, the results show GumTree as the best approach compared. As GumTree represent the best research tool currently available, GumTree is selected for this study.

GitHub’s difference view is a line-based differencing approach to source-code differencing that also highlights within-line changes. It supports both a side-by-side view

and a unified view of changes. It is limited in the fact that it can only be used to view changes in repositories hosted by GitHub. However, as it is used by GitHub, it sees heavy use and is therefore selected for study.

Mergely is another line-based differencing approach that supports highlighting of within-line changes. The within-line changes are computed in a separate manner than GitHub's difference view. It supports a side-by-side view of changes. As Mergely was used in a comparison with GumTree [Falleri et al. 2014], it provides an excellent baseline and point of comparison between these studies.

As previous approaches aim at an optimal edit script, edit script size and number-of-mappings are often used to evaluate different syntactic-differencing approaches [Falleri et al. 2014; Hashimoto, Mori 2008]. However, such an evaluation is not appropriate here. Edit script size does not take into account the understandability of the resultant delta as we have demonstrated previously. Additionally, edit script cannot be used for comparing syntactic approaches to line-differencing approaches. Lastly, the AST used in srcDiff and GumTree differ drastically (e.g., different number of nodes, different syntactic categories) and a comparison of edit script is therefore not very meaningful.

Since the goal of srcDiff is to produce a delta that is both accurate and understandable to programmers, an online within-participant survey is conducted as an evaluation. The study consists only of side-by-side views of source-code changes in order to provide for a fair comparison with GumTree, which only supports a side-by-side view. srcDiff and GitHub also support a unified view. Replication package and research data for the study can be found at sdml.cs.kent.edu/srcDiff/dissertation/evaluation.

7.1 Data Collection and Preparation

Since, GumTree was evaluated on the Java [Falleri et al. 2014] programming language, Java is used as the language for samples in the study. In total, fourteen samples of source-code changes are collected from four separate Java systems: Elasticsearch, Google Guava, SLF4J, and jEdit. The following explains how change samples were collected for the study. In general, the collection procedure aims at getting a wider variety of changes to increase the generalizability of the results.

First, 100 revisions are selected at random for each system from all revisions with modified files. From these, two shorter methods, 5 to 30 lines, with smaller amounts of change, 2 to 10 lines as reported by a whitespace-ignoring line diff, and one larger method, 20 to 50 lines, with larger amounts of changes, 10 to 25 lines, are selected at random from Elasticsearch, Google Guava, and SLF4J. These criteria are used to provide for a wide coverage of change complexity while keeping the size of the methods and amount of change manageable for a survey. One sample was redrawn, as the change was too simple to produce variation in the output of each tool. The remaining five samples were chosen to supplement the randomly-drawn methods with different change types and selected by querying the 100 random revisions (via srcML on srcDiff documents). They are: a replaced method (Google Guava), a heavily modified class (Google Guava), and three examples of changes to conditions and conditionals (jEdit and Google Guava). All the samples can be found in APPENDIX D or as part of the replication package.

Ideally, a larger number of samples would have been included in the study, however, any larger of a size is prohibitive and greatly decrease the number of participants

and quality of the results [Deutskens, de Ruyter, Wetzels, Oosterveld 2004; Galesic, Bosnjak 2009]. As such, jEdit is not used for the shorter and longer methods, as this generates too many examples for a survey. jEdit is used as part of the supplemental changes to provide both an additional system and a system using Subversion version control system (others utilized Git).

In the survey, GumTree is presented in HTML to allow for the interaction it supports via Javascript. To be comparable, srcDiff is also presented in HTML, although no Javascript interaction is needed. For GitHub and Mergely, we attempted to present them in HTML, however they are best presented as images. To control for any possible problems, directions are given in the survey to compensate for participants with small screens/resolutions. This worked well, as only three participants noted having trouble viewing GitHub/Mergely. One could not view them at all, so was removed from the pool of results, and the other two reported only inconvenience. For two with inconvenience, the results were analyzed with and without them. Since, removal of these two participants negatively affected GitHub and Mergely (clearly showing that having GitHub and Mergely as images was not a threat to validity) they were kept in the study. Several user responses about how much they liked GitHub's view also shows that this was acceptable.

GumTree is unable to report changes to comments, so, comments are stripped from the samples. All the tools are run to ignore whitespace when computing a difference. Additionally, since GumTree is unable to report changes to whitespace, srcDiff is set to not report changes to whitespace. Unfortunately, GitHub and Mergely always show changes to whitespace and have no option to disable output of whitespace changes. As

such, participants are instructed to ignore changes to whitespace when answering questions. With exception of GumTree, all the tools are run on only the code samples. As GumTree requires syntactically-complete code, the code is wrapped with a class before it can be run. This is stripped out in the survey so that all tools show the same code. GumTree is also run on the full classes to check the results. There is no difference on ten of the fourteen samples. However, on the four remaining complete files, GumTree produced a slightly worse (sample 3, 5, 10) to far worse (sample 4) delta. On sample 13, GumTree reported two different changes depending on the run. The best is chosen. The latest release of GumTree (2.0.0) is used. The previous release (1.0.0) did not perform as well. That is, GumTree is shown in the best possible light for the evaluation.

Since, each tool outputs the differences with a slightly different representation (e.g., highlighting colors), the participants are given instructions on how the representations show the same concepts. Each sample is presented separately and the participant is given the output of the four approaches along with the original and modified code. Participants are instructed to answer in any order and only for the current sample. The samples are shown in random order. The study is blind with regard to the tools. That is, the approaches output is explained to the participants, however, the names of the approaches are not given.

7.2 Measures

The survey evaluated the approaches along four separate criteria: understandability (**RQ1**), readability (**RQ1**), personal preference (**RQ2**), and accuracy (**RQ3**). Table 7.1 contains a listing of the questions. The first two questions for accuracy use a standard unipolar rating scale of *None at all*, *A slight amount*, *A moderate amount*, *A large amount*,

and *A very large amount* (coded as 6-10). The remaining are constant sum questions, selected to allow for ties and to allow the participants to provide the extent for which a single approach is better than the other. All data is ordinal scale, that is, the median is calculated, while the mean is not valid [Good, Hardin 2012]. Lastly, the participants were given the option to provide comments. The questions are asked individually for each sample and after answering the questions for all samples, the participants are asked the same questions about the approaches overall.

Table 7.1. Evaluation survey questions.

Criteria	Question
<i>Accuracy - Unchanged</i>	How much of the code is marked as unchanged that should be marked otherwise?
<i>Accuracy - Changed</i>	How much of the code is marked as changed that should be marked otherwise?
<i>Accuracy</i>	Rank the tools by distributing 100 points among the tools relative to how well each tool does at capturing the differences between the original and modified code.
<i>Understandability</i>	Rank the tools by distributing 100 points among the tools relative to how easy each tool's output is to understand.
<i>Readability</i>	Rank the tools by distributing 100 points among the tools relative to how readable each tool's output is.
<i>Preference</i>	Rank the following by distributing 100 points among each relative to your preference.

7.3 Results and Discussion

The survey was given as a project in two software engineering classes and to a number of graduate students. Sixty-nine students (5 were experts) finished the survey. Table 7.2 and Table 7.3 show the results. As there is almost no variation in the first question, it is not reported. In preference, the participants are also asked to rank the

Original and *Modified* source code. As this received a median of zero in all cases, this is left out. All the research data and full charts are available in the replication package.

For understandability/readability questions, srcDiff is rated the best approach, most frequently (most points including ties), while GitHub is ranked second. This provides direct support for **RQ1**, that is, srcDiff is easier to read and understand. For the preference question, srcDiff is also rated best, most frequently (by a large margin). This also provides direct support for **RQ2**: srcDiff is preferred over the other approaches. In terms of accuracy, GumTree and srcDiff are tied. The reason is that the survey instructs participants to compare the approaches to what they determined is actually changed by the developer. Without enough experience examining changes to software, abstracting out what actually changed is a difficult task. That is, a large portion of the participants do not have significant experience with differencing tools and looking at changes relative to what a developer actually performed. While these participants are fully capable of evaluating the readability, understandability, and preference criteria, they did not have enough expertise to evaluate the accuracy. Results on experts (those rated as excellent skills using differencing tools) for the ranking questions, rate srcDiff as the most accurate 15/15 times, most understandable 15/15 times, most readable 15/15 times, and most preferred 14/15 times. No other tool is ranked the best more than 2 times on the combined samples and overall. In essence, GumTree appears more accurate to less skilled developers, possibly due to the additional fine-grained markup on different code, however, since these changes are not meaningful, they are unable to understand the changes. This supports **RQ3**. srcDiff

produces a delta that is more accurate than the other approaches. The authors invite the reader to view the changes produced by the approaches in the provided replication package.

Table 7.2. Results for each of the samples and post-questionnaire (Overall). All reported measures are medians with GitHub (GH), Mergerly (M), srcDiff (SD), and GumTree (GT). Last line shows a count of the number of samples (and Overall) for which the approaches achieved the best result (including ties) out of all the approaches. For Changed, smaller number is better, others larger is better.

	Changed				Accuracy				Understandability			
<i>Samples</i>	<i>GH</i>	<i>M</i>	<i>SD</i>	<i>GT</i>	<i>GH</i>	<i>M</i>	<i>SD</i>	<i>GT</i>	<i>GH</i>	<i>M</i>	<i>SD</i>	<i>GT</i>
<i>Sample 1</i>	6	7	6	6	25	23	27	25	25	24	29	22
<i>Sample 2</i>	6	7	6	6	25	20	25	25	25	20	25	25
<i>Sample 3</i>	7	7	6	7	25	25	25	25	30	25	30	15
<i>Sample 4</i>	8	8	6	6	17	15	30	35	20	20	25	30
<i>Sample 5</i>	6	6	6	7	25	25	25	25	26	25	30	18
<i>Sample 6</i>	7	6	6	6	25	25	25	25	25	20	30	15
<i>Sample 7</i>	7	7	6	6	25	20	25	25	25	24	25	25
<i>Sample 8</i>	6	7	6	6	20	20	25	30	25	20	26	25
<i>Sample 9</i>	6	6	6	6	25	25	27	25	25	25	28	25
<i>Sample 10</i>	7	7	6	6	25	20	25	25	25	20	30	20
<i>Sample 11</i>	6	7	6	6	24	21	25	26	25	20	28	25
<i>Sample 12</i>	6	6	6	6	25	22	30	25	25	20	30	22
<i>Sample 13</i>	7	7	6	6	25	20	25	25	30	20	27	20
<i>Sample 14</i>	7	6	7	6	23	25	25	25	25	25	25	25
<i>Overall</i>	7	7	7	7	25	20	25	25	25	20	28	20
<i>Count</i>	8	6	14	13	8	4	12	12	5	1	13	4

In [Falleri et al. 2014], GumTree is compared to Mergely via a manual investigation by the authors. Our study also supports that GumTree is better than Mergely. However, for preference, understandability, and readability, GitHub is more often rated higher than GumTree. GumTree is often ranked lower than all other approaches when presented with more complicated changes (e.g., sample 3, 5, 6), while in the GumTree study, only simple changes are used (those that ChangeDistiller reported as a single change). In contrast, srcDiff is ranked on par with or as the highest. This validates a major claim of this

dissertation: optimal tree-differences produce poor results on more complicated changes, while srcDiff performs very well.

Table 7.3. Results for each of the samples and post-questionnaire (Overall). All reported measures are medians with GitHub (GH), Mergerly (M), srcDiff (SD), and GumTree (GT). Last line shows a count of the number of samples (and Overall) for which the approaches achieved the best result (including ties) out of all the approaches. Larger is better.

	Readability				Preference			
<i>Samples</i>	<i>GH</i>	<i>M</i>	<i>SD</i>	<i>GT</i>	<i>GH</i>	<i>M</i>	<i>SD</i>	<i>GT</i>
<i>Sample 1</i>	25	25	30	20	25	20	25	20
<i>Sample 2</i>	25	20	26	25	25	20	25	25
<i>Sample 3</i>	25	24	30	15	25	20	25	15
<i>Sample 4</i>	25	20	25	30	20	15	28	30
<i>Sample 5</i>	27	25	30	16	25	24	30	15
<i>Sample 6</i>	30	20	30	15	25	20	30	15
<i>Sample 7</i>	25	21	27	25	20	20	25	22
<i>Sample 8</i>	25	20	30	25	20	20	25	25
<i>Sample 9</i>	25	20	30	25	22	20	25	25
<i>Sample 10</i>	28	20	26	24	25	20	30	20
<i>Sample 11</i>	25	20	28	25	25	20	25	25
<i>Sample 12</i>	25	20	30	20	22.5	20	30	20
<i>Sample 13</i>	25	20	30	20	25	20	30	20
<i>Sample 14</i>	25	25	25	20	20	22	20	20
<i>Overall</i>	25	20	30	20	25	20	25	20
<i>Count</i>	3	1	13	1	5	1	13	5

The data was investigated for integrity and comments read. Nothing was suspicious about the expert's data. From reading the comments on all participants, two people claimed inability to see code on the overall questions (which had no code, so possibly answered randomly), two had small inconvenience in viewing GitHub and GumTree images, and two may have made up their minds in the middle of the exam and thus stopped answering additional samples wholeheartedly (i.e., given up). The affect each had individually and combined was verified on the data. Removal of those who may not have answered overall

correctly changes the results slightly in favor of srcDiff. Those with some inconvenience with GitHub and Mergely images have a slight negative affect of them when removed. Those who may have given up had mostly slight affects. The result on the conclusions (all participants) is that srcDiff is slightly less accurate than GumTree, but GumTree and GitHub are even less preferable and understandable. This makes GumTree slightly more preferable than GitHub and GitHub roughly on par with Mergely. Looking at the data when removing all of these participants slightly benefits srcDiff. As these involve some amount of speculation, and the effects are minor, as they do not affect the conclusion of srcDiff being the most accurate (which comes from expert data) and the other criteria on either data set, these participants were left in with the presented data.

Also, of note the completion time was investigated for all participants. Two non-experts had a somewhat quick, but not impossibly quick response time. The data was also investigated without these two participants. As the results were in favor of srcDiff and do not affect the conclusions, they are also left in the presented data.

7.4 Timing

To illustrate srcDiff's efficiency, we ran timings on srcDiff and GumTree using changes between 1.509.4 and 1.532.2 of the continuous integration server Jenkins (i.e., same releases as in GumTree [Falleri et al. 2014]). However, we did not discard any files, running both tools on all modified files in revisions between releases. In total, git log reports 1309 modified Java files. In contrast to [Falleri et al. 2014] and to better reflect a user's expectations, we time entire runs of each using the time command. For srcDiff, a unified diff is computed (i.e., typical user request). As GumTree's human-readable output

is to a URL requiring human interaction, its JSON output (latest revision March 2016 develop build) is used. The minimum of three runs on each file is taken. srcDiff took a mean wall clock time of 0.1 seconds, while GumTree took 1.17 seconds. Although GumTree's human readable output is setup to listen to changes (i.e., subsequent comparisons faster than reported here), it is significantly slower in initial startup and thus in computing a delta for analysis. All timings run on iMac running El Capitan with 2.7GHz i5/8GB RAM.

7.5 Threats to Validity

The following threats to validity are identified. As a survey is used, only fourteen samples are chosen so that the survey can be conducted in a manageable amount of time necessary to obtain responses and quality answers [Deutskens, de Ruyter, Wetzels, Oosterveld 2004; Galesic, Bosnjak 2009]. Adding questions makes the survey too long and most likely drastically lower the response rate. To mitigate potential problems with response rate/quality, subjects are informed that they can do the survey in multiple sittings and given proper incentive. While all possible change situations cannot be covered, nine of the fourteen are drawn at random, and thus, represent more typical change situations. The remaining are used to supplement the randomly drawn in order to make sure certain types of changes are covered, e.g., changes to conditions and conditionals which we regard as very important for differencing. That is, we can claim that srcDiff is better in both the typical cases and important cases selected for the survey.

One threat may be that a single representation of changes was not used for all four approaches. If all the formats were unified into the GumTree view, then as the GitHub and

Mergely have no equivalent markup or report moves and updates, the conversion of such would have been arbitrary (biased). If all were unified into one of the other simpler views, then the possible accuracy that GumTree is afforded by the additional markup would be lost (bias) and could make the output more difficult to read and understand (bias). That is, we feel that the explanation of the different approaches is the best course of action.

In addition, accuracy is asked in regard to how well the developers changes are recovered and since the differences in markup of each type is explained, it is largely agnostic to how it is stylistically marked up. Understandability is asked in relation to how easy it is to understand the output. By definition, to understand is “to perceive the meaning” [Dictionary.com 2017]. So, the question asks how easily they can perceive the meaning of the output. As the participants were instructed in how each tool presents changes (i.e., they are equally capable of understanding how each change type is stylistically marked up), this is largely what is marked up and as what operation. To many, understandability and readability are the same measure (at least one comment in the data attests to this), to others they are not. As the results are close, they were regarded as mostly the same. Preference is based on the individual and their weighting of the other criteria.

One aspect that could affect ability to evaluate the approaches is color blindness, which is controlled for in the study. The removal of all color-blind participants is negligible and does not affect the conclusions.

One threat to validity may be the experience of the participants. A broader set of industry professionals may give different results. This threat is mitigated as some

participants have expert-level programming/differencing experience, and/or have worked in industry.

The rules are derived from examining real-world software projects and our experience as developers. Situations may exist for which no current rule applies. We designate this future work and note that srcDiff is highly adaptable (i.e., new rules easily added) and not fixed on immutable algorithms. We are confident in the rules presented, as shown by the evaluation.

srcDiff is developed primarily to support C/C++. Minimal effort went into adding support for Java. Different systems are used for investigation (android libcore, lucene-solr, and hadoop) and multiple systems for evaluation. As such, there is no over-fitting present.

Comparison to other approaches may yield different results. However, since we compared multiple approaches (line and state-of-the-art syntactic) we are confident results generalize.

People commit to different version control systems (VCS) in different granularities, especially centralized/decentralized systems. Both Subversion and Git are used in the survey/investigation, although Subversion is used to a lesser extent.

CHAPTER 8

Investigating Method Stereotype History

In this chapter, the first application of srcDiff to a software engineering problem is presented. Broadly, the task is to investigate changes to method stereotypes. That is, we are interested in seeing how each individual method's stereotype evolves over time. This study answers the following research questions:

- **RQ1:** Is a method's stereotype stable?
- **RQ2:** What consequences are there to a method's stereotype changing?

In Section 8.1, method stereotypes are introduced along with the relevant related work. In Section 8.2, the problem is redefined and stated in more detail. Section 8.3 provides the details of data collection. The results and discussion are given in Section 8.4 and threats to validity in Section 8.5. The general conclusions are provided in Section 8.6.

8.1 Method Stereotypes

A brief introduction to method stereotypes is provided in this section. For a more complete description see [Dragan, Collard, Maletic 2006]. The work of Dragan et al. [Dragan, Collard, Maletic 2006] introduces a taxonomy for method stereotypes. Table 8.1 shows a version of the taxonomy given by Dragan et al. [Dragan, Collard, Hammad, Maletic 2011; Dragan, Collard, Maletic 2010]. A method stereotype concisely represents behavioral aspects of a method. Method stereotypes are separated into five broad categories (far-left column):

- *Accessors* – query the state of an object on which it is called.
- *Mutators* – modify the state of an object on which it is called.
- *Creational* – create/provide new objects.
- *Collaborational* – work on objects pertaining to classes other than itself.
- *Degenerate* – structural/collaborational stereotypes are limited. Simplest stereotypes.

Table 8.1. Method stereotype taxonomy. The far-left column is the broad stereotype categories, the middle column the individual stereotypes, and the right column is a description of each stereotype

Stereotype Category	Stereotype	Description
<i>Structural Accessor</i>	get	Returns a data member.
	predicate	Returns Boolean value that is not a data member.
	property	Returns info about data members.
	void-accessor	Returns information via a parameter.
<i>Structural Mutator</i>	set	Sets a data member.
	command	Performs a complex change to the object's state.
	non-void-command	
<i>Creational</i>	constructor, copy-const, destructor, factory	Creates and/or destroys objects.
<i>Collaborational</i>	collaborator	Works with objects (parameter, local or return value).
<i>Degenerate</i>	stateless	No data members read/written directly and can have only one call to other class methods
	empty	Has no statements.

The individual stereotypes (middle column with its description in the far-right column) indicate a refinement of the broad behavior described by the category. As an example, *accessor* methods (general category) query an object's state. A predicate method

is a special type of *accessor* that returns a computed Boolean containing some information about the state of the object on which it is called.

Method stereotypes are not necessarily exclusive. That is, a method may have multiple different stereotypes. For example, it is common for any of the stereotypes to also have *collaborator* as a secondary stereotype. In addition, while considered an anti-pattern, a method can both query an objects state (*accessor*) and modify that state as well (*mutator*).

In addition to the taxonomy, Dragan et al. [Dragan, Collard, Maletic 2006] also presents a set of rules for automatically identifying a method’s stereotype from its source-code. These rules are encoded and available in the publically available tool StereoCode (<https://github.com/srcML/stereocode>). StereoCode reports a superset of the stereotypes defined above. For instance, a pure *get* method is required to be `const` and thus cannot be both an *accessor* and *mutator*. However, the tool will also identify non-`const` *get* methods.

Method stereotypes have been used to support various software-analysis tasks. The distribution of the different types of method stereotypes forms the basis for class stereotypes [Dragan, Collard, Maletic 2010]. The stereotypes of added/deleted methods have been used to categorize commits [Dragan, Collard, Hammad, Maletic 2011]. In addition, class/method stereotypes have been used to formulate documentation for methods [Abid, Dragan, Collard, Maletic 2015] and classes [Moreno et al. 2013], and method and commit stereotypes have been used for commit-message generation [Cortes-Coy, Linares-Vasquez, Aponte, Poshyvanyk 2014].

The closest work to the work done here is Dragan et al. [Dragan, Collard, Hammad, Maletic 2011], which looked at only added and deleted methods to categorize commits, and Dragan [Dragan 2010] who looked at the distribution of method stereotypes in a few releases (~20) of two systems and found the distribution of method stereotypes to be fairly consistent for one system and unstable for the other. For this study, we will investigate the stability and consequences of the stereotypes to modified methods (methods that were changed in some way) using the entire history provided by version-control systems. That is, this study is a compliment and a large extension to Dragan’s work [Dragan 2010].

8.2 Evolution of a Method’s Stereotype

We now restate and expand upon the research problem and discuss theoretically the consequences of a change in method stereotype (**RQ2**). A manual investigation of the consequences is provided in Section 8.4.2.

Method stereotypes capture behavioral aspects of a method. That is, a method is created for a specific purpose/intent to accomplish a specific goal and a method stereotype is a high-level description of this purpose. Throughout the lifetime of a method, if it is well designed, a method’s purpose should not change, and thus a method’s stereotype should remain constant. That is, we hypothesize that method stereotypes are resilient to change (i.e., they are stable). Furthermore, certain types of changes in method stereotype can be either neutral, positive, or negative indicators. For example, a transition from `non-const get` to a `get` is an example of a positive transition, while the opposite, a transition from a `get` method to a `non-const get` method is a negative transition that indicates a degradation in the design of the system (i.e., poor design). Note, the addition of a `non-const get`

method is often necessary in programming and therefore valid. However, with the transition removing a `const` accessor (which is necessary) for a non-`const` `get` method is negative. In this research, we focus on changes in stereotype only. Table 8.2 and Table 8.3 contain a categorization of different types of important stereotype transitions. This categorization is not meant to be complete, but to highlight important transitions. Category-column contains the category name, description-column gives a description of what stereotype transitions are a part of that category, and notes-column gives notes about the significance along with if such transitions are generally positive, negative, or neutral. Group-column groups similar stereotype change categories. This provides a partial answer to **RQ2**. In Section 8.4.2, we will investigate this further in a manual investigation.

The goal of this study is then, to investigate how often stereotypes change (i.e., **RQ1**) and if they change, how often and what to do stereotypes change. To do this, the entire history of several systems are analyzed and data collected on changes (or lack-

Table 8.2. Categories of important stereotype transitions. The category-column lists the categories of stereotype changes. The group-column groups similar categories, the description-column is a description of the category and what belong as part of that category, and the notes-column contains notes about the transitions.

Group	Category	Description	Notes
<i>Unclassified</i>	<i>Move to/from Unclassified</i>	Method transitioned to/from <i>Unclassified</i>	• Methods that cannot be classified lack a clear abstraction. • To <i>unclassified</i> (negative) • From <i>unclassified</i> (positive)
<i>Structural Accessor</i>	<i>Move to/from Non-const get</i>	Method transitioned between non- <code>const</code> <code>get</code> and another <i>Accessor</i> .	• From <i>Accessor</i> to non-<code>const</code> <code>get</code> breaks ability use on constants/degrades design (negative). • From non-<code>const</code> <code>get</code> to <i>Accessor</i> increases information hiding. Method most likely should have always been <i>Accessor</i> (positive).

Table 8.3. Method stereotype transition categories (continued).

<i>Collaborational</i>	<i>Add Collaborational</i>	A method adds a <i>Collaborational</i> stereotype or transitions from another category to a <i>Collaborational</i> stereotype.	• Addition of <i>Collaborational</i> indicates increased dependency to other object(s).
	<i>Remove Collaborational</i>	A method removes a <i>Collaborational</i> stereotype or transitions from a <i>Collaborational</i> stereotype to one of another category.	• Removal of <i>Collaborational</i> indicates decreased dependency to other object(s).
<i>Degenerate</i>	<i>Add Degenerate</i>	A method adds a <i>Degenerate</i> stereotype or transitions from another category to a <i>Degenerate</i> stereotype.	• Method's functionality has been diminished. • Indicates method does not have enough responsibility and consider removal.
	<i>Remove Degenerate</i>	A method removes a <i>Degenerate</i> stereotype or transitions from a <i>Degenerate</i> stereotype to one of another category.	• <i>Degenerate</i> methods do not provide enough functionality. • Removal is generally a positive. • Indicates addition of more functionality or increased complexity.
<i>Multiple Categories</i>	<i>Cross Stereotype Boundaries</i>	Method's that change from <i>Structural Accessor</i> , <i>Structural Mutator</i> , and <i>Creational</i> to a different one of those categories.	• Massive change to function behavior. • Change is suspicious and should be investigated.
	<i>Add/Remove/Replace Categories</i>	Method that has multiple stereotypes from at least two of the following: <i>Structural Accessor</i> , <i>Structural Mutator</i> , and <i>Creational</i> .	• Method has too much responsibility. • Presence is possible code smell. • Transition that adds additional method stereotypes indicates degrade in design. • Transition that removes stereotypes indicates a design improvement. Transitions where categories are replaced with others indicates poor design.

thereof) to method stereotypes. As the transition categories provided in Table 8.2/Table 8.3 is primarily theoretical, an investigation into the changes that induce a change in method stereotype will be investigated manually on a number of different examples.

8.3 Data Collection

To investigate changes in method stereotypes, fifty open-source software systems are selected for study. Table 8.4 and Table 8.5 provide a listing of the selected systems (left-column). The number of revisions in each that were non-merges and contained a modified C++ file are in the middle column, and the right column has the location from which the repository was taken. When the repository location is not a fully qualified URL, the repository comes from GitHub and has the following complete location: `https://github.com/{Location}.git`. The selected systems cover many different domains and repository sizes and therefore, provide a good representative sample.

The data collection process consists of the following parts:

1. Identification of all revisions and files changed in those revisions.
2. Application of srcDiff to the original and modified version of all changed C++ files to generate srcDiff archives of each revision.
3. Analysis of the srcDiff archives to locate changed methods.
4. Collecting of the original and modified stereotype of each changed method.

More specifically, data is collected for each system in the following manner. First, a local clone of the repository is taken. Next, a srcDiff archive of every modified C++ file

Table 8.4. List of selected software systems by alphabetical order. Left column is system, middle is number of revisions with modified C++ files, and right is the repository location. Unqualified locations are from GitHub: <https://github.com/{Location}.git>.

System	# Revisions	Location
<i>antimony</i>	1,737	mkeeter/antimony
<i>appleseed</i>	5,369	appleseedhq/appleseed
<i>BansheeEngine</i>	3,058	BearishSun/BansheeEngine
<i>bitcoin</i>	5,785	bitcoin/bitcoin
<i>blender</i>	24,215	git://git.blender.org/blender.git
<i>cgal</i>	39,583	CGAL/cgal
<i>ChaiScript</i>	1,312	ChaiScript/ChaiScrip
<i>citra</i>	2,908	citra-emu/citra
<i>Clang</i>	56,631	http://llvm.org/git/clang
<i>cocos2d-x</i>	13,890	cocos2d/cocos2d-x
<i>codeblocks</i>	7,484	jenslody/codeblocks
<i>codelite</i>	7,918	eranif/codelite
<i>CRYENGINE</i>	1,092	CRYTEK/CRYENGINE
<i>CTK</i>	2,823	commontk/CTK
<i>Dealii</i>	20,340	dealii/dealii
<i>distortos</i>	2,498	DISTORTEC/distortos
<i>Dlib</i>	4,366	davisking/dlib
<i>engine</i>	2,078	flutter/engine
<i>folly</i>	3,277	facebook/folly
<i>GamePlay</i>	1,354	gameplay3d/GamePlay
<i>gnuradio</i>	3,191	gnuradio/gnuradio
<i>grieffly</i>	1,074	grieffly/grieffly
<i>irods</i>	2,860	irods/irods
<i>json</i>	1,049	nlohmann/json
<i>kdevelop</i>	16,663	KDE/kdevelop
<i>kokkos</i>	745	kokkos/kokkos
<i>Mantella</i>	1,320	Mantella/Mantella
<i>Mongo</i>	26,345	mongodb/mongo
<i>MultiMC5</i>	1,267	MultiMC/MultiMC5
<i>natron</i>	10,064	MrKepzie/Natron
<i>newton-dynamics</i>	3,124	MADEAPPS/newton-dynamics
<i>nix</i>	2,704	NixOS/nix
<i>ogre</i>	5,280	OGRE/Cave/ogre
<i>oiio</i>	2,445	OpenImageIO/oiio
<i>ola</i>	4,470	OpenLightingProject/ola
<i>opencv</i>	11,328	opencv/opencv
<i>openFrameworks</i>	6,620	openframeworks/openFrameworks

Table 8.5. Selected systems (continued).

System	# Revisions	Location
<i>Plasma</i>	1,395	H-uru/Plasma
<i>ppsspp</i>	15,272	hrydgard/ppsspp
<i>Qt</i>	28,069	qt/qt
<i>QuantLib</i>	8,526	lballabio/QuantLib
<i>Rcpp</i>	1,947	RcppCore/Rcpp
<i>rdkit</i>	2,474	rdkit/rdkit
<i>stellarium</i>	8,677	Stellarium/stellarium
<i>supertux</i>	4,283	SuperTux/supertux
<i>swift</i>	30,343	apple/swift
<i>tera</i>	832	baidu/tera
<i>tfs-old-svn</i>	4,665	otland/tfs-old-svn
<i>Urho3D</i>	5,225	urho3d/Urho3D
<i>xbmc</i>	25,280	xbmc/xbmc
Total	445,255	

for each revision of the default branch is computed (parts 1 and 2). This is done via a short Python program written specifically for this task. In order to generate the srcDiff archives for each revision, the Python program generates a list of each revision and the modified C++ files contained within each of those revisions (part 1). To generate this list, the Python program uses `git-log`. With the correct options, `git-log` will report (in a manner that can be processed by the Python program) all modified files and their associated revisions. Since we are interested in changes to existing methods and added and delete files can only contain added and deleted methods, added and deleted files are ignored. For similar

reasons, copied and renamed files are also ignored⁶. In addition, `git-log` is set to not report merge revisions. As `git-log` reports modified files irrespective of contents, when processing the report provided by `git-log`, the Python program uses file extensions to identify C++ files and filter out non-C++ files. The file extensions explicitly supported by srcML for C++ are used with `.h` (a typical C-language extension that is still used in C++) treated as C++ code. Then, the Python program runs `srcDiff` (without any approximation) on all the identified modified C++ files for each revision and generates a single `srcDiff` archive for each revision (part 2).

Once the `srcDiff` archive is created for each revision, the data can be examined for stereotype changes (parts 3 and 4). To examine, the `srcDiff` files, a separate Python program was written to parse the `srcDiff` archive, identify and collect the signatures of changed methods (part 3), and then apply `StereoCode` to compute the original and modified method stereotypes for the changed method (part 4). In order to identify changed methods and collect their signatures, a Simple API for XML (SAX) [SAX 2001] parsing technique is used. A method is marked as changed and the signature collected if, it contains a change to the text (added, deleted, or modified) comprising the method, the text changed is not whitespace or part of a comment, and the method itself is not inserted or deleted.

⁶ A renamed file may also be modified. Git has heuristics to detect these, however, for this experiment only files explicitly reported by `git-log` as modified are used. Type changed, unmerged, had their pairings broken, and unknown status files are also ignored.

Whitespace and comments (i.e., non-source code) changes are ignored, as they are non-functional/stylistic changes. In addition to the signature, for each changed method, the line number, class the method belongs to if part of a class definition, and the file containing the method are also collected⁷. These additional attributes are collected to make a unique ID for each changed method.

After the information for each changed method is collected, `srcml` is used on the `srcDiff` archive to generate the `srcML` archive of the original and modified versions. Then, `StereoCode` is used on the original and modified `srcML` archives to compute the stereotype of each method. `StereoCode` can report these in a CSV format containing the method stereotype and the same ID information collected for the changed methods (line number, file name, class name, and method signature). This is used to lookup the original and modified stereotypes. The reason that `StereoCode` is applied to the entire file as opposed to just the original and modified version of each method, is to avoid error due to any dependency `StereoCode` may have on contextual information. For example, if a method is defined within a class definition, and `StereoCode` is only applied to the method, then `StereoCode` will fail to recognize that the method is a method because of

⁷ As they can vary between the original and modified versions of code, both the original and modified versions of each of the ID elements (signature, line number, class name, and file name) is gathered.

the lack of any identification that it is so (i.e., has a class namespace as part of the name or is in a class definition).

Finally, information is recorded in two ways. First, each instance (stereotypes and ID information), is recorded to disk, and secondly running totals on the counts of the number of times each stereotype transition occurred is updated.

All programs written for the data collection process were tested for and verified for accuracy.

8.4 Results and Discussion

Data was collected from all investigated systems. In Section 8.4.1, empirical results are presented and in Section 8.4.2, the manual investigation results are presented.

8.4.1 Empirical Results

The full data is too large to put in the dissertation format, so for the full data see http://sdml.cs.kent.edu/srcDiff/dissertation/srcdiff_applications/. Between all systems, there was a total of 1,361,348 changed methods resulting in 2,007 unique stereotype transitions. The top ten stereotype transitions are presented in Table 8.6. The from-column is the original version stereotype, to-column is the modified version stereotype, count-column is the number of occurrences, and %-column is the percentage out of all changed methods. The percentage drops sharply from *collaborator command* to itself being 43.7% of all transitions, down to 0.9% for *collaborator non_void_command nonconstget* to itself. The majority (1,243) of the unique transitions occurred only 10 times or less. All the top ten transitions showed no change in stereotype (i.e., agnostic to change).

Table 8.6. Top ten most common stereotype transitions. from-column is original version stereotype, to-column is the modified version stereotype, count is the number of times it occurred, and % is the percentage of occurrence out of the all transitions.

from	to	count	%
collaborator command	collaborator command	595,085	43.7%
collaborator non void command	collaborator non void command	164,289	12.1%
collaborator property	collaborator property	76,635	5.6%
command	command	65,473	4.8%
collaborator	collaborator	63,473	4.7%
collaborator voidaccessor	collaborator voidaccessor	49,312	3.6%
collaborator non void command factory	collaborator non void command factory	40,183	3.0%
collaborator get	collaborator get	23,614	1.7%
collaborator predicate	collaborator predicate	21,425	1.6%
collaborator non_void_command nonconstget	collaborator non_void_command nonconstget	11,926	0.9%

The top-ten transitions where the stereotype changed are shown in Table 8.7. The from-column is the original version stereotype, to-column is modified version stereotype, count is the number of occurrences, and %-column is the percentage of times it occurred out of all transitions. For the most part, the transitions are neutral or positive (e.g., *collaborator stateless property* to *collaborator property*). The two types of transitions that maybe negative transitions are *collaborator command* to *collaborator* as the method may no longer be contributing much and *collaborator non-void-command* to *collaborator property* which indicates a significant deviation in behavior.

Also of note, each of the most frequent stereotype transitions with changes in stereotype are rare. Collectively, all stereotype transitions with changes in stereotype from

the original to modified versions is about 9.4%. This means that 90.6% of method changes resulted in no change to stereotype. With this we can now answer **RQ1**, method stereotypes change infrequently due to method changes and thus, method stereotypes are very stable.

Table 8.7. Top ten stereotype type transitions where the stereotype changed. from-column is the original version stereotype, to-column is modified version stereotype, count-column is the number of occurrences, and %-column is the percentage it occurred out of all transitions.

from	to	count	%
collaborator	collaborator command	7,633	0.6%
command	collaborator command	6,327	0.5%
collaborator command	collaborator	4,761	0.3%
collaborator command	command	4,358	0.3%
collaborator	collaborator non_void_command	2,407	0.2%
collaborator non_void_command	collaborator command	2,257	0.2%
collaborator set	collaborator command	1,907	0.1%
collaborator non_void_command	collaborator property	1,889	0.1%
collaborator get	collaborator property	1,815	0.1%
collaborator stateless property	collaborator property	1,658	0.1%

We now turn to how stereotypes change when they change. Over, 71% of the time that a non-const *get* method changed, it changed to a *get*. In fact, it is almost as common for a non-const *get* method to change into a *get* method, as it is to remain a non-const *get*. The opposite (*get* to non-const *get*) is exceedingly rare (22 total instances over all projects). That is, *get* methods are mistakenly written as non-const *get* methods and then corrected at a later date.

When a method stereotype changed, 52.7% of the time it only inserted or deleted one or more stereotypes. A summarization of the top-10 deleted and inserted groups of stereotypes are in Table 8.8 and Table 8.9 respectively. For both, the left-column is the stereotypes inserted or deleted, the middle-column is the number of occurrences, and the right-column is the percentage out of all the times method stereotypes changed (not the same between the original and modified versions). Also, for both, the most common consisted only of inserting or deleting a single stereotype. The addition of another stereotype (29.1%) is more frequent than removal of a stereotype (23.6%). In either case, *collaborator* is the most commonly inserted and deleted stereotype. When just considering method stereotypes that purely inserted or deleted stereotypes, this points to increased collaboration between classes in a system over time (the remaining pure insert and delete data that is not shown here does not offset this trend).

Table 8.8. Top ten purely deleted stereotypes. Left-column is the stereotypes deleted, the middle-column is the number of times those stereotypes were the only change in method stereotype, and the right-column is the percent out of all method stereotypes that changed.

stereotype	count	%
<i>collaborator</i>	8,496	6.7%
<i>stateless</i>	7,498	5.9%
<i>command</i>	4,859	3.8%
<i>non_void_command</i>	2,283	1.8%
<i>nonconstget</i>	1,891	1.5%
<i>factory</i>	1,479	1.2%
<i>collaborational_command</i>	1,280	1.0%
<i>empty</i>	356	0.3%
<i>set</i>	269	0.2%
<i>pure_stateless</i>	242	0.2%

Table 8.9. Top ten purely inserted stereotypes. Left-column is the stereotypes inserted, the middle-column is the number of times those stereotypes were the only change in method stereotype, and the right-column is the percent out of all method stereotypes that changed.

stereotype	count	%
<i>collaborator</i>	12,532	9.8%
<i>command</i>	7,705	6.0%
<i>stateless</i>	5,263	4.1%
<i>non_void_command</i>	3,618	2.8%
<i>nonconstget</i>	1,423	1.1%
<i>collaborational_property</i>	1,126	0.9%
<i>property</i>	1,037	0.8%
<i>factory</i>	1,000	0.8%
<i>collaborational_command</i>	827	0.6%
<i>predicate</i>	415	0.3%

The remaining times a stereotype changed was 19.5% completely changed from one stereotype to another, and 27.8% one or more stereotypes were replaced with one or more other stereotypes. Table 8.10 and Table 8.11 provide the top ten completely changed method stereotypes and replaced stereotypes respectively.

In Table 8.10, the from-column is the original version method stereotype, the to-column is the modified version method stereotype, the count-column is the number of occurrences, and %-column is the percentage out of all method stereotypes that changed. The most frequently completely changed stereotypes are *set* to *command* (within same category), *unclassified* to *command* (positive), and *non-const get* to *get* (positive). Out of all the top ten, the only possibly negative transitions are those that are no longer able to be classified by `StereoCode`, which may indicate that the methods have become convoluted, lacking a clear abstraction.

Table 8.10. Top ten completely changed stereotypes. The from-column is the original version stereotype, the to-column is the modified version stereotype, count-column is the number of occurrences and %-column is the percentage of occurrence out of all method stereotypes that changed.

from	to	count	%
set	command	1,567	1.2%
unclassified	command	1,519	1.2%
nonconstget	get	1,245	1.0%
unclassified	collaborator	1,077	0.8%
command	unclassified	955	0.7%
collaborator	unclassified	890	0.7%
command	set	856	0.7%
empty	collaborator command	531	0.4%
stateless	unclassified	506	0.4%
empty	command	428	0.3%

Table 8.11. Top ten replaced stereotypes. The from-column is what was replaced, the to-column what it was replaced with, count-column is the number of occurrences, and %-column is the percentage of occurrence out of all method stereotypes that changed.

from	to	count	%
non_void_command	command	3,003	2.4%
non_void_command	property	2,500	2.0%
set	command	1,907	1.5%
get	property	1,861	1.5%
command	non_void_command	1,519	1.2%
nonconstget	get	1,344	1.1%
property	get	1,212	0.9%
command	voidaccessor	1,147	0.9%
command	set	1,051	0.8%
empty	command	895	0.7%

As for Table 8.11, the most common replacement of stereotypes are *non-void-command* with *command* (neutral) or *property* (which, is worth looking into, as it could be adding a missing `const` or something more serious). The remaining are largely replacements from within the same category. With the exceptions, *empty* to *command* is a positive transition, and *command void-accessor* crosses categories and may be worth investigation.

In essence, method's do not change stereotypes very often. When they do change, most of the most frequent types of changes are of little concern. There are, however, a few among the top-ten that may be worth investigating. Due to their relative rarity, an automatic detection and reporting/alerting tool may be of use. The design of which, we leave for future work.

8.4.2 Manual Investigation

In Section 8.2, we presented a theory on the consequences in a method's stereotype changing. In Section 8.4.1, we showed that method stereotypes are stable to method change with a method's stereotype changed roughly 10% of the time. In addition, the most frequent changes in method stereotype are generally safe and unsuspecting.

In this section, we manually investigate method changes that caused changes to method stereotypes. This method investigation utilized a preliminary copy of the empirical results. The approach taken is as follows. First, the developer of method stereotypes was contacted and asked to make notes about the changes in method stereotypes that we encountered. Next, we looked at one revision for each of the changes in method stereotype noted by the method stereotype developer, and stopped when we reached ten or less

instances of a particular method stereotype change. Manual investigation is a slow and tedious process. In order to make the investigation process more manageable srcDiff's unified difference view and StereoCode were used to locate the changed methods.

The results of the manual investigation are fifty-seven methods contained within thirty-five revisions from eight of the investigated repositories. Some revisions contained multiple versions of the same stereotype change. They were also mostly identical changes to the methods. In order that one group is not over-represented by the same change in method stereotype only one instance from each is used. One revision contained a srcDiff error where the wrong methods were paired together resulting in an incorrect report of a method-stereotype change. The incorrect srcDiff output was unrelated to the srcDiff rules and presented algorithm. The fault is fixed for the empirical results presented in the previous section. Another contained a srcML error. In total, there were 33 changes in method stereotypes investigated. Table 8.12 presents a summary of the results with each of the changes in method stereotype grouped according to the taxonomy presented in Section 8.2. *Other* consisted of insignificant changes not included in the taxonomy (four instances of changes within the same stereotype category and one *command collaborator* to *collaborator*). In addition, one method change included both the *Cross Stereotype Boundary* and *Multiple Category Stereotype*, and is counted as part of both. For each of the categories present, a count of the number of transitions belonging to those categories along with a count and percentage of the number of times those changed methods were considered of no concern (*Benign*) or worthy of inspection by a lead developer or manager (*Suspicious*). The *Benign* category were typically neutral changes or ones that fixed a bug

or code smell. *Suspicious* changes were changes that were clearly wrong or indicated a problem with the function itself. With corrections for the one change counted twice, 32% of the changes in method stereotype were *Suspicious*, while 68% were *Benign*.

In agreement with the taxonomy of Section 8.2, *Remove Degenerate*, *Add Collaborational*, *Remove Collaborational*, *Non-const get* (convert from non-const *get* to *get*), and *Other* were not negative indicators. The one instance of stereotype becoming *unclassified* was suspicious (method is poorly designed). *Cross Stereotype Boundaries*, and *Add/Remove/Replace Categories* (both those that become ones and those that lost the multiple categories) were mixed. That is, the method stereotype change belonging to the *Cross Stereotype Boundaries* and *Add/Remove/Replace Categories* are indicators of problems with the method design or change itself, but only a portion of the time.

Table 8.12. Summary of manual investigation. The category-column contains method stereotype change categories. Count is the number of times that category appeared. The suspicious-column contains the count and percent of the number of times that category had a change that should be investigated more deeply, and the benign-column is the count and percentage of the times that category was of no concern.

Category	Count	Suspicious		Benign	
		#	%	#	%
<i>To Unclassified</i>	1	1	100%	0	0%
<i>From Non-const get</i>	1	0	0%	1	100%
<i>Add Collaborational</i>	1	0	0%	1	100%
<i>Remove Collaborational</i>	1	0	0%	1	100%
<i>Remove Degenerate</i>	2	0	0%	2	100%
<i>Add Categories</i>	8	4	50%	4	50%
<i>Remove Categories</i>	4	2	50%	2	50%
<i>Cross Stereotype Boundaries</i>	11	4	36%	7	64%
<i>Other</i>	5	0	0%	5	100%
<i>Total</i>	33	11	32%	23	68%

From this, we can conclude that although not perfect, certain categories of method-stereotype changes (*To Unclassified*, *Cross Stereotype Boundaries*, and *Add/Remove/Replace Categories*) can be useful indicators for method design problems or inappropriate method changes. That is, aided by the fact that changes in method stereotype by themselves are rare with specific categories being even rarer, investigation by a developer would not require substantial time.

From the manual investigation, it is noted that a finer-grained categorization may produce more accurate predictions. For example, many of the *Cross Stereotype Boundaries* did so because the method was made `const` or the `const` was removed. The addition of `const` in these cases, allowed StereoCode to make a correct assessment of behavior and were positive changes, while the removal of `const` was more negative. That is, refining the *Cross Stereotype Boundaries* into cases where adding `const` or removing `const` will increase prediction ability. Similar types of improvements may be made to *Add/Remove/Replace Categories*.

With this we finish answering **RQ2**. That is, changes in method stereotypes are rare. When they do change, many categories of changes are of little concern. However, other categories of changes in method stereotypes (*To Unclassified*, *Cross Stereotype Boundaries*, and *Add/Remove/Replace Categories*) can, a portion of the time, indicate method design problems and inappropriate changes to code.

8.5 Threats to Validity

The following threats to validity are identified with the study.

First, srcDiff, srcML, Git, and other tooling used as part of the study are complicated pieces of software, and as such contain faults. The parsing of C++ code is notoriously difficult, in part, due to preprocessor statements making C++ a non-context free grammar. The extent to which the tools report inappropriate ASTs is not known, yet srcML has a rigorous test suite and srcDiff is also well tested. Despite this, a negligible number of revisions failed to report a correct XML document (possible due to program crash, etc.). A negligible number of commits will not affect the vast amount of data collected.

Only standard C++ extensions were used to locate C++ files. Most of the projects used only these extensions, however, some used unusual extensions which possibly contained C++ code. For example, a C++ extension followed by `.in`, which would possibly denote a file that is used to generate code as part of the build system, and `.hpp`, which could possibly contain template code. These were only noticed in a few of the repositories. For future work, these files can be examined more closely and a determination made as to whether they should be included as part of the data collected.

The `.h` extension was treated as C++, however, it is used both as a C and C++ header extension. Some of the projects included both C and C++ code and used `.h` for both. As C++ is largely a super-set of C (with a few exceptions), the parsing of C code as C++ code is a minor threat.

srcML provides support for the markup of Qt extensions. Currently, these are always on. Some code may have been incorrectly converted to srcML when a non-Qt

project used an identifier that is part of the Qt extensions. In the future, it is planned for srcML to support Qt extensions optionally.

Some of the projects have committed code that was auto-generated by some tooling. Auto-generated files may exist throughout the history of a project or may have appeared and disappeared at any point in the history of the software. For the most part, we are uninterested in changes to auto-generated code, however, as they can appear at any point in history, they may be difficult to find. Investigation into auto-detection of all generated files through the history of projects can be a valid research topic. With the systems used as part of the study, two instances of auto-generated files were found. One was in `ppspp`, and the other in `codelite`. They were found due to srcDiff's current difficulty in handling files with, for example, large static arrays and string literals, as they result in elements with exceedingly large numbers of children and text nodes for which Myers' algorithm $O(ND)$ algorithm [Myers 1986] is not quick enough. In these two cases, the file list generation part of the data collection was instructed to ignore these auto-generated files.

Only C++ was used as part of the study. Results may be somewhat different for other languages. Currently, StereoCode only supports C++, and is thus a limitation.

Lastly, manual investigation is subject to human error. Mitigating this, the process was done carefully, and detailed notes taken. When the encoder was unsure about the change, another expert programmer was consulted. In addition, a breadth instead of depth look at the changes in method stereotypes is performed. One sample is certainly not representative of every possible change that could induce that particular change in

stereotype. That is, we can only conclude that changes in method stereotype can indicate problems, however, we cannot give the extent to which they do.

8.6 Conclusion

We investigated changes in method stereotype theoretically, and via a large empirical study and small manual investigation. A taxonomy of changes in method stereotypes was presented, and the manual investigation studied many different types of changes in stereotype belonging to those categories. We conclude that certain categories of method stereotype changes can be indicators of poor method design or of inappropriate changes to the methods themselves. In the empirical study, we conclude that changes in method stereotype are rare, and report various statistics on how and what changes when a method stereotype changes.

CHAPTER 9

Investigating Code Rewrite/Replacement and Code Conversion

This chapter presents the second application of srcDiff to a software engineering problem. Two aspects of programming (rewriting and replacing code and conversion of code from one element to another) are investigated. Both code replace and code conversion are explicitly supported by srcDiff and are easily identifiable in a srcDiff archive. As they are not supported directly (or at all) by other syntactical differencers, this study is a measure of how often srcDiff is able to capture information missed or marked incorrectly by other syntactical differencers.

The study answers the following research questions:

- **RQ1:** How commonly do developers replace and rewrite code?
- **RQ2:** What are the consequences of replace and rewriting code?
- **RQ3:** How often do developers convert code from one type to another?
- **RQ4:** What are the consequence of converting code?

The remainder of this chapter is laid out as following. Section 9.1 introduced code rewrite and replacement, how it is supported via srcDiff, and the significance. Section 9.2 explains code replacement, how srcDiff supports code conversion, and the significance. Section 9.3 explains the data collection procedures. Section 9.4 presented the results and discussion, the threats to validity are given in Section 9.5, and Section 9.6 presents the overall conclusions.

9.1 Rewrite and Replacement

When developing code, developers delete portions of code, expressions, statements, function, etc. and insert separate code in that location. This is what we term rewritten or replaced code. From now on, we will use rewrite and replace interchangeably. Figure 9.1 shows an example code replacement with the original and modified code on the top, left and right respectively, and the srcDiff of the change on the bottom. This example is a simplified version of Martin Fowler's example of a *Substitute Algorithm* refactoring [Fowler 1999]. The control-group (initialization, condition and increment) of the for-statement is the same between both versions, however, the body of the for-statement has been substituted with a different algorithm, i.e., it is replaced.

Original	Modified
<pre>for (int i = 0; i < people.length; i++) { if (people[i].equals ("Don")){ return "Don"; } if (people[i].equals ("John")){ return "John"; } if (people[i].equals ("Kent")){ return "Kent"; } }</pre>	<pre>List candidates = Arrays.asList(new String[] {"Don", , "John", "Kent"}); for (int i = 0; i < people.length; i++) if (candidates.contains(people[i])) return people[i];</pre>
srcDiff	
<pre>List candidates = Arrays.asList(new String[] {"Don", "John", "Kent"}); for (int i = 0; i < people.length; i++) if (people[i].equals ("Don")){ return "Don"; } if (people[i].equals ("John")){ return "John"; } if (people[i].equals ("Kent")){ return "Kent"; } } if (candidates.contains(people[i])) return people[i];</pre>	

Figure 9.1. Example code replacement. The body of the for-statement is completely deleted and replaced with a different body.

The significance of this is that srcDiff is able to recognize the replacement, while a tree-edit distance based algorithm will achieve a shorter edit script by marking the body and if-statement components as modified in some manner. That is, by measuring how often replacement occurs, we get an additional measure of how much srcDiff outperforms tree-edit distance algorithms for syntactic differencing (**RQ2**).

Replacement can occur at any level of the Abstract Syntax Tree (AST). That is, at the file-level such as function replacement, at statement level, and at the expression/sub-expression level (e.g., identifier rename). In addition, AST elements can be replaced with separate AST elements, e.g., and if-statement can be replaced with a declaration-statement, and the number of elements deleted and inserted does not have to be equal. That is, a declaration-statement, if-statement, and while-statement, might be replaced with an expression-statement. The major restriction is that replaced elements are children of some common node (i.e., siblings) that occur in the same location.

srcDiff identifies these as a side-effect of the srcDiff rules. That is, when the rules fail and there are inserted and deleted children that are not identified as nested, converted, or previous versions of each other, they are marked as replacements. Figure 9.2 shows an example of how srcDiff reports code replacement. The delete portion of the replacement is marked with a `diff:delete` tag and type `replace`. Following after, the insert portion is reported with a `diff:insert` tag and type `replace`. In the example in Figure 9.2, an expression-statement, if-statement, and while-statement are replaced with a declaration-statement and a separate expression-statement. The contents of the statements are left out, and colors added for readability.

srcDiff
<pre> <unit xmlns="http://www.srcML.org/srcML/src" xmlns:diff="http://www.srcML.org/srcDiff /" revision="0.9.5" language="C++"> ... <diff:delete type="replace"><expr_stmt></expr_stmt><diff:ws> </diff:ws><if></if><diff:ws> </diff:ws><while></while></diff:delete><diff:insert type="replace"><decl_stmt></decl_stmt><diff:ws> </diff:ws><expr_stmt></expr_stmt></diff:insert> ... </unit> </pre>

Figure 9.2. Simplified example of srcDiff markup of code replacement. Red is delete and green is insert. A `diff:delete` tag and `diff:insert` tag are issued consecutively (one after the other) and they both are given the type `replace`. Here an expression-statement, if-statement, and while-statement are replaced with a declaration-statement and a different expression-statement.

9.2 Conversion

When writing code, developers occasionally switch from using one syntactic element to another syntactic element of a separate type. As an example, a developer may have initially written a C++ `struct` with the intention of it only being used to store plain-old-data (similar to C usage). However, over time additional functionality such as constructors and methods are added to the `struct` such that it makes more sense to the developer to declare the same `struct` as a `class` instead. As another example, a statement originally written as a `for`-statement, may be refactored at a later date to be a `while`-statement.

We call these code conversions. A code example is provided in Figure 9.3. In Figure 9.3, the original code is on the top-left, the modified code on the top-right, and the srcDiff is on the bottom. The original `for`-statement is converted into a `while` statement in the modified with the body of the `for`-statement left largely the same between both versions.

Original	Modified
<pre>int itemCount = d.items.count(); for(int i = itemCount-1; i >= 0; --i) { Item &sbItem = d.items[i]; if (sbItems.widget() == widget) { // several common lines } }</pre>	<pre>int itemCount = d.items.count() - 1; while(i >= 0) { Item &sbItem = d.items[i]; if (sbItems.widget() == widget) { // several common lines } --i; }</pre>
srcDiff	
<pre>int itemCount = d.items.count() - 1; forwhile(int i = itemCount-1; (i >= 0;) --i) { Item &sbItem = d.items[i]; if (sbItems.widget() == widget) { // several common lines } --i; }</pre>	

Figure 9.3. Example code conversion. The original code is on the top-left, modified top-right, and srcDiff on the bottom. In the example, the for-statement was converted into an equivalent while-statement. The body of the for-statement is largely unchanged.

srcDiff directly supports the detection and markup of code conversions. It does so for three specific reasons. First, for example, an if-statement and while statement are different AST elements, so a syntactic difference may never identify that the one element was converted to the other and identify the code as a replacement instead. Secondly, for example, not all instances of for-statements replaced with while-statements are conversions. That is, a tree-edit based syntactic difference could over-zealously pair parts of both elements and arrive at a difficult to understand delta. Lastly, without specific markup it can be difficult to detect that one element was converted to another, and to differentiate a convert with some other type of change operation (such as nesting). As an example, Figure 9.4 shows GumTree’s output on a simple code conversion [Falleri et al. 2014]. The example shows the original while-statement (top-left) converted into a for-statement (top-right). The bottom shows GumTree’s output. As GumTree does not support

code conversion, the while-statement is marked as deleted, for-statement inserted, and a series of moves between the two.

Original	Modified
<pre>int i = 0; while(i < 10) { System.out.println(i); ++i; }</pre>	<pre>for(int i = 0; i < 10; ++i) { System.out.println(i); }</pre>
GumTree	
<pre>int i = 0; while(i < 10) { System.out.println(i); ++i; }</pre>	<pre>for(int i = 0; i < 10; ++i) { System.out.println(i); }</pre>

Figure 9.4. Output of GumTree on a simple code conversion. Red is delete, yellow-green insert, and blue is move. A while-statement is converted to a for-statement. GumTree marks the while-statement deleted and for-statement inserted and a series of moves between the two.

Table 9.1 shows the conversions supported by srcDiff (repeated from Section 4.3.2). The left column is the category and the right column are the syntactic elements that are convertible with each other. foreach-statements are a Qt macro that were added to the *Logical* category beyond what was presented in Section 4.3.2.

Once detected, srcDiff outputs the conversion with a specific signature and markup. Figure 9.5 shows an example of a code conversion marked up in srcDiff. Colors are added and inner parts of the statements removed to simplify reading. The example is of a while-statement converted to a for-statement. In addition to the tags being marked with the type `convert`, a `convert` has a very specific pattern. It is always a `diff:delete` around the

original version element (while-statement) and keyword if present followed by a `diff:insert` around the modified version element (for-statement) and keyword if present. After this, the remaining parts then are marked with additional `diff:common`, `diff:insert`, and `diff:delete` tags as required.

Table 9.1. Code conversion supported by srcDiff. Left is category and right are the syntactic elements that can be converted to each other.

Rule	Syntactic Category
<i>Class</i>	class, struct, union, enum
<i>Access Regions</i>	private, protected, public
<i>Logical</i>	if, while, for, foreach
<i>Else</i>	else, elseif
<i>Statement</i>	expr_stmt, decl_stmt, return

srcDiff
<pre> <unit xmlns="http://www.srcML.org/srcML/src" xmlns:diff="http://www.srcML.org/srcDiff /" revision="0.9.5" language="C++"> ... <diff:delete type="convert"><while>while</diff:delete><diff:insert type="convert"><for>for ... </for></diff:insert></while></diff:delete> ... </unit> </pre>

Figure 9.5. Example markup of a code conversion in srcDiff. Red is delete and green is insert. A `diff:delete` is wrapped around the original syntactic element (and keyword if present) with the type `convert`, and a `diff:insert` is wrapped around the modified syntactic element (and keyword if present) with type `convert`. The contents are then marked as common, inserted, and deleted as necessary.

9.3 Data Collection

This section is largely a repeat of Section 8.3. It is reproduced here to make the chapter more self-contained. The write-up deviates mostly after the discussion of how the srcDiff archives are created.

To investigate changes in code conversion and replacement, 50 open-source software systems are selected for study. Table 9.2 and Table 9.3 provide a listing of the selected systems (left-column). The number of revisions in each that were non-merges and contained a modified C++ file are in the middle column, and the right column has the location from which the repository was taken. When the repository location is not a fully qualified URL, the repository comes from GitHub and has the following complete location: `https://github.com/{Location}.git`. The selected systems cover many different domains and repository sizes and therefore, provide a good representative sample.

The data collection process consists of the following parts:

1. Identification of all revisions and files changed in those revisions.
2. Application of srcDiff to the original and modified version of all changed C++ files to generate srcDiff archives of each revision.
3. Analysis of the srcDiff archives to locate code replacement.
4. Analysis of the srcDiff archives to locate code conversion.

More specifically, data is collected for each system in the following manner. First, a local clone of the repository is taken. Next, a srcDiff archive of every modified C++ file for each revision of the default branch is computed (parts 1 and 2). This is done via a short Python program written specifically for this task. In order to generate the srcDiff archives for each revision, the Python program generates a list of each revision and the modified C++ files contained within each of those revisions (part 1). To generate this list, the Python program uses `git-log`. With the correct options, `git-log` will report (in a manner that can be processed by the Python program) all modified files and their associated revisions.

Table 9.2. List of selected software systems by alphabetical order. Left column is system, middle is number of revisions with modified C++ files, and right is the repository location. Unqualified locations are from GitHub: <https://github.com/{Location}.git>.

System	# Revisions	Location
<i>antimony</i>	1,737	mkeeter/antimony
<i>appleseed</i>	5,369	appleseedhq/appleseed
<i>BansheeEngine</i>	3,058	BearishSun/BansheeEngine
<i>bitcoin</i>	5,785	bitcoin/bitcoin
<i>blender</i>	24,215	git://git.blender.org/blender.git
<i>cgal</i>	39,583	CGAL/cgal
<i>ChaiScript</i>	1,312	ChaiScript/ChaiScrip
<i>citra</i>	2,908	citra-emu/citra
<i>Clang</i>	56,631	http://llvm.org/git/clang
<i>cocos2d-x</i>	13,890	cocos2d/cocos2d-x
<i>codeblocks</i>	7,484	jenslody/codeblocks
<i>codelite</i>	7,918	eranif/codelite
<i>CRYENGINE</i>	1,092	CRYTEK/CRYENGINE
<i>CTK</i>	2,823	commontk/CTK
<i>Dealii</i>	20,340	dealii/dealii
<i>distortos</i>	2,498	DISTORTEC/distortos
<i>Dlib</i>	4,366	davisking/dlib
<i>engine</i>	2,078	flutter/engine
<i>folly</i>	3,277	facebook/folly
<i>GamePlay</i>	1,354	gameplay3d/GamePlay
<i>gnuradio</i>	3,191	gnuradio/gnuradio
<i>grieffly</i>	1,074	grieffly/grieffly
<i>irods</i>	2,860	irods/irods
<i>json</i>	1,049	nlohmann/json
<i>kdevelop</i>	16,663	KDE/kdevelop
<i>kokkos</i>	745	kokkos/kokkos
<i>Mantella</i>	1,320	Mantella/Mantella
<i>Mongo</i>	26,345	mongodb/mongo
<i>MultiMC5</i>	1,267	MultiMC/MultiMC5
<i>natron</i>	10,064	MrKepzie/Natron
<i>newton-dynamics</i>	3,124	MADEAPPS/newton-dynamics
<i>nix</i>	2,704	NixOS/nix
<i>ogre</i>	5,280	OGRECave/ogre
<i>oiio</i>	2,445	OpenImageIO/oiio
<i>ola</i>	4,470	OpenLightingProject/ola
<i>opencv</i>	11,328	opencv/opencv
<i>openFrameworks</i>	6,620	openframeworks/openFrameworks

Table 9.3. Selected systems (continued).

System	# Revisions	Location
<i>Plasma</i>	1,395	H-uru/Plasma
<i>ppsspp</i>	15,272	hrydgard/ppsspp
<i>Qt</i>	28,069	qt/qt
<i>QuantLib</i>	8,526	lballabio/QuantLib
<i>Rcpp</i>	1,947	RcppCore/Rcpp
<i>rdkit</i>	2,474	rdkit/rdkit
<i>stellarium</i>	8,677	Stellarium/stellarium
<i>supertux</i>	4,283	SuperTux/supertux
<i>swift</i>	30,343	apple/swift
<i>tera</i>	832	baidu/tera
<i>tfs-old-svn</i>	4,665	otland/tfs-old-svn
<i>Urho3D</i>	5,225	urho3d/Urho3D
<i>xbmc</i>	25,280	xbmc/xbmc
Total	445,255	

Since we are interested in code conversion and code replace which only exist in modified files, added and deleted files are ignored. For similar reasons, copied and renamed files are also ignored⁸. In addition, `git-log` is set to not report merge revisions. As `git-log` reports modified files irrespective of contents, when processing the report provided by `git-log`, the Python program uses file extensions to identify C++ files and filter out non-C++ files. The file extensions explicitly supported by srcML for C++ are used with `.h` (a typical C-language extension that is still used in C++) treated as C++ code. Then, the Python program runs `srcDiff` (without any approximation) on all the identified modified

⁸ A renamed file may also be modified. Git has heuristics to detect these, however, for this experiment only files explicitly reported by `git-log` as modified are used. Type changed, unmerged, had their pairings broken, and unknown status files are also ignored.

C++ files for each revision and generates a single srcDiff archive for each revision (part 2).

Once the srcDiff archive is created for each revision, the analysis of code replacement (part 3) and code conversion (part 4) is performed. A Python program is created for collecting code replacement and a separate Python program is created to collect code conversions. For code replacement, the Python program records a replace record each time a one `diff:delete/diff::insert` sequence with type `replace` is encountered. What is recorded, is the count of what elements were deleted, a count of what elements were inserted, as well as the original and modified parent elements. The parent element is gathered primarily to tell what kind of replacement occurred when text was replaced with text. For example, if text is replaced with text and both have a `name` tag as a parent, then a rename occurred. For a replace, comments appearing as part of the replace are counted, but whitespace is ignored. In addition, moves that occur as part of a replace are ignored. A replace is not counted unless a non-move and non-whitespace element was part of both the delete and insert parts. These records are then aggregated over all revisions while keeping track of the number of records and revisions. In addition, the number of times there was an element replaced with another one of the same type is recorded. When there are not the same number of elements (e.g., three expression statements inserted and two deleted), the minimum of these is counted (e.g., two). To further differentiate changes to text, changes to text of comments is marked as `comment text`, changes to text of a name as `rename`, changes to literal text as `literal value`, and everything else as `other text`. This information is then reported by the Python program in a series of

three CSV files (one for the deleted elements, one for the inserted elements, and a count of what elements were replaced with themselves) along with the number of revisions and replace records.

For convert analysis, each time a convert is encountered in the srcDiff archive, a count of the number of times the transition occurred is incremented. A map of maps is used to keep track of the transition counts. For example, a convert of a for-statement to while-statement is encountered in the srcDiff archive. Once encountered, the map is indexed first for the for-statement, which results in a second map. Then, the second map is indexed with the while-statement and the count is incremented. The convert counts for all revisions are then aggregated while keeping track of the number of revisions. This is then reported by the Python program in a CSV file.

All data analysis tools were tested to ensure accuracy.

9.4 Results and Discussion

Data was successfully gathered for all fifty systems. As each system resulted in a separate CSV file (one for each type of data collected), the data (of one type) for all systems were combined into a single table using JMP's concatenate table feature. Apple's Numbers program was then used to generate the data presented here. As the full tables are too large to fit into this dissertation format, they are available at http://sdml.cs.kent.edu/srcDiff/dissertation/srcdiff_applications/. The results and discussion for replace and convert are provided separately in Sections 9.4.1 and 9.4.2 respectively.

9.4.1 Replace Results and Discussion

Table 9.4 and Table 9.5 show the number of replaces for each system. The left column is the name of the system with the systems sorted alphabetically. The middle column is the number replaces (i.e., delete and insert pairs) that occurred, and the last column is the average number of replaces per revision (number of replaces divided by number of revisions). In total, there were over 4.8 million replaces across all revisions for all systems with an average of about 11 per revision. That per system average varied between system with distortos having the most per revision (61.5) and antimony having the least (3.0). The correlation between number of replaces and number of revisions is 0.88, and Figure 9.6 shows a plot of the number of revisions for a system on the x-axis with the number of replaces for that system on the y-axis. Both show that the number of replaces varies fairly linear with the number of revisions and is mostly likely fairly consistent.

With this information, we can partially answer **RQ1**. Code replacement as a whole is an extremely common event. This explains the poor results of tree-edit distance based syntactic differencers. The extent of this is, however, relative to the size of the structure that is replaced. That is, a rename is atomic, having only a text token replaced with another text token, and so, there is no error in a tree-edit distance approach, while a statement replace, function replace, or a replace of several different types with each other would show considerable deficiency by the syntactic difference.

Table 9.4. Number of replaces per system (alphabetically). First column is system, the second is the number of replaces, and the average per revision.

System	# Replaces	# Per Revision
<i>antimony</i>	5,242	3.0
<i>appleseed</i>	69,439	12.9
<i>BansheeEngine</i>	87,759	28.7
<i>bitcoin</i>	37,661	6.5
<i>blender</i>	200,596	8.3
<i>cgal</i>	491,979	12.4
<i>ChaiScript</i>	13,717	10.5
<i>citra</i>	21,357	7.3
<i>Clang</i>	357,243	6.3
<i>cocos2d-x</i>	261,194	18.8
<i>codeblocks</i>	87,576	11.7
<i>codelite</i>	107,021	13.5
<i>CRYENGINE</i>	27,786	25.4
<i>CTK</i>	22,806	8.1
<i>Dealii</i>	224,615	11.0
<i>distortos</i>	153,740	61.5
<i>Dlib</i>	23,783	5.4
<i>engine</i>	47,301	22.8
<i>folly</i>	19,864	6.1
<i>GamePlay</i>	32,845	24.3
<i>gnuradio</i>	28,620	9.0
<i>griefly</i>	6,845	6.4
<i>irods</i>	35,208	12.3
<i>json</i>	13,214	12.6
<i>kdevelop</i>	161,016	9.7
<i>kokkos</i>	8,809	11.8
<i>Mantella</i>	19,684	14.9
<i>Mongo</i>	223,300	8.5
<i>MultiMC5</i>	10,805	8.5
<i>natron</i>	194,015	19.3
<i>newton-dynamics</i>	174,221	55.8
<i>nix</i>	15,429	5.7
<i>ogre</i>	78,849	14.9
<i>oiio</i>	21,115	8.6
<i>ola</i>	49,425	11.1
<i>opencv</i>	134,100	11.8
<i>openFrameworks</i>	57,647	8.7
<i>Plasma</i>	66,014	47.3
<i>ppsspp</i>	166,901	10.9
<i>Qt</i>	295,292	10.5

Table 9.5. Number of replaces (continued).

<i>QuantLib</i>	106,641	12.5
<i>Rcpp</i>	39,345	20.2
<i>rdkit</i>	18,951	7.7
<i>stellarium</i>	80,393	9.3
<i>supertux</i>	37,320	8.7
<i>swift</i>	199,229	6.6
<i>tera</i>	10,655	12.8
<i>tfs-old-svn</i>	42,968	9.2
<i>Urho3D</i>	105,204	20.1
<i>xbmc</i>	160,122	6.3
Total	4,854,861	10.9

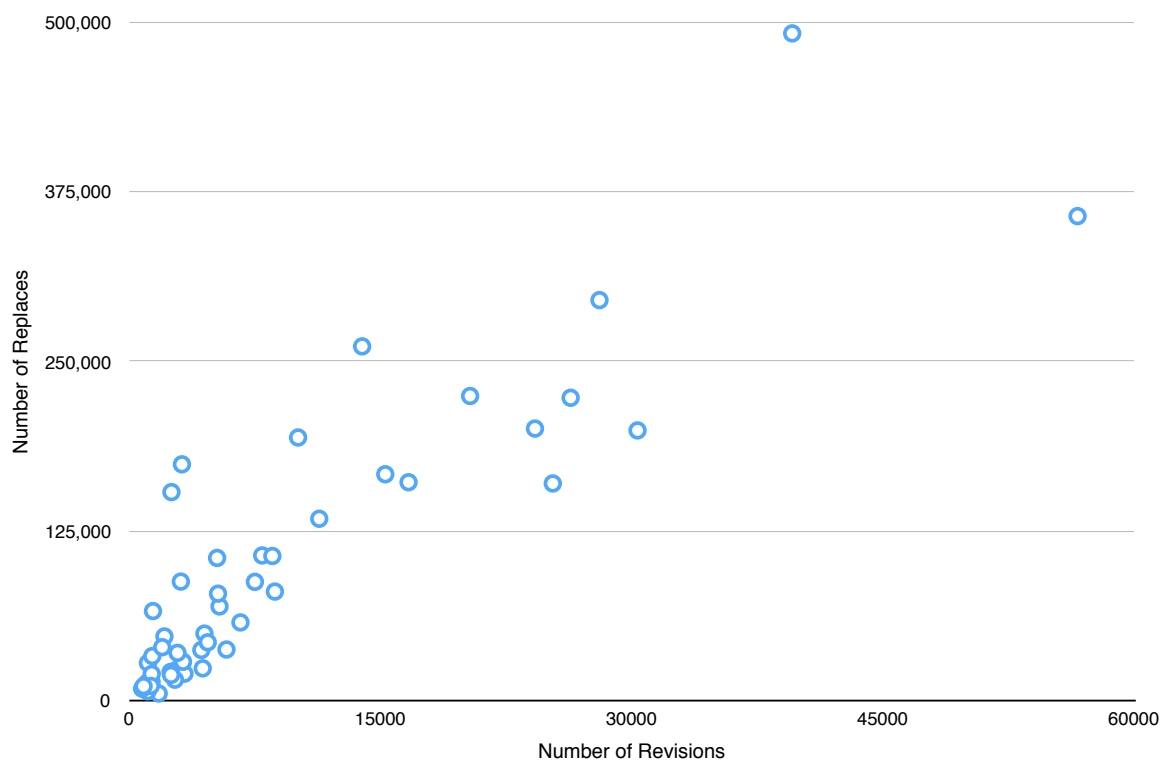


Figure 9.6. The total number of revisions plotted against the number of replaces. X-axis is number of revisions and y-axis is the number of replaces. The result is fairly linear.

Next, the results of what elements are replaced with what elements are reported in Table 9.6, Table 9.7, and Table 9.8. For each of the three tables, the left column contains the element affected, the middle column the number of occurrences, and the last column is the number of occurrence divided by the number of revisions. Table 9.6 reports the elements that were deleted as part of a replace, Table 9.7 reports the elements inserted as part of a replace, and Table 9.8 reports the elements that were part of a replace with an element of the same type (self-replace). For each of the three tables, only the 20 most frequent are given. In addition, the normalized version of each table is also provided in Table 9.9, Table 9.10, and Table 9.11. For the normalized versions, the per revision for each system is computed and the median of all systems is presented. Normalization does not have a major affect on the data. That is, replace per revision is fairly consistent.

In all three cases, text is by far the most frequently replaced code element with over eleven text tokens deleted and inserted per revision (on average). In addition, renames occur about four times a revision (on average). That is, developers perform a lot of renames. Comment text is replaced fairly frequently (which most syntactic differencers do not report). After text, expressions and other subparts of statements are replaced with some relative frequency, followed by several types of statements and then functions.

Expression-statements and declaration-statements are the most frequently replaced statements. For expression-statements there are about 0.50 deleted and 0.42 inserted per revision (on average), which amounts to almost every other revision contained one as part of a replace. An expression-statement is replaced with itself (appeared in both the delete and insert) almost once every five revisions on average. Declaration-statements were

Table 9.6. Deleted elements in a replace. Left column is what element is deleted, the middle column is how many instances were deleted in all replaces, and the third column is the number each is replaced per revision.

Element	Count	Per Revision
<i>text</i>	4,992,841	11.21
<i>name</i>	428,616	0.96
<i>expr</i>	374,995	0.84
<i>expr_stmt</i>	221,339	0.50
<i>operator</i>	212,156	0.48
<i>comment</i>	209,506	0.47
<i>call</i>	166,444	0.37
<i>decl_stmt</i>	148,784	0.33
<i>function</i>	84,241	0.19
<i>if</i>	72,505	0.16
<i>literal</i>	65,426	0.15
<i>parameter</i>	64,408	0.14
<i>cpp:value</i>	60,002	0.13
<i>function_decl</i>	37,802	0.08
<i>return</i>	24,656	0.06
<i>index</i>	23,853	0.05
<i>macro</i>	21,808	0.05
<i>argument_list</i>	21,226	0.05
<i>cpp:macro</i>	20,881	0.05
<i>cpp:define</i>	19,906	0.04

Table 9.7. Inserted elements in a replace. Left column is what element is deleted, the middle column is how many instances were deleted in all replaces, and the third column is the number each is replaced per revision.

Element	Count	Per Revision
<i>text</i>	5,079,077	11.41
<i>name</i>	482,282	1.08
<i>expr</i>	379,892	0.85
<i>comment</i>	248,617	0.56
<i>operator</i>	216,847	0.49
<i>expr_stmt</i>	187,572	0.42
<i>call</i>	173,519	0.39
<i>decl_stmt</i>	154,274	0.35
<i>if</i>	66,061	0.15
<i>function</i>	64,533	0.14
<i>cpp:value</i>	60,009	0.13
<i>parameter</i>	59,126	0.13
<i>function_decl</i>	36,843	0.08
<i>literal</i>	34,263	0.08
<i>argument_list</i>	25,999	0.06
<i>return</i>	24,220	0.05
<i>index</i>	22,695	0.05
<i>macro</i>	22,283	0.05
<i>cpp:macro</i>	20,881	0.05
<i>block</i>	20,158	0.05

Table 9.8. Number times an element was replaced with itself. The left column is the element, the middle column is number of occurrence, and the right column is the number (on average) that occurred in each revision.

Element	Count	Per Revision
<i>rename</i>	1,780,920	4.00
<i>comment text</i>	1,278,269	2.87
<i>other text</i>	413,419	0.93
<i>literal value</i>	332,861	0.75
<i>expr</i>	328,508	0.74
<i>name</i>	156,090	0.35
<i>call</i>	88,444	0.20
<i>expr_stmt</i>	79,809	0.18
<i>cpp:value</i>	59,909	0.13
<i>operator</i>	58,511	0.13
<i>parameter</i>	54,330	0.12
<i>decl_stmt</i>	53,125	0.12
<i>comment</i>	34,739	0.08
<i>if</i>	26,762	0.06
<i>cpp:macro</i>	20,881	0.05
<i>function</i>	16,924	0.04
<i>index</i>	16,667	0.04
<i>return</i>	15,858	0.04
<i>else</i>	10,158	0.02
<i>function_decl</i>	8,771	0.02

Table 9.9. Normalized deleted elements in a replace. The left-column is the deleted element. The right-column (normalized per revision) is calculated as: for each system, each element is normalized by the number of revisions for that system and the median across all systems is taken.

Element	Normalized Per Revision
<i>text</i>	10.65
<i>name</i>	0.84
<i>expr_stmt</i>	0.54
<i>expr</i>	0.44
<i>operator</i>	0.42
<i>comment</i>	0.41
<i>call</i>	0.36
<i>decl_stmt</i>	0.32
<i>function</i>	0.18
<i>if</i>	0.17
<i>literal</i>	0.14
<i>parameter</i>	0.12
<i>function_decl</i>	0.08
<i>index</i>	0.05
<i>return</i>	0.05
<i>macro</i>	0.04
<i>for</i>	0.04
<i>argument_list</i>	0.03
<i>block</i>	0.03
<i>cpp:macro</i>	0.03

Table 9.10. Normalized inserted elements in a replace. The left-column is the inserted element. The right-column (normalized per revision) is calculated as: for each system, each element is normalized by the number of revisions for that system and the median across all systems is taken.

Element	Normalized Per Revision
<i>text</i>	10.41
<i>name</i>	1.00
<i>expr_stmt</i>	0.51
<i>comment</i>	0.46
<i>operator</i>	0.41
<i>expr</i>	0.39
<i>call</i>	0.36
<i>decl_stmt</i>	0.28
<i>function</i>	0.15
<i>if</i>	0.14
<i>parameter</i>	0.11
<i>function_decl</i>	0.09
<i>literal</i>	0.07
<i>return</i>	0.05
<i>index</i>	0.05
<i>argument_list</i>	0.04
<i>macro</i>	0.04
<i>block</i>	0.04
<i>decl</i>	0.03
<i>for</i>	0.03

Table 9.11. Normalized times an element was replaced with itself. The left-column is the element. The right-column (normalized per revision) is calculated as: for each system, each element is normalized by the number of revisions for that system and the median across all systems is taken.

Element	Normalized Per Revision
<i>rename</i>	3.79
<i>comment text</i>	1.86
<i>other text</i>	0.78
<i>literal value</i>	0.56
<i>name</i>	0.32
<i>expr</i>	0.30
<i>expr_stmt</i>	0.19
<i>call</i>	0.17
<i>operator</i>	0.11
<i>decl_stmt</i>	0.11
<i>parameter</i>	0.10
<i>comment</i>	0.06
<i>if</i>	0.06
<i>function</i>	0.04
<i>index</i>	0.04
<i>return</i>	0.03
<i>cpp:macro</i>	0.03
<i>else</i>	0.02
<i>function_decl</i>	0.02
<i>cpp:value</i>	0.01

slightly less frequent with 0.33 deleted and 0.35 inserted per revision on average (about one every three revisions on average). A declaration-statement was replaced with itself (appeared in both the delete and insert) almost once out of eight times on average (0.12). Lastly, functions were deleted as part of a replace 0.19 time per revision on average and inserted 0.14 times per revision on average. A function replaced itself (appeared in both the delete and insert) about once every twenty-five revisions on average.

This means that some expression-level replacements and statement-level expressions occur frequently, and that function replacements are not uncommon. We now turn to answering **RQ1** and **RQ2**. For **RQ1**, code replacement at the text, expression, and statement level are frequently occurrences, and functions are not uncommonly replaced. The consequences of this (**RQ2**) are that replacement is sufficiently common enough at function, statement, and expression level that tree-edit distance algorithms will make inappropriate differences fairly often, which explains a major reason for their failure to arrive at a meaning difference and why they have not seen wide spread adoption.

9.4.2 Convert Results and Discussion

At least one type of code conversion (one transition), was present in all fifty systems. Table 9.12 and Table 9.13 show the total converts for each system (alphabetically). The left column gives the system, the middle column is the total number of conversions of all types that occurred over all revisions, and the last column normalizes the total number of conversion by the number of revisions. In total, there are almost 53,000 conversions between all systems, which averages to almost 12% of revisions containing a conversion (almost 1 in 8 revisions). As for individual systems, GamePlay had the most

conversions per revision (79.5%) and distortos has the smallest number of conversions per revision (3.4%). For the most part, however, the number of conversions per revisions is fairly consistent between systems. To confirm this, the number of revisions was plotted against the number of conversions. The results are shown in Figure 9.7. The x-axis is the number of revisions and the y-axis the total number of conversions. The resulting graph is fairly linear. The correlation between number of revisions and number of conversions is about 0.89, meaning that the number of conversions correlate highly with the number of revisions. From this we answer **RQ3**, concluding that conversions are not an uncommon event and that they are fairly consistent with the number of revisions.

In total, 35 different types of conversions occurred between the systems, each with a separate frequency. The results are shown in Table 9.14 and Table 9.15. Table 9.14 displays the frequency each type of transition occurred. The first column is the conversion with what it was converted from before the – and what it is converted to after the –. The middle column is the number times that conversion occurred in total across all systems. The last column is the percentage that conversion occurred out of all conversions. The most frequent by far are the conversions in the *Statement* category (82.1%).

Individually, conversions from expression-statements to declaration-statements are the most common being 34.8% of all conversions, and declaration-statement to expression-statement being the next most common at 24.5% of all conversions. After that, the percentage each type of conversion occurs drops quickly.

In order to adjust for systems with a large number of conversions, the data is also normalized by the total number of conversions that occurred in that system. The result is

Table 9.12. Total number of conversions for each system across all revisions. Left-column is system, middle is the number of converts, and the right column is the number of converts divided by revision (as percent). Last row contains total overall systems.

System	Total Converts	Percent Converts Per Revision
<i>antimony</i>	93	5.4%
<i>appleseed</i>	617	11.5%
<i>BansheeEngine</i>	823	26.9%
<i>bitcoin</i>	445	7.7%
<i>blender</i>	1,780	7.4%
<i>cgal</i>	4,889	12.4%
<i>ChaiScript</i>	261	19.9%
<i>citra</i>	262	9.0%
<i>Clang</i>	4,314	7.6%
<i>cocos2d-x</i>	2,015	14.5%
<i>codeblocks</i>	1,104	14.8%
<i>codelite</i>	2,550	32.2%
<i>CRYENGINE</i>	212	19.4%
<i>CTK</i>	256	9.1%
<i>Dealii</i>	1,026	5.0%
<i>distortos</i>	85	3.4%
<i>Dlib</i>	198	4.5%
<i>engine</i>	206	9.9%
<i>folly</i>	185	5.6%
<i>GamePlay</i>	1,077	79.5%
<i>gnuradio</i>	194	6.1%
<i>griefly</i>	54	5.0%
<i>irods</i>	925	32.3%
<i>json</i>	112	10.7%
<i>kdevelop</i>	2,299	13.8%
<i>kokkos</i>	80	10.7%
<i>Mantella</i>	67	5.1%
<i>Mongo</i>	3,261	12.4%
<i>MultiMC5</i>	176	13.9%
<i>natron</i>	2,535	25.2%
<i>newton-dynamics</i>	416	13.3%
<i>nix</i>	579	21.4%
<i>ogre</i>	1,647	31.2%
<i>oiio</i>	245	10.0%
<i>ola</i>	386	8.6%
<i>opencv</i>	1,524	13.5%
<i>openFrameworks</i>	516	7.8%

Table 9.13. Total conversions (continued).

<i>Plasma</i>	305	21.9%
<i>ppsspp</i>	1,507	9.9%
<i>Qt</i>	3,082	11.0%
<i>QuantLib</i>	744	8.7%
<i>Rcpp</i>	1,092	56.1%
<i>rdkit</i>	238	9.6%
<i>stellarium</i>	962	11.1%
<i>supertux</i>	278	6.5%
<i>swift</i>	3,424	11.3%
<i>tera</i>	54	6.5%
<i>tfs-old-svn</i>	596	12.8%
<i>Urho3D</i>	1,020	19.5%
<i>xbmc</i>	2,250	8.9%
Total	52,966	11.9%

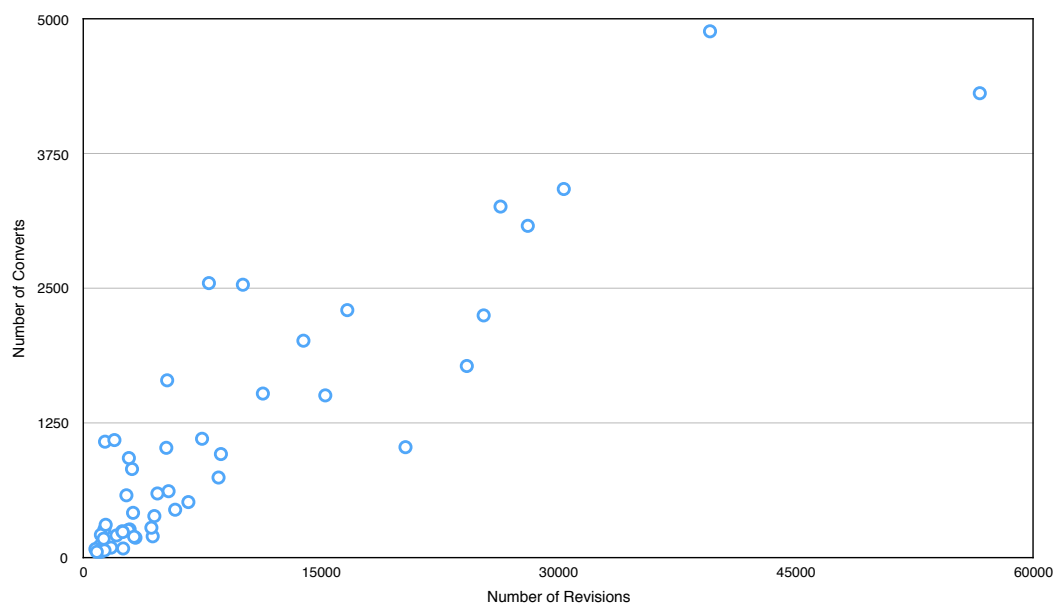


Figure 9.7. The total number of revisions plotted against the number of conversions. X-axis is number of revisions and y-axis is the number of conversions. The result is fairly linear.

Table 9.14. Frequency of each time of conversion. The left column is the number of conversions, the middle column is the raw number of how many times the conversion occurred between all systems, and the right column is the percentage that conversion occurred out of all types.

Conversion	Count	Percentage
<i>expr_stmt-decl_stmt</i>	18,432	34.8%
<i>decl_stmt-expr_stmt</i>	12,976	24.5%
<i>expr_stmt-return</i>	3,652	6.9%
<i>return-expr_stmt</i>	3,332	6.3%
<i>return-decl_stmt</i>	3,317	6.3%
<i>decl_stmt-return</i>	1,775	3.4%
<i>else-elseif</i>	1,588	3.0%
<i>private-public</i>	1,067	2.0%
<i>public-private</i>	911	1.7%
<i>struct-class</i>	893	1.7%
<i>private-protected</i>	693	1.3%
<i>elseif-else</i>	671	1.3%
<i>while-for</i>	590	1.1%
<i>class-struct</i>	589	1.1%
<i>for-while</i>	335	0.6%
<i>foreach-for</i>	306	0.6%
<i>protected-private</i>	266	0.5%
<i>protected-public</i>	244	0.5%
<i>for-foreach</i>	237	0.4%
<i>if-while</i>	212	0.4%
<i>if-for</i>	206	0.4%
<i>for-if</i>	198	0.4%
<i>public-protected</i>	175	0.3%
<i>while-if</i>	122	0.2%
<i>class-enum</i>	48	0.1%
<i>struct-enum</i>	46	0.1%
<i>union-struct</i>	24	0.0%
<i>while-foreach</i>	21	0.0%
<i>struct-union</i>	14	0.0%
<i>class-union</i>	5	0.0%
<i>enum-struct</i>	5	0.0%
<i>foreach-if</i>	5	0.0%
<i>if-foreach</i>	5	0.0%
<i>enum-class</i>	3	0.0%
<i>foreach-while</i>	3	0.0%
<i>Total</i>	52,966	100.0%

Table 9.15. Percentage of each type of conversion normalized for each system by the total number of conversions.

Conversion	Normalized Median
<i>expr_stmt-decl_stmt</i>	31.4%
<i>decl_stmt-expr_stmt</i>	23.1%
<i>expr_stmt-return</i>	5.3%
<i>return-expr_stmt</i>	4.7%
<i>return-decl_stmt</i>	4.6%
<i>decl_stmt-return</i>	2.5%
<i>else-elseif</i>	2.3%
<i>struct-class</i>	1.1%
<i>elseif-else</i>	1.1%
<i>private-public</i>	1.0%
<i>private-protected</i>	0.8%
<i>while-for</i>	0.8%
<i>public-private</i>	0.6%
<i>class-struct</i>	0.6%
<i>for-while</i>	0.4%
<i>protected-private</i>	0.4%
<i>if-while</i>	0.3%
<i>if-for</i>	0.3%
<i>for-if</i>	0.2%
<i>while-if</i>	0.2%
<i>protected-public</i>	0.1%
<i>public-protected</i>	0.1%
<i>class-enum</i>	0.0%
<i>struct-enum</i>	0.0%
<i>for-foreach</i>	0.0%
<i>foreach-for</i>	0.0%
<i>struct-union</i>	0.0%
<i>union-struct</i>	0.0%
<i>class-union</i>	0.0%
<i>enum-class</i>	0.0%
<i>enum-struct</i>	0.0%
<i>foreach-while</i>	0.0%
<i>while-foreach</i>	0.0%
<i>foreach-if</i>	0.0%
<i>if-foreach</i>	0.0%

in Table 9.15. The first column gives the conversion, and the second column is calculated as follows. First, the percentage each conversion occurred for a system is computed, then the median between all systems is taken. Normalization does not have a major effect on the data, meaning that the distribution per system is fairly consistent.

In conclusion, conversion occurs on average once every eight revisions, and 82% of the time it is between declaration-statements, expression-statements, and return-statements. The consequences (**RQ4**), is that the frequency for which conversions occur justify the necessity to have such detection in a syntactic differencer, but the biggest gain and most often required is in the *Statement* category. A lack of support for conversion, will lead to fairly frequent incorrect difference reporting that is also difficult to detect without specific markup.

9.5 Threats to Validity

The following threats to validity are identified with the study.

First, srcDiff, srcML, Git, and other tooling used as part of the study are complicated pieces of software, and as such contain faults. The parsing of C++ code is notoriously difficult, in part, due to preprocessor statements making C++ a non-context free grammar. The extent to which the tools report inappropriate ASTs is not known, yet srcML has a rigorous test suite and srcDiff is also well tested. Despite this, a negligible number of revisions failed to report a correct XML document (possible due to program crash, etc.). A negligible number of commits will not affect the vast amount of data collected.

Only standard C++ extensions were used to locate C++ files. Most of the projects used only these extensions, however, some used unusual extensions which possibly contained C++ code. For example, a C++ extension followed by `.in`, which would possibly denote a file that is used to generate code as part of the build system, and `.tpp`, which could possibly contain template code. These were only noticed in a few of the repositories. For future work, these files can be examined more closely and a determination made as to whether they should be included as part of the data collected.

The `.h` extension was treated as a C++, however, it is used both as a C and C++ header extension. Some of the projects included both C and C++ code and used `.h` for both. As C++ is largely a super-set of C (with a few exceptions), the parsing of C code as C++ code is a minor threat.

srcML provides support for the markup of Qt extensions. Currently, these are always on. Some code may have been incorrectly converted to srcML when a non-Qt project used an identifier that is part of the Qt extensions. In the future, it is planned for srcML to support Qt extensions optionally.

Some of the projects have committed code that was auto-generated by some tooling. Auto-generated files may exist throughout the history of a project or may have appeared and disappeared at any point in the history of the software. For the most part, we are uninterested in changes to auto-generated code, however, as they can appear at any point in history, they may be difficult to find. Investigation into auto-detection of all generated files through the history of projects can be a valid research topic. With the systems used as part of the study, two instances of auto-generated files were found. One was in `ppspp`,

and the other in `codelite`. They were found due to `srcDiff`'s current difficulty in handling files with, for example, large static arrays and string literals, as they result in elements with exceedingly large numbers of children and text nodes for which Myers' algorithm $O(ND)$ algorithm [Myers 1986] is not quick enough. In these two cases, the file list generation part of the data collection was instructed to ignore these auto-generated files.

Lastly, only C++ was used as part of the study. Results may be somewhat different for other languages. Although, we have experimented a little with conversion on Java (look at a 100 random revisions of a few systems) and have found a consistent amount of function conversions. Future work applying the approach to other languages may be worthwhile.

9.6 Conclusion

An investigation of fifty software systems to determine the prevalence and consequences of code replace and code conversion is performed. The results are that code replace is a frequently occurring event at the text, expression, and statement level, and not uncommon for functions (**RQ1**). Code conversion is also a sufficiently common event primarily between declaration-statements, expression-statements, and return statements (**RQ3**).

The consequences of both (**RQ2** and **RQ4**) are that support for both are needed in a syntactic differencer, and that differencers that solely use tree-edit distance algorithms will often arrive at an incorrect difference. That is, code replacement and code conversion are a major reason why `srcDiff` outperforms current syntactical differencing approaches.

CHAPTER 10

Conclusion

In this dissertation, the syntactic-differencing approach, srcDiff, is presented along with two evaluations. In addition, the syntactic-differencing approach is applied to two software engineering research problems: investigation into changes in method stereotype and investigation into code replace and code conversion.

In total, ten research questions (4 are evaluations and 6 for the two studies using srcDiff) are proposed and answered. The research questions are repeated here and answered, separately for the evaluations and software engineering research applications.

- **Evaluation RQ1:** Does srcDiff produce changes that are easier to read and more understandable than the other approaches?
- **Evaluation RQ2:** Is srcDiff preferred over the other approaches?
- **Evaluation RQ3:** Does srcDiff result in a delta that is more accurate than the other approaches?
- **Evaluation RQ4:** Is srcDiff scalable and practical to use in a software development environment?

The study shows that srcDiff is overwhelmingly chosen over GumTree [Falleri et al. 2014], GitHub’s difference view [GitHub 2016], and Mergely [Peabody 2016] as the easiest to read and understand (**Evaluation RQ1**) and the most preferable (**Evaluation RQ2**). With all participants, srcDiff is about equally as accurate as GumTree. With experts, srcDiff is overwhelmingly considered the most accurate (**Evaluation RQ3**). The

timing comparison in CHAPTER 7, shows that srcDiff has a quicker execution time than GumTree, and the applications of srcDiff to full history of many software projects (CHAPTER 8 and CHAPTER 9) shows that srcDiff is highly scalable and practical to use in a software development environment (**Evaluation RQ4**).

- **Stereotype RQ1:** How stable are a method's stereotype?
- **Stereotype RQ2:** What consequences are there to a method's stereotype changing?

The results of CHAPTER 8 conclude that method stereotypes are highly stable (**Stereotype RQ1**) and that certain types of method stereotypes can indicate poor design or inappropriate function changes (**Stereotype RQ2**).

- **Replace RQ1:** How commonly do developers replace and rewrite code?
- **Replace RQ2:** What are the consequences of replace and rewriting code?

The results of CHAPTER 9 on replace show that replace occurs extremely frequently on average at the text level, and frequently at the expression and statement level. In addition, function replacements are not uncommon (**Replace RQ1**). The consequences of replaced code and its prevalence gives a measure to how much and how often srcDiff can outperform tree-edit distance based syntactic differencers. Replace code must be handled by a syntactic difference to arrive at meaningful differences (**Replace RQ2**).

- **Convert RQ1:** How often do developers convert code from one type to another?
- **Convert RQ2:** What are the consequence of converting code?

The results of CHAPTER 9 for converts show that converts are fairly common, but not as common as code replace (**Convert RQ1**). The consequences are analogous to code replace. Code conversion occurs frequently enough that differencers must support code conversion detection and markup. Without support, differencers will very frequently report poor differences (**Convert RQ2**).

srcDiff is beyond a research prototype, however, it is constantly being improved through the addition and refinement of rules, as well as, other steps to the algorithm. For example, an edit script correction step has been added after the use of Myers' algorithm [Myers 1986] to avoid large structures and statements from being reported as inserted and deleted because they have a small amount of change and a much smaller entity was reported as common between them.

Future work will consist of application of srcDiff to various other software engineering tasks. Since srcDiff boasts greater accuracy, one avenue of research is repeating former research tasks that used other syntactic differencers and seeing if better results are achieved. Other future work consists of investigating what developer's consider a move and adding extended support to srcDiff for move detection, and an investigation into software merging.

APPENDIX A

Code for srcDiff Tool Integration into Git

The following is bash script for integrating the srcDiff tool into a Git repository.

```
#!/bin/sh
echo "---" $1
echo "+++" $1

if [[ $2 = "/dev/null" ]]; then

    touch srcdiff_temp$(bash -c "echo $5 | grep -o '\.[^.]*$'")
    [ $# -eq 7 ] && srcdiff --unified=3 srcdiff_temp$(bash -c "echo $5 | grep
-o '\.[^.]*$'") "$5"
    rm -f srcdiff_temp$(bash -c "echo $5 | grep -o '\.[^.]*$'")

elif [[ $5 = "/dev/null" ]]; then

    touch srcdiff_temp$(bash -c "echo $2 | grep -o '\.[^.]*$'")
    [ $# -eq 7 ] && srcdiff --unified=3 "$2" srcdiff_temp$(bash -c "echo $2 |
grep -o '\.[^.]*$'")
    rm -f srcdiff_temp$(bash -c "echo $2 | grep -o '\.[^.]*$'")

else

    [ $# -eq 7 ] && srcdiff --unified=3 "$2" "$5"

fi
```

APPENDIX B

Complete srcDiff Tool Options

The following is a complete list of the srcDiff Tool options as produced by the `--help` option.

```
Usage: srcdiff [options] <original_src_infile modified_src_infile>... [-o
<srcDiff_outfile>]
```

Translates C, C++, and Java source code into the XML source-code representation srcDiff.

Multiple files are stored in a srcDiff archive.

General Options:

<code>-h [--help]</code>	Output srcdiff help message
<code>-V [--version]</code>	Output srcdiff version
<code>-o [--output] arg (==)</code>	Specify output filename
<code>-z [--compress]</code>	Compress the output
<code>-v [--verbose]</code>	Verbose messaging
<code>-q [--quiet]</code>	Silence messaging

Input Options:

<code>--files-from arg</code>	Set the input to be a list of file pairs from the provided file. Pairs are of the format: original modified
<code>--git arg</code>	Input from a Subversion repository. Example: <code>--git http://example.org@HASH-HASH</code>

srcml Options:

<code>-n [--archive]</code>	Output srcDiff as an archive
<code>-t [--src-encoding] arg (=ISO-8859-1)</code>	Set the input source encoding
<code>-x [--xml-encoding] arg (=UTF-8)</code>	Set the output XML encoding
<code>-l [--language] arg (=C++)</code>	Set the input source programming language
<code>-f [--filename] arg</code>	Override unit filename
<code>--register-ext arg</code>	Register an extension to language pair to be used during parsing
<code>--url arg</code>	Set the root url attribute
<code>-s [--src-version] arg</code>	Set the root version attribute
<code>--xmlns arg</code>	Set the prefix associated with a namespace or register a new one. of the form <code>--xmlns:prefix=url</code> or <code>--xmlns=url</code> for default prefix.
<code>--position</code>	Output additional position information on the srcML elements
<code>--tabs arg (=8)</code>	Set the tabstop size
<code>--no-xml-decl</code>	Do not output the xml declaration
<code>--cpp-markup-else</code>	Markup up #else contents (default)

```

--cpp-text-else          Do not markup #else contents
--cpp-markup-if0        Markup up #if 0 contents
--cpp-text-if0          Do not markup #if 0 contents (default)

srcdiff Options:
-m [ --method ] arg (=collect,group-diff)    Set srcdiff parsing method
--burst                                     Output each input file to a single
                                           srcDiff document. -o gives output
                                           directory
--srcml                                     Also, output the original and modified
                                           srcML of each file when burst enabled
-u [ --unified ] [=arg(=3)]                Output as colorized unified diff with
                                           provided context. Number is lines of
                                           context, 'all' or -1 for entire file,
                                           'function' for encompassing function
                                           (default = 3)
-y [ --side-by-side ] [=arg(=7)]           Output as colorized side-by-side diff
--html                                     Output unified/side-by-side view in
                                           html
-W [ --ignore-all-space ]                 Ignore all whitespace when outputting
                                           unified/side-by-side view
-w [ --ignore-space ]                      Ignore whitespace when outputting
                                           unified/side-by-side view
-c [ --ignore-comments ]                   Ignore comments when outputting
                                           unified/side-by-side view
--highlight [=arg(=partial)] (=partial)    Syntax-highlighting for
                                           unified/side-by-side view. none,
                                           partial (default), or full
--theme arg (=default)                     Select theme for syntax-highlighting.
                                           default or monokai
--summary [=arg(=text)]                    Output a summary of the differences.
                                           Options 'text' and/or 'table' summary.
                                           Default 'text'

```

Report bugs to mdecker6@kent.edu

APPENDIX C

Source-code Changes Used in Preliminary Study

This contains the source code change example and samples used in the preliminary study. For each, the original and modified code, GNU `diff`, and `srcDiff` are provided each as a separate image. In the actual study, the original and modified were side-by-side, but as we are not able to do so for all samples, we place them one after the other. Even one after the other, some images may be altered to fit. The complete survey is available at sdml.cs.kent.edu/srcDiff/dissertation/preliminary_study as part of the replication package.

Example Source Original

Original
<code>foo * bar = 0;</code>

Example Source Modified

Modified
<code>foo * bar = NULL;</code>

Example GNU diff**diff**

```
-foo * bar = 0;  
+foo * bar = NULL;
```

Example srcDiff**srcDiff**

```
foo * bar = 0NULL;
```

Sample 1 Source Original

Original
<pre>style.addProperty("fo:font-weight", boldness * 10, KoGenStyle::TextType);</pre>

Sample 1 Source Modified

Modified
<pre>style.addProperty("fo:font-weight", qBound(10, boldness, 90) * 10, KoGenStyle::TextType);</pre>

Sample 1 GNU diff

diff
<pre>-style.addProperty("fo:font-weight", boldness * 10, KoGenStyle::TextType); +style.addProperty("fo:font-weight", qBound(10, boldness, 90) * 10, KoGenStyle::TextType);</pre>

Sample 1 srcDiff

srcDiff
<pre>style.addProperty("fo:font-weight", qBound(10, boldness, 90) * 10, KoGenStyle::TextType);</pre>

Sample 2 Source Original**Original**

```
d->fallBackColorSpace = _fallBackToCs->dstColorSpace();
```

Sample 2 Source Modified**Modified**

```
d->fallBackColorSpace = _fallBackToCs->srcColorSpace();
```

Sample 2 GNU diff**diff**

```
-d->fallBackColorSpace = _fallBackToCs->dstColorSpace();  
+d->fallBackColorSpace = _fallBackToCs->srcColorSpace();
```

Sample 2 srcDiff**srcDiff**

```
d->fallBackColorSpace = _fallBackToCs->dstColorSpacesrcColorSpace();
```

Sample 3 Source Original

Original
<pre> /* This file is part of the KDE project * Copyright (C) 2008 Fredy Yanardi <fyanardi@gmail.com> * Copyright (C) 2008 Thorsten Zachmann <zachmann@kde.org> * * This library is free software; you can redistribute it and/or * modify it under the terms of the GNU Library General Public * License as published by the Free Software Foundation; either * version 2 of the License, or (at your option) any later version. * * This library is distributed in the hope that it will be useful, * but WITHOUT ANY WARRANTY; without even the implied warranty of * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU * Library General Public License for more details. * * You should have received a copy of the GNU Library General Public License * along with this library; see the file COPYING.LIB. If not, write to * the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, * Boston, MA 02110-1301, USA. */ </pre>

Sample 3 Source Modified

Modified
<pre> /* This file is part of the KDE project * Copyright (C) 2008 Fredy Yanardi <fyanardi@gmail.com> * Copyright (C) 2008-2009 Thorsten Zachmann <zachmann@kde.org> * * This library is free software; you can redistribute it and/or * modify it under the terms of the GNU Library General Public * License as published by the Free Software Foundation; either * version 2 of the License, or (at your option) any later version. * * This library is distributed in the hope that it will be useful, * but WITHOUT ANY WARRANTY; without even the implied warranty of * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU * Library General Public License for more details. * * You should have received a copy of the GNU Library General Public License * along with this library; see the file COPYING.LIB. If not, write to * the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, * Boston, MA 02110-1301, USA. */ </pre>

Sample 3 GNU diff

diff	
	/* This file is part of the KDE project
	* Copyright (C) 2008 Fredy Yanardi <fyanardi@gmail.com>
-	* Copyright (C) 2008 Thorsten Zachmann <zachmann@kde.org>
+	* Copyright (C) 2008-2009 Thorsten Zachmann <zachmann@kde.org>
	*
	* This library is free software; you can redistribute it and/or
	* modify it under the terms of the GNU Library General Public
	* License as published by the Free Software Foundation; either
	* version 2 of the License, or (at your option) any later version.
	*
	* This library is distributed in the hope that it will be useful,
	* but WITHOUT ANY WARRANTY; without even the implied warranty of
	* MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
	* Library General Public License for more details.
	*
	* You should have received a copy of the GNU Library General Public License
	* along with this library; see the file COPYING.LIB. If not, write to
	* the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor,
	* Boston, MA 02110-1301, USA.
	*/

Sample 3 srcDiff

srcDiff	
	/* This file is part of the KDE project
	* Copyright (C) 2008 Fredy Yanardi <fyanardi@gmail.com>
	* Copyright (C) 2008 2008-2009 Thorsten Zachmann <zachmann@kde.org>
	*
	* This library is free software; you can redistribute it and/or
	* modify it under the terms of the GNU Library General Public
	* License as published by the Free Software Foundation; either
	* version 2 of the License, or (at your option) any later version.
	*
	* This library is distributed in the hope that it will be useful,
	* but WITHOUT ANY WARRANTY; without even the implied warranty of
	* MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
	* Library General Public License for more details.
	*
	* You should have received a copy of the GNU Library General Public License
	* along with this library; see the file COPYING.LIB. If not, write to
	* the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor,
	* Boston, MA 02110-1301, USA.
	*/

Sample 4 Source Original

Original
<pre>if (!numberedParagraph && e.tagName() != "list-item" && !listHeader) continue; if (!firstTime) { // use empty formats to not inherit from the prev parag QTextBlockFormat bf; QTextCharFormat cf; cursor.insertBlock(bf, cf); } else { firstTime = false; } QTextBlock current = cursor.block();</pre>

Sample 4 Source Modified

Modified
<pre>if (!numberedParagraph && e.tagName() != "list-item" && !listHeader) continue; if (!firstTime && !numberedParagraph) { // use empty formats to not inherit from the prev parag QTextBlockFormat bf; QTextCharFormat cf; cursor.insertBlock(bf, cf); } firstTime = false; QTextBlock current = cursor.block();</pre>

Sample 4 GNU diff

diff
<pre> if (!numberedParagraph && e.tagName() != "list-item" && !listHeader) continue; -if (!firstTime) { +if (!firstTime && !numberedParagraph) { // use empty formats to not inherit from the prev parag QTextBlockFormat bf; QTextCharFormat cf; cursor.insertBlock(bf, cf); -} else { - firstTime = false; -} +firstTime = false; QTextBlock current = cursor.block(); </pre>

Sample 4 srcDiff

srcDiff
<pre> if (!numberedParagraph && e.tagName() != "list-item" && !listHeader) continue; if (!firstTime && !numberedParagraph) { // use empty formats to not inherit from the prev parag QTextBlockFormat bf; QTextCharFormat cf; cursor.insertBlock(bf, cf); } else { firstTime = false; } firstTime = false; QTextBlock current = cursor.block(); </pre>

Sample 5 Source Original

Original
<pre>KoView* KoMainWindow::currentView() const { // XXX if (d->activeView) { return d->activeView; } else { return d->rootViews.first(); } }</pre>

Sample 5 Source Modified

Modified
<pre>KoView* KoMainWindow::currentView() const { // XXX if (d->activeView) { return d->activeView; } else if (!d->rootViews.isEmpty()) { return d->rootViews.first(); } return 0; }</pre>

Sample 5 GNU diff

diff	
	<pre> KoView* KoMainWindow::currentView() const { // XXX if (d->activeView) { return d->activeView; } - else { + else if (!d->rootViews.isEmpty()) { return d->rootViews.first(); } + return 0; } </pre>

Sample 5 srcDiff

srcDiff	
	<pre> KoView* KoMainWindow::currentView() const { // XXX if (d->activeView) { return d->activeView; } else if (!d->rootViews.isEmpty()) { return d->rootViews.first(); } return 0;↵ } </pre>

Sample 6 Source Original**Original**

```
if (m_textShapeData->endPosition() < 0) { // not layouted yet.  
    if (lay == 0)  
        kWarning(32500) << "Painting shape that doesn't have a  
            kotext doc-layout, which can't work";  
    else if (! lay->hasLayouter())  
        lay->setLayout(new Layout(lay));  
    return;  
}
```

Sample 6 Source Modified**Modified**

```
if (m_textShapeData->endPosition() < 0) { // not layouted yet.  
    if (lay == 0) {  
        kWarning(32500) << "Painting shape that doesn't have a  
            kotext doc-layout, which can't work";  
        return;  
    }  
    else if (! lay->hasLayouter()) {  
        lay->setLayout(new Layout(lay));  
    }  
    if (!m_pageProvider) {  
        return;  
    }  
}
```

Sample 6 GNU diff**diff**

```
if (m_textShapeData->endPosition() < 0) { // not layouted yet.
-   if (lay == 0)
+   if (lay == 0) {
+       kWarning(32500) << "Painting shape that doesn't have a
+           kotext doc-layout, which can't work";
-   else if (! lay->hasLayouter())
+       return;
+   }
+   else if (! lay->hasLayouter()) {
+       lay->setLayout(new Layout(lay));
-   return;
+   }
+   if (!m_pageProvider) {
+       return;
+   }
+ }
```

Sample 6 srcDiff

```
srcDiff

if (m_textShapeData->endPosition() < 0) { // not layouted yet.
    if (lay == 0)
    {
        kWarning(32500) << "Painting shape that doesn't have a
            kotext doc-layout, which can't work";
        return;
    }
    else if (! lay->hasLayouter())
    {
        lay->setLayout(new Layout(lay));
    }
    if (!m_pageProvider) {
        return;
    }
}
```

Sample 7 Source Original

Original
<pre>QWidget* KisToolSelectPath::optionWidget() { return m_optWidget; }</pre>

Sample 7 Source Modified

Modified
<pre>QMap<QString, QWidget *> KisToolSelectPath::createOptionWidgets() { QMap<QString, QWidget *> map = KoCreatePathTool::createOptionWidgets(); map.insert(i18n("Tool Options"), createOptionWidget()); return map; }</pre>

Sample 7 GNU diff

diff
<pre>-QWidget* KisToolSelectPath::optionWidget() +QMap<QString, QWidget *> KisToolSelectPath::createOptionWidgets() { - return m_optWidget; + QMap<QString, QWidget *> map = KoCreatePathTool::createOptionWidgets(); + map.insert(i18n("Tool Options"), createOptionWidget()); + return map; }</pre>

Sample 7 srcDiff

```
srcDiff

QWidget* KisToolSelectPath::optionWidget()
{
    return m_optWidget;
}

 QMap<QString, QWidget *> KisToolSelectPath::createOptionWidgets()
{
    QMap<QString, QWidget *> map = KoCreatePathTool::createOptionWidgets();
    map.insert(i18n("Tool Options"), createOptionWidget());
    return map;
}
```

Sample 8 Source Original**Original**

```
void KoMainWindow::slotCloseAllViews()
{
    // Attention: Very touchy code... you know what you're doing? Gooooood :)
    d->forQuit = true;
    if (queryClose()) {
        // In case the document is embedded we close all open "extra-shells"
        if (d->rootDoc && d->rootDoc->isEmbedded()) {
            hide();
            d->rootDoc->removeShell(this);
            QList<KoMainWindow*> shells = d->rootDoc->shells();
            while (!shells.isEmpty()) {
                KoMainWindow* window = shells.takeFirst();
                window->hide();
                delete window;
                d->rootDoc = 0;
            }
        }
        // not embedded -> destroy the document and all shells/views ;)
        else {
            setRootDocument(0);
        }
        close(); // close this window (and quit the app if necessary)
    }
    d->forQuit = false;
}
```

Sample 8 Source Modified**Modified**

```
void KoMainWindow::slotCloseAllViews()
{
    d->forQuit = true;
    if (queryClose()) {
        hide();
        d->rootDoc->removeShell(this);
        QList<KoMainWindow*> shells = d->rootDoc->shells();
        d->rootDoc = 0;
        while (!shells.isEmpty()) {
            KoMainWindow* window = shells.takeFirst();
            window->hide();
            delete window;
        }
        close(); // close this window (and quit the app if necessary)
    }
    d->forQuit = false;
}
```


Sample 8 GNU diff

diff

```

void KoMainWindow::slotCloseAllViews()
{
-   // Attention: Very touchy code... you know what you're doing? Gooooood :)
  d->forQuit = true;
  if (queryClose()) {
-   // In case the document is embedded we close all open "extra-shells"
-   if (d->rootDoc && d->rootDoc->isEmbedded()) {
-       hide();
-       d->rootDoc->removeShell(this);
-       QList<KoMainWindow*> shells = d->rootDoc->shells();
-       while (!shells.isEmpty()) {
-           KoMainWindow* window = shells.takeFirst();
-           window->hide();
-           delete window;
-           d->rootDoc = 0;
-       }
-   }
-   // not embedded -> destroy the document and all shells/views ;)
-   else {
-       setRootDocument(0);
+       hide();
+       d->rootDoc->removeShell(this);
+       QList<KoMainWindow*> shells = d->rootDoc->shells();
+       d->rootDoc = 0;
+       while (!shells.isEmpty()) {
+           KoMainWindow* window = shells.takeFirst();
+           window->hide();
+           delete window;
+       }
+       close(); // close this window (and quit the app if necessary)
  }
  d->forQuit = false;
}

```

Sample 8 srcDiff

```

srcDiff

void KoMainWindow::slotCloseAllViews()
{
    // Attention: Very touchy code... you know what you're doing? Cooooed :)
    d->forQuit = true;
    if (queryClose()) {
        // In case the document is embedded we close all open "extra-shells"
        if (d->rootDoc && d->rootDoc->isEmbedded()) {
            hide();
            d->rootDoc->removeShell(this);
            QList<KoMainWindow*> shells = d->rootDoc->shells();
            d->rootDoc = 0;
            while (!shells.isEmpty()) {
                KoMainWindow* window = shells.takeFirst();
                window->hide();
                delete window;
            }
            d->rootDoc = 0;
        }
        // not embedded -> destroy the document and all shells/views :)
        else {
            setRootDocument(0);
        }
        close(); // close this window (and quit the app if necessary)
    }
    d->forQuit = false;
}

```

Sample 9 Source Original

Original
<pre> if (shape->hasAdditionalAttribute("text:anchor-type")) { QString anchorType = shape->additionalAttribute("text:anchor-type"); // page anchored shapes are handled differently if (anchorType != "page") { KoTextAnchor *anchor = new KoTextAnchor(shape); anchor->loadOdfFromShape(element); d->textSharedData->shapeInserted(shape, element, d->context); KoTextDocumentLayout *layout = qobject_cast<KoTextDocumentLayout*>(cursor.block().document()->documentLayout()); if (layout) { KoInlineTextObjectManager *textObjectManager = layout->inlineTextObjectManager(); if (textObjectManager) { textObjectManager->insertInlineObject(cursor, anchor); } } } else { d->textSharedData->shapeInserted(shape, element, d->context); } } </pre>

Sample 9 Source Modified

Modified
<pre> QString anchorType; if (shape->hasAdditionalAttribute("text:anchor-type")) anchorType = shape->additionalAttribute("text:anchor-type"); else if (element.hasAttributeNS(KoXmlNS::text, "anchor-type")) anchorType = element.attributeNS(KoXmlNS::text, "anchor-type"); else anchorType = "as-char"; // default value // page anchored shapes are handled differently if (anchorType != "page") { KoTextAnchor *anchor = new KoTextAnchor(shape); anchor->loadOdfFromShape(element); d->textSharedData->shapeInserted(shape, element, d->context); KoTextDocumentLayout *layout = qobject_cast<KoTextDocumentLayout*>(cursor.block().document()->documentLayout()); if (layout) { KoInlineTextObjectManager *textObjectManager = layout->inlineTextObjectManager(); if (textObjectManager) { textObjectManager->insertInlineObject(cursor, anchor); } } } else { d->textSharedData->shapeInserted(shape, element, d->context); } </pre>

Sample 9 GNU diff

diff
<pre> - if (shape->hasAdditionalAttribute("text:anchor-type")) { - QString anchorType = shape->additionalAttribute("text:anchor-type"); - // page anchored shapes are handled differently - if (anchorType != "page") { - KoTextAnchor *anchor = new KoTextAnchor(shape); - anchor->loadOdfFromShape(element); - d->textSharedData->shapeInserted(shape, element, d->context); + QString anchorType; + if (shape->hasAdditionalAttribute("text:anchor-type")) + anchorType = shape->additionalAttribute("text:anchor-type"); + else if (element.hasAttributeNS(KoXmlNS::text, "anchor-type")) + anchorType = element.attributeNS(KoXmlNS::text, "anchor-type"); + else + anchorType = "as-char"; // default value - KoTextDocumentLayout *layout - = qobject_cast<KoTextDocumentLayout*>(cursor.block().document()->documentLayout()); - if (layout) { - KoInlineTextObjectManager *textObjectManager = layout->inlineTextObjectManager(); - if (textObjectManager) { - textObjectManager->insertInlineObject(cursor, anchor); - } + // page anchored shapes are handled differently + if (anchorType != "page") { + KoTextAnchor *anchor = new KoTextAnchor(shape); + anchor->loadOdfFromShape(element); + d->textSharedData->shapeInserted(shape, element, d->context); + + KoTextDocumentLayout *layout + = qobject_cast<KoTextDocumentLayout*>(cursor.block().document()->documentLayout()); + if (layout) { + KoInlineTextObjectManager *textObjectManager = layout->inlineTextObjectManager(); + if (textObjectManager) { + textObjectManager->insertInlineObject(cursor, anchor); + } - } - else { - d->textSharedData->shapeInserted(shape, element, d->context); - } + } else { + d->textSharedData->shapeInserted(shape, element, d->context); + } else { + d->textSharedData->shapeInserted(shape, element, d->context); + } </pre>

Sample 9 srcDiff

```

srcDiff

if (shape->hasAdditionalAttribute("text:anchor-type")) {
    QString anchorType = shape->additionalAttribute("text:anchor-type");
    if (shape->hasAdditionalAttribute("text:anchor-type"))
        anchorType = shape->additionalAttribute("text:anchor-type");
    else if (element.hasAttributeNS(KoXmlNS::text, "anchor-type"))
        anchorType = element.attributeNS(KoXmlNS::text, "anchor-type");
    else
        anchorType = "as-char"; // default value
    // page anchored shapes are handled differently
    if (anchorType != "page") {
        KoTextAnchor *anchor = new KoTextAnchor(shape);
        anchor->loadOdfFromShape(element);
        d->textSharedData->shapeInserted(shape, element, d->context);

        KoTextDocumentLayout *layout
            = qobject_cast<KoTextDocumentLayout*>(cursor.block().document()->documentLayout());
        if (layout) {
            KoInlineTextObjectManager *textObjectManager = layout->inlineTextObjectManager();
            if (textObjectManager) {
                textObjectManager->insertInlineObject(cursor, anchor);
            }
        }
    }
} else {
    d->textSharedData->shapeInserted(shape, element, d->context);
}
}

```

Sample 10 Source Original

Original
<pre> void KisQPainterCanvas::tabletEvent(QTabletEvent *e) { setFocus(Qt::OtherFocusReason); m_d->blockMouseEvent.start(100); qreal subpixelX = e->hiResGlobalX(); subpixelX = subpixelX - ((int) subpixelX); // leave only part behind the dot qreal subpixelY = e->hiResGlobalY(); subpixelY = subpixelY - ((int) subpixelY); // leave only part behind the dot QPointF pos(e->x() + subpixelX + m_d->documentOffset.x() - m_d->origin.x(), e->y() + subpixelY + m_d->documentOffset.y() - m_d->origin.y()); m_d->toolProxy->tabletEvent(e, m_d->viewConverter->viewToDocument(pos)); } </pre>

Sample 10 Source Modified

Modified
<pre> void KisQPainterCanvas::tabletEvent(QTabletEvent *e) { setFocus(Qt::OtherFocusReason); m_d->blockMouseEvent.start(100); m_d->toolProxy->tabletEvent(e, m_d->viewConverter->viewToDocument(e->hiResGlobalPos() - mapToGlobal(QPoint(0,0)) + m_d->documentOffset - m_d->origin)); } </pre>

Sample 10 GNU diff

diff	
	<pre> void KisQPainterCanvas::tabletEvent(QTabletEvent *e) { setFocus(Qt::OtherFocusReason); m_d->blockMouseEvent.start(100); - qreal subpixelX = e->hiResGlobalX(); - subpixelX = subpixelX - ((int) subpixelX); // leave only part behind the dot - qreal subpixelY = e->hiResGlobalY(); - subpixelY = subpixelY - ((int) subpixelY); // leave only part behind the dot - QPointF pos(e->x() + subpixelX + m_d->documentOffset.x() - m_d->origin.x(), - e->y() + subpixelY + m_d->documentOffset.y() - m_d->origin.y()); - m_d->toolProxy->tabletEvent(e, m_d->viewConverter->viewToDocument(pos)); + + m_d->toolProxy->tabletEvent(e, m_d->viewConverter->viewToDocument(+ e->hiResGlobalPos() - mapToGlobal(QPoint(0,0)) + m_d->documentOffset - m_d->origin)); } </pre>

Sample 10 srcDiff

srcDiff	
	<pre> void KisQPainterCanvas::tabletEvent(QTabletEvent *e) { setFocus(Qt::OtherFocusReason); m_d->blockMouseEvent.start(100); - qreal subpixelX = e->hiResGlobalX(); - subpixelX = subpixelX - ((int) subpixelX); // leave only part behind the dot - qreal subpixelY = e->hiResGlobalY(); - subpixelY = subpixelY - ((int) subpixelY); // leave only part behind the dot - QPointF pos(e->x() + subpixelX + m_d->documentOffset.x() - m_d->origin.x(), - e->y() + subpixelY + m_d->documentOffset.y() - m_d->origin.y()); - m_d->toolProxy->tabletEvent(e, m_d->viewConverter->viewToDocument(- pose->hiResGlobalPos() - mapToGlobal(QPoint(0,0)) + m_d->documentOffset - m_d->origin)); } </pre>

Sample 11 Source Original

Original

```

for (it1 = leads.begin(); it1 != leads.end(); ++it1)
{
    //get sheet-pointer - needed to handle inter-sheet dependencies correctly
    KSpreadSheet *sh = (*it1).sheet;
    if (!sh)
        sh = sheet;

    if (sh->dependencies()->deps->rangeDeps.contains (*it1))
    {
        QList<RangeDependency>::iterator it3;
        QList<RangeDependency> rdeps =
            sh->dependencies()->deps->rangeDeps[*it1];
        it3 = rdeps.begin();
        //erase all range dependencies of this cell in this cell-chunk
        while (it3 != rdeps.end())
            if (((*it3).cellrow == cell.row()) &&
                ((*it3).cellcolumn == cell.column()))
                it3 = rdeps.erase (it3);
            else
                ++it3;
        //erase the list if we no longer need it
        if (rdeps.empty())
            sh->dependencies()->deps->rangeDeps.erase (*it1);
    }
}

```


Sample 11 Source Modified

Modified

```

for (it1 = leads.begin(); it1 != leads.end(); ++it1)
{
    //get sheet-pointer - needed to handle inter-sheet dependencies correctly
    KSpreadSheet *sh = (*it1).sheet;
    if (!sh)
        sh = sheet;

    if (sh->dependencies()->deps->rangeDeps.contains (*it1))
    {
        QValueList<RangeDependency>::iterator it3;
        it3 = sh->dependencies()->deps->rangeDeps[*it1].begin();
        //erase all range dependencies of this cell in this cell-chunk
        while (it3 != sh->dependencies()->deps->rangeDeps[*it1].end())
            if (((*it3).cellrow == cell.row()) &&
                ((*it3).cellcolumn == cell.column()))
                it3 = sh->dependencies()->deps->rangeDeps[*it1].erase (it3);
            else
                ++it3;
        //erase the list if we no longer need it
        if (sh->dependencies()->deps->rangeDeps[*it1].empty())
            sh->dependencies()->deps->rangeDeps.erase (*it1);
    }
}

```

Sample 11 GNU diff

diff

```

for (it1 = leads.begin(); it1 != leads.end(); ++it1)
{
    //get sheet-pointer - needed to handle inter-sheet dependencies correctly
    KSpreadSheet *sh = (*it1).sheet;
    if (!sh)
        sh = sheet;

    if (sh->dependencies()->deps->rangeDeps.contains (*it1))
    {
        QList<RangeDependency>::iterator it3;
-       QList<RangeDependency> rdeps =
-           sh->dependencies()->deps->rangeDeps[*it1];
-       it3 = rdeps.begin();
+       it3 = sh->dependencies()->deps->rangeDeps[*it1].begin();
        //erase all range dependencies of this cell in this cell-chunk
-       while (it3 != rdeps.end())
+       while (it3 != sh->dependencies()->deps->rangeDeps[*it1].end())
            if (((*it3).cellrow == cell.row()) &&
                ((*it3).cellcolumn == cell.column()))
-               it3 = rdeps.erase (it3);
+               it3 = sh->dependencies()->deps->rangeDeps[*it1].erase (it3);
            else
                ++it3;
        //erase the list if we no longer need it
-       if (rdeps.empty())
+       if (sh->dependencies()->deps->rangeDeps[*it1].empty())
            sh->dependencies()->deps->rangeDeps.erase (*it1);
    }
}

```

Sample 11 srcDiff

srcDiff

```

for (it1 = leads.begin(); it1 != leads.end(); ++it1)
{
    //get sheet-pointer - needed to handle inter-sheet dependencies correctly
    KSpreadSheet *sh = (*it1).sheet;
    if (!sh)
        sh = sheet;

    if (sh->dependencies()->deps->rangeDeps.contains (*it1))
    {
        QList<RangeDependency>::iterator it3;
QValueList<RangeDependency> rdeps =
sh->dependencies()->deps->rangeDeps[*it1];
        it3 = sh->dependencies()->deps->rangeDeps[*it1].rdeps.begin();
        //erase all range dependencies of this cell in this cell-chunk
        while (it3 != sh->dependencies()->deps->rangeDeps[*it1].rdeps.end())
            if (((*it3).cellrow == cell.row()) &&
                ((*it3).cellcolumn == cell.column()))
                it3 = sh->dependencies()->deps->rangeDeps[*it1].rdeps.erase (it3);
            else
                ++it3;
        //erase the list if we no longer need it
        if (sh->dependencies()->deps->rangeDeps[*it1].rdeps.empty())
            sh->dependencies()->deps->rangeDeps.erase (*it1);
    }
}

```

Sample 12 Source Original**Original**

```
while (tile = iter.tile()) {
    tile->extent().getRect(&x, &y, &width, &height);
    sprintf(str, "%d,%d,%d,%d\n", x, y, width, height);
    store->write(str, strlen(str));

    tile->lockForRead();
    store->write((char *)tile->data(), tileDataSize);
    tile->unlock();

    ++iter;
}
```

Sample 12 Source Modified**Modified**

```
while (!(tile = iter.tile())) {
    tile->extent().getRect(&x, &y, &width, &height);
    sprintf(str, "%d,%d,%d,%d\n", x, y, width, height);
    store->write(str, strlen(str));

    tile->lockForRead();
    store->write((char *)tile->data(), tileDataSize);
    tile->unlock();

    ++iter;
}
```

Sample 12 GNU diff**diff**

```
-while (tile = iter.tile()) {  
+while (!(tile = iter.tile())) {  
    tile->extent().getRect(&x, &y, &width, &height);  
    sprintf(str, "%d,%d,%d,%d\n", x, y, width, height);  
    store->write(str, strlen(str));  
  
    tile->lockForRead();  
    store->write((char *)tile->data(), tileDataSize);  
    tile->unlock();  
  
    ++iter;  
}
```

Sample 12 srcDiff**srcDiff**

```
while (!(tile = iter.tile())) {  
    tile->extent().getRect(&x, &y, &width, &height);  
    sprintf(str, "%d,%d,%d,%d\n", x, y, width, height);  
    store->write(str, strlen(str));  
  
    tile->lockForRead();  
    store->write((char *)tile->data(), tileDataSize);  
    tile->unlock();  
  
    ++iter;  
}
```

APPENDIX D

Source-code Changes Used in the Evaluation

This contains the source-code change example and samples used in the evaluation. For each, the original and modified code, GitHub view, Mergely, GumTree, and srcDiff are provided each as a separate image. In the actual study, the original and modified of each representation was displayed side-by-side, but as we are not able to do so for all samples, we place them one after the other. Even one after the other, some images may be altered to fit. Also of note, GumTree also contained Javascript and a legend as part of the output. These are not reproduced here. The complete survey and the approach output is available at sdml.cs.kent.edu/srcDiff/dissertation/evaluation as part of the replication package.

Example Source Original

Original
<pre>foo bar = get_foo(); if(bar != null && !bar.empty()) bar.first().compute(); bar = null;</pre>

Example Source Modified

Modified
<pre>foo bar = foo(); boolean is_empty = (bar != null && !bar.empty()); if(is_empty) bar.first().compute();</pre>

Example GitHub Original

...	@@ -1,4 +1,4 @@
1	-foo bar = get_foo();
2	-if(bar != null && !bar.empty())
3	bar.first().compute();
4	-bar = null;

Example GitHub Modified

1	+foo bar = foo();
2	+boolean is_empty = (bar != null && !bar.empty());
3	+if(is_empty)
4	bar.first().compute();

Example Mergely Original

```

1  foo bar = get_foo();
2  if(bar != null && !bar.empty())
3      bar.first().compute();
4  bar = null;
5

```

Example Mergely Modified

```

1  foo bar = foo();
2  boolean is_empty = (bar != null && !bar.empty());
3  if(is_empty)
4      bar.first().compute();
5

```

Example srcDiff Original

Original	
1	foo bar = get_foo();
2	if(bar != null && !bar.empty())
3	bar.first().compute();
4	bar = null;
5	

Example srcDiff Modified

Modified	
1	foo bar = foo();
2	boolean is_empty = (bar != null && !bar.empty());
3	if(is_empty)
4	bar.first().compute();
5	

Example GumTree Original

```
foo bar = get_foo();
if(bar != null && !bar.empty())
    bar.first().compute();
bar = null;
```

Example GumTree Modified

```
foo bar = foo();
boolean is_empty = (bar != null && !bar.empty());
if(is_empty)
    bar.first().compute();
```

Sample 1 Source Original

Original
<pre>@Test public void testGeoHashValue() throws Exception { String mapping = XContentFactory.jsonBuilder() .startObject().startObject("type") .startObject("properties").startObject("point") .field("type", "geo_point").field("lat_lon", true) .field("geohash", true).endObject().endObject() .endObject().endObject().string(); DocumentMapper defaultMapper = MapperTestUtils.newParser().parse(mapping); ParsedDocument doc = defaultMapper.parse("type", "1", XContentFactory .jsonBuilder() .startObject() .field("point", GeoHashUtils.encode(1.2, 1.3)) .endObject() .bytes()); assertThat(doc.rootDoc().getField("point.lat"), notNullValue()); assertThat(doc.rootDoc().getField("point.lon"), notNullValue()); assertThat(doc.rootDoc().get("point.geohash"), equalTo(GeoHashUtils.encode(1.2, 1.3))); }</pre>

Sample 1 Source Modified

Modified
<pre> @Test public void testGeoHashValue() throws Exception { String mapping = XContentFactory.jsonBuilder() .startObject().startObject("type") .startObject("properties").startObject("point") .field("type", "geo_point").field("lat_lon", true) .field("geohash", true).endObject().endObject() .endObject().endObject().string(); DocumentMapper defaultMapper = createIndex("test").mapperService().documentMapperParser().parse(mapping); ParsedDocument doc = defaultMapper.parse("type", "1", XContentFactory .jsonBuilder() .startObject() .field("point", GeoHashUtils.encode(1.2, 1.3)) .endObject() .bytes()); assertThat(doc.rootDoc().getField("point.lat"), notNullValue()); assertThat(doc.rootDoc().getField("point.lon"), notNullValue()); assertThat(doc.rootDoc().get("point.geohash"), equalTo(GeoHashUtils.encode(1.2, 1.3))); } </pre>

Sample 1 GitHub Original

```

... @@ -1,14 +1,14 @@
1  @Test
2  public void testGeoHashValue() throws Exception {
3      String mapping = XContentFactory.jsonBuilder()
4          .startObject().startObject("type")
5          .startObject("properties").startObject("point")
6          .field("type", "geo_point").field("lat_lon", true)
7          .field("geohash", true).endObject().endObject()
8          .endObject().endObject().string();
9
10     DocumentMapper defaultMapper
11 -    = MapperTestUtils.newParser().parse(mapping);
12
13     ParsedDocument doc = defaultMapper.parse("type", "1", XContentFactory
14         .jsonBuilder()
15         .startObject()
16         .field("point", GeoHashUtils.encode(1.2, 1.3))
17         .endObject()
18         .bytes());
19
20     assertThat(doc.rootDoc().getField("point.lat"), notNullValue());
21     assertThat(doc.rootDoc().getField("point.lon"), notNullValue());
22     assertThat(doc.rootDoc().get("point.geohash"),
23         equalTo(GeoHashUtils.encode(1.2, 1.3)));
24 }

```

Sample 1 GitHub Modified

```

1  @Test
2  public void testGeoHashValue() throws Exception {
3      String mapping = XContentFactory.jsonBuilder()
4          .startObject().startObject("type")
5          .startObject("properties").startObject("point")
6          .field("type", "geo_point").field("lat_lon", true)
7          .field("geohash", true).endObject().endObject()
8          .endObject().endObject().string();
9
10     DocumentMapper defaultMapper
11 +     = createIndex("test").mapperService().documentMapperParser().parse(mapping);
12
13     ParsedDocument doc = defaultMapper.parse("type", "1", XContentFactory
14         .jsonBuilder()
15         .startObject()
16         .field("point", GeoHashUtils.encode(1.2, 1.3))
17         .endObject()
18         .bytes());
19
20     assertThat(doc.rootDoc().getField("point.lat"), notNullValue());
21     assertThat(doc.rootDoc().getField("point.lon"), notNullValue());
22     assertThat(doc.rootDoc().get("point.geohash"),
23         equalTo(GeoHashUtils.encode(1.2, 1.3)));
24 }

```

Sample 1 Mergely Original

```

1 @Test
2 public void testGeoHashValue() throws Exception {
3     String mapping = XContentFactory.jsonBuilder()
4         .startObject().startObject("type")
5         .startObject("properties").startObject("point")
6         .field("type", "geo_point").field("lat_lon", true)
7         .field("geohash", true).endObject().endObject()
8         .endObject().endObject().string();
9
10    DocumentMapper defaultMapper
11    = MapperTestUtils.newParser().parse(mapping);
12
13    ParsedDocument doc = defaultMapper.parse("type", "1", XContentFactory
14        .jsonBuilder()
15        .startObject()
16        .field("point", GeoHashUtils.encode(1.2, 1.3))
17        .endObject()
18        .bytes());
19
20    assertThat(doc.rootDoc().getField("point.lat"), notNullValue());
21    assertThat(doc.rootDoc().getField("point.lon"), notNullValue());
22    assertThat(doc.rootDoc().get("point.geohash"),
23        equalTo(GeoHashUtils.encode(1.2, 1.3)));
24 }
25

```

Sample 1 Mergely Modified

```

1  @Test
2  public void testGeoHashValue() throws Exception {
3      String mapping = XContentFactory.jsonBuilder()
4          .startObject().startObject("type")
5          .startObject("properties").startObject("point")
6          .field("type", "geo_point").field("lat_lon", true)
7          .field("geohash", true).endObject().endObject()
8          .endObject().endObject().string();
9
10     DocumentMapper defaultMapper
11     = createIndex("test").mapperService().documentMapperParser().parse(mapping);
12
13     ParsedDocument doc = defaultMapper.parse("type", "1", XContentFactory
14         .jsonBuilder()
15         .startObject()
16         .field("point", GeoHashUtils.encode(1.2, 1.3))
17         .endObject()
18         .bytes());
19
20     assertThat(doc.rootDoc().getField("point.lat"), notNullValue());
21     assertThat(doc.rootDoc().getField("point.lon"), notNullValue());
22     assertThat(doc.rootDoc().get("point.geohash"),
23         equalTo(GeoHashUtils.encode(1.2, 1.3)));
24 }
25

```

Sample 1 srcDiff Original

Original

```
1 @Test
2 public void testGeoHashValue() throws Exception {
3     String mapping = XContentFactory.jsonBuilder()
4         .startObject().startObject("type")
5         .startObject("properties").startObject("point")
6         .field("type", "geo_point").field("lat_lon", true)
7         .field("geohash", true).endObject().endObject()
8         .endObject().endObject().string();
9
10    DocumentMapper defaultMapper
11    = MapperTestUtils.newParser().parse(mapping);
12
13    ParsedDocument doc = defaultMapper.parse("type", "1", XContentFactory
14        .jsonBuilder()
15        .startObject()
16        .field("point", GeoHashUtils.encode(1.2, 1.3))
17        .endObject()
18        .bytes());
19
20    assertThat(doc.rootDoc().getField("point.lat"), notNullValue());
21    assertThat(doc.rootDoc().getField("point.lon"), notNullValue());
22    assertThat(doc.rootDoc().get("point.geohash"),
23        equalTo(GeoHashUtils.encode(1.2, 1.3)));
24 }
25
```


Sample 1 srcDiff Modified

Modified

```

1  @Test
2  public void testGeoHashValue() throws Exception {
3      String mapping = XContentFactory.jsonBuilder()
4          .startObject().startObject("type")
5          .startObject("properties").startObject("point")
6          .field("type", "geo_point").field("lat_lon", true)
7          .field("geohash", true).endObject().endObject()
8          .endObject().endObject().string();
9
10     DocumentMapper defaultMapper
11     = createIndex("test").mapperService().documentMapperParser().parse(mapping);
12
13     ParsedDocument doc = defaultMapper.parse("type", "1", XContentFactory
14         .jsonBuilder()
15         .startObject()
16         .field("point", GeoHashUtils.encode(1.2, 1.3))
17         .endObject()
18         .bytes());
19
20     assertThat(doc.rootDoc().getField("point.lat"), notNullValue());
21     assertThat(doc.rootDoc().getField("point.lon"), notNullValue());
22     assertThat(doc.rootDoc().get("point.geohash"),
23         equalTo(GeoHashUtils.encode(1.2, 1.3)));
24 }
25

```

Sample 1 GumTree Original

```
@Test
public void testGeoHashValue() throws Exception {
    String mapping = XContentFactory.jsonBuilder()
        .startObject().startObject("type")
        .startObject("properties").startObject("point")
        .field("type", "geo_point").field("lat_lon", true)
        .field("geohash", true).endObject().endObject()
        .endObject().endObject().string();

    DocumentMapper defaultMapper
    = MapperTestUtils.newParser().parse(mapping);

    ParsedDocument doc = defaultMapper.parse("type", "1", XContentFactory
        .jsonBuilder()
        .startObject()
        .field("point", GeoHashUtils.encode(1.2, 1.3))
        .endObject()
        .bytes());

    assertThat(doc.rootDoc().getField("point.lat"), notNullValue());
    assertThat(doc.rootDoc().getField("point.lon"), notNullValue());
    assertThat(doc.rootDoc().get("point.geohash"),
        equalTo(GeoHashUtils.encode(1.2, 1.3)));
}
```

Sample 1 GumTree Modified

```

@Test
public void testGeoHashValue() throws Exception {
    String mapping = XContentFactory.jsonBuilder()
        .startObject().startObject("type")
        .startObject("properties").startObject("point")
        .field("type", "geo_point").field("lat_lon", true)
        .field("geohash", true).endObject().endObject()
        .endObject().endObject().string();

    DocumentMapper defaultMapper
    = createIndex("test").mapperService().documentMapperParser().parse(mapping);

    ParsedDocument doc = defaultMapper.parse("type", "1", XContentFactory
        .jsonBuilder()
        .startObject()
        .field("point", GeoHashUtils.encode(1.2, 1.3))
        .endObject()
        .bytes());

    assertThat(doc.rootDoc().getField("point.lat"), notNullValue());
    assertThat(doc.rootDoc().getField("point.lon"), notNullValue());
    assertThat(doc.rootDoc().get("point.geohash"),
        equalTo(GeoHashUtils.encode(1.2, 1.3)));
}

```

Sample 2 Source Original

Original
<pre> public Field getCompletionField(ContextMapping.Context ctx, String input, BytesRef payload) { final String originalInput = input; if (input.length() > maxInputLength) { final int len = correctSubStringLength(input, Math.min(maxInputLength, input.length())); input = input.substring(0, len); } for (int i = 0; i < input.length(); i++) { if (isReservedChar(input.charAt(i))) { throw new IllegalArgumentException("Illegal input [" + originalInput + "] UTF-16 codepoint [0x" + Integer.toHexString((int) input.charAt(i)) .toUpperCase(Locale.ROOT) + "] at position " + i + " is a reserved character"); } } return new SuggestField(fieldType.names().indexName(), ctx, input, this.fieldType, payload, fieldType().analyzingSuggestLookupProvider); } </pre>

Sample 2 Source Modified

Modified
<pre> public Field getCompletionField(ContextMapping.Context ctx, String input, BytesRef payload) { final String originalInput = input; if (input.length() > maxInputLength) { final int len = correctSubStringLength(input, Math.min(maxInputLength, input.length())); input = input.substring(0, len); } for (int i = 0; i < input.length(); i++) { if (isReservedChar(input.charAt(i))) { throw new IllegalArgumentException("Illegal input [" + originalInput + "] UTF-16 codepoint [0x" + Integer.toHexString((int) input.charAt(i)) .toUpperCase(Locale.ROOT) + "] at position " + i + " is a reserved character"); } } return new SuggestField(fieldType().names().indexName(), ctx, input, fieldType(), payload, fieldType().analyzingSuggestLookupProvider); } </pre>

Sample 2 GitHub Original

```

... @@ -1,21 +1,21 @@
1  public Field getCompletionField(ContextMapping.Context ctx,
2      String input, BytesRef payload) {
3      final String originalInput = input;
4      if (input.length() > maxInputLength) {
5          final int len = correctSubStringLength(input,
6              Math.min(maxInputLength, input.length()));
7          input = input.substring(0, len);
8      }
9      for (int i = 0; i < input.length(); i++) {
10         if (isReservedChar(input.charAt(i))) {
11             throw new IllegalArgumentException("Illegal input ["
12                 + originalInput + "] UTF-16 codepoint [0x"
13                 + Integer.toHexString((int) input.charAt(i))
14                 .toUpperCase(Locale.ROOT)
15                 + "] at position " + i + " is a reserved character");
16         }
17     }
18 -     return new SuggestField(fieldType.names().indexName(),
19 -         ctx, input, this.fieldType, payload,
20             fieldType().analyzingSuggestLookupProvider);
21 }

```

Sample 2 GitHub Modified

```

1  public Field getCompletionField(ContextMapping.Context ctx,
2                                String input, BytesRef payload) {
3      final String originalInput = input;
4      if (input.length() > maxInputLength) {
5          final int len = correctSubStringLength(input,
6          Math.min(maxInputLength, input.length()));
7          input = input.substring(0, len);
8      }
9      for (int i = 0; i < input.length(); i++) {
10         if (isReservedChar(input.charAt(i))) {
11             throw new IllegalArgumentException("Illegal input ["
12             + originalInput + "] UTF-16 codepoint [0x"
13             + Integer.toHexString((int) input.charAt(i))
14             .toUpperCase(Locale.ROOT)
15             + "] at position " + i + " is a reserved character");
16         }
17     }
18 +     return new SuggestField(fieldType().names().indexName(),
19 +         ctx, input, fieldType(), payload,
20 +         fieldType().analyzingSuggestLookupProvider);
21 }

```

Sample 2 Mergely Original

```

1 public Field getCompletionField(ContextMapping.Context ctx,
2                               String input, BytesRef payload) {
3     final String originalInput = input;
4     if (input.length() > maxInputLength) {
5         final int len = correctSubStringLength(input,
6         Math.min(maxInputLength, input.length()));
7         input = input.substring(0, len);
8     }
9     for (int i = 0; i < input.length(); i++) {
10        if (isReservedChar(input.charAt(i))) {
11            throw new IllegalArgumentException("Illegal input ["
12            + originalInput + "] UTF-16 codepoint [0x"
13            + Integer.toHexString((int) input.charAt(i))
14            .toUpperCase(Locale.ROOT)
15            + "] at position " + i + " is a reserved character");
16        }
17    }
18    return new SuggestField(fieldType.names().indexName(),
19    ctx, input, this.fieldType, payload,
20    fieldType().analyzingSuggestLookupProvider);
21 }
22

```


Sample 2 Mergely Modified

```

1 public Field getCompletionField(ContextMapping.Context ctx,
2                               String input, BytesRef payload) {
3     final String originalInput = input;
4     if (input.length() > maxInputLength) {
5         final int len = correctSubStringLength(input,
6         Math.min(maxInputLength, input.length()));
7         input = input.substring(0, len);
8     }
9     for (int i = 0; i < input.length(); i++) {
10        if (isReservedChar(input.charAt(i))) {
11            throw new IllegalArgumentException("Illegal input ["
12            + originalInput + "] UTF-16 codepoint [0x"
13            + Integer.toHexString((int) input.charAt(i))
14            .toUpperCase(Locale.ROOT)
15            + "] at position " + i + " is a reserved character");
16        }
17    }
18    return new SuggestField(fieldType().names().indexName(),
19    ctx, input, fieldType(), payload,
20    fieldType().analyzingSuggestLookupProvider);
21 }
22

```

Sample 2 srcDiff Original

Original

```

1 public Field getCompletionField(ContextMapping.Context ctx,
2                               String input, BytesRef payload) {
3     final String originalInput = input;
4     if (input.length() > maxInputLength) {
5         final int len = correctSubStringLength(input,
6         Math.min(maxInputLength, input.length()));
7         input = input.substring(0, len);
8     }
9     for (int i = 0; i < input.length(); i++) {
10        if (isReservedChar(input.charAt(i))) {
11            throw new IllegalArgumentException("Illegal input ["
12            + originalInput + "] UTF-16 codepoint [0x"
13            + Integer.toHexString((int) input.charAt(i))
14            .toUpperCase(Locale.ROOT)
15            + "] at position " + i + " is a reserved character");
16        }
17    }
18    return new SuggestField(fieldType.names().indexName(),
19        ctx, input, this.fieldType, payload,
20        fieldType().analyzingSuggestLookupProvider);
21 }
22

```

Sample 2 srcDiff Modified

Modified

```

1 public Field getCompletionField(ContextMapping.Context ctx,
2                               String input, BytesRef payload) {
3     final String originalInput = input;
4     if (input.length() > maxInputLength) {
5         final int len = correctSubStringLength(input,
6         Math.min(maxInputLength, input.length()));
7         input = input.substring(0, len);
8     }
9     for (int i = 0; i < input.length(); i++) {
10        if (isReservedChar(input.charAt(i))) {
11            throw new IllegalArgumentException("Illegal input ["
12            + originalInput + "] UTF-16 codepoint [0x"
13            + Integer.toHexString((int) input.charAt(i))
14            .toUpperCase(Locale.ROOT)
15            + "] at position " + i + " is a reserved character");
16        }
17    }
18    return new SuggestField(fieldType().names().indexName(),
19        ctx, input, fieldType(), payload,
20        fieldType().analyzingSuggestLookupProvider);
21 }
22

```

Sample 2 GumTree Original

```
public Field getCompletionField(ContextMapping.Context ctx,
                               String input, BytesRef payload) {
    final String originalInput = input;
    if (input.length() > maxInputLength) {
        final int len = correctSubStringLength(input,
            Math.min(maxInputLength, input.length()));
        input = input.substring(0, len);
    }
    for (int i = 0; i < input.length(); i++) {
        if (isReservedChar(input.charAt(i))) {
            throw new IllegalArgumentException("Illegal input ["
                + originalInput + "] UTF-16 codepoint [0x"
                + Integer.toHexString((int) input.charAt(i))
                .toUpperCase(Locale.ROOT)
                + "] at position " + i + " is a reserved character");
        }
    }
    return new SuggestField(fieldType.names().indexName(),
        ctx, input, this.fieldType, payload,
        fieldType().analyzingSuggestLookupProvider);
}
```

Sample 2 GumTree Modified

```

public Field getCompletionField(ContextMapping.Context ctx,
                               String input, BytesRef payload) {
    final String originalInput = input;
    if (input.length() > maxInputLength) {
        final int len = correctSubStringLength(input,
            Math.min(maxInputLength, input.length()));
        input = input.substring(0, len);
    }
    for (int i = 0; i < input.length(); i++) {
        if (isReservedChar(input.charAt(i))) {
            throw new IllegalArgumentException("Illegal input ["
                + originalInput + "] UTF-16 codepoint [0x"
                + Integer.toHexString((int) input.charAt(i))
                .toUpperCase(Locale.ROOT)
                + "] at position " + i + " is a reserved character");
        }
    }
    return new SuggestField(fieldType().names().indexName(),
        ctx, input, fieldType(), payload,
        fieldType().analyzingSuggestLookupProvider);
}

```

Sample 3 Source Original

Original	
<pre> @Test public void passQueryAsJSONStringTest() throws Exception { client().admin().indices().prepareCreate("test").setSettings(ImmutableSettings.settingsBuilder().put("index.number_of_shards", 1)).execute().actionGet(); client().prepareIndex("test", "type1", "1") .setSource("field1", "value1_1", "field2", "value2_1") .setRefresh(true).execute().actionGet(); WrapperQueryBuilder wrapper = new WrapperQueryBuilder("{ \"term\" : { \"field1\" : \"value1_1\" } }"); CountResponse countResponse = client().prepareCount() .setQuery(wrapper).execute().actionGet(); assertHitCount(countResponse, 11); BoolQueryBuilder bool = new BoolQueryBuilder(); bool.must(wrapper); bool.must(new TermQueryBuilder("field2", "value2_1")); countResponse = client().prepareCount().setQuery(wrapper).execute().actionGet(); assertHitCount(countResponse, 11); } </pre>	<pre> { } : : </pre>

Sample 3 Source Modified

Modified
<pre>@Test public void passQueryAsJSONStringTest() throws Exception { assertAked(prepareCreate("test").setSettings(SETTING_NUMBER_OF_SHARDS, 1)); client().prepareIndex("test", "type1", "1") .setSource("field1", "value1_1", "field2", "value2_1") .setRefresh(true).get(); WrapperQueryBuilder wrapper = new WrapperQueryBuilder("{ \"term\" : { \"field1\" : \"value1_1\" } }"); assertHitCount(client().prepareCount() .setQuery(wrapper).get(), 11); BoolQueryBuilder bool = boolQuery().must(wrapper).must(new TermQueryBuilder("field2", "value2_1")); assertHitCount(client().prepareCount().setQuery(bool).get(), 11); }</pre>

Sample 3 GitHub Original

```

... @@ -1,23 +1,18 @@
1  @Test
2  public void passQueryAsJSONStringTest() throws Exception {
3  -  client().admin().indices().prepareCreate("test").setSettings(
4  -    ImmutableSettings.settingsBuilder().put("index.number_of_shards", 1)).execute().actionGet();
5
6  client().prepareIndex("test", "type1", "1")
7    .setSource("field1", "value1_1", "field2", "value2_1")
8  -  .setRefresh(true).execute().actionGet();
9
10  WrapperQueryBuilder wrapper
11    = new WrapperQueryBuilder("{ \"term\" : { \"field1\" : \"value1_1\" } }");
12  -  CountResponse countResponse = client().prepareCount()
13  -    .setQuery(wrapper).execute().actionGet();
14  -  assertHitCount(countResponse, 11);
15
16  -  BoolQueryBuilder bool = new BoolQueryBuilder();
17  -  bool.must(wrapper);
18  -  bool.must(new TermQueryBuilder(
19    "field2", "value2_1"));
20  -
21  -  countResponse = client().prepareCount().setQuery(wrapper).execute().actionGet();
22  -  assertHitCount(countResponse, 11);
23  }

```


Sample 3 GitHub Modified

```

1  @Test
2  public void passQueryAsJSONStringTest() throws Exception {
3  +  assertAked(prepareCreate("test").setSettings(SETTING_NUMBER_OF_SHARDS,
4  +  1));
5
6      client().prepareIndex("test", "type1", "1")
7          .setSource("field1", "value1_1", "field2", "value2_1")
8  +  .setRefresh(true).get();
9
10     WrapperQueryBuilder wrapper
11         = new WrapperQueryBuilder("{ \"term\" : { \"field1\" : \"value1_1\" } }");
12 +  assertHitCount(client().prepareCount()
13 +  .setQuery(wrapper).get(), 11);
14
15 +  BoolQueryBuilder bool = boolQuery().must(wrapper).must(new TermQueryBuilder(
16
17     "field2", "value2_1"));
17 +  assertHitCount(client().prepareCount().setQuery(bool).get(), 11);
18
18 }

```

Sample 3 Mergely Original

```

1 @Test
2 public void passQueryAsJSONStringTest() throws Exception {
3     client().admin().indices().prepareCreate("test").setSettings(
4         ImmutableSettings.settingsBuilder().put("index.number_of_shards", 1)).execute().actionGet();
5
6     client().prepareIndex("test", "type1", "1")
7         .setSource("field1", "value1_1", "field2", "value2_1")
8         .setRefresh(true).execute().actionGet();
9
10    WrapperQueryBuilder wrapper
11        = new WrapperQueryBuilder("{ \"term\" : { \"field1\" : \"value1_1\" } }");
12    CountResponse countResponse = client().prepareCount()
13    .setQuery(wrapper).execute().actionGet();
14    assertHitCount(countResponse, 1);
15
16    BoolQueryBuilder bool = new BoolQueryBuilder();
17    bool.must(wrapper);
18    bool.must(new TermQueryBuilder(
19        "field2", "value2_1"));
20
21    countResponse = client().prepareCount().setQuery(wrapper).execute().actionGet();
22    assertHitCount(countResponse, 1);
23 }
24

```

Sample 3 Mergely Modified

```

1 @Test
2 public void passQueryAsJSONStringTest() throws Exception {
3     assertAked(prepareCreate("test").setSettings(SETTING_NUMBER_OF_SHARDS,
4         1));
5
6     client().prepareIndex("test", "type1", "1")
7         .setSource("field1", "value1_1", "field2", "value2_1")
8         .setRefresh(true).get();
9
10    WrapperQueryBuilder wrapper
11        = new WrapperQueryBuilder("{ \"term\" : { \"field1\" : \"value1_1\" } }");
12    assertHitCount(client().prepareCount()
13        .setQuery(wrapper).get(), 1);
14
15    BoolQueryBuilder bool = boolQuery().must(wrapper).must(new TermQueryBuilder(
16        "field2", "value2_1"));
17    assertHitCount(client().prepareCount().setQuery(bool).get(), 1);
18 }
19

```

Sample 3 srcDiff Original

Original

```

1 @Test
2 public void passQueryAsJSONStringTest() throws Exception {
3     client().admin().indices().prepareCreate("test").setSettings(
4         ImmutableSettings.settingsBuilder().put("index.number_of_shards", 1)).execute().actionGet();
5
6     client().prepareIndex("test", "type1", "1")
7         .setSource("field1", "value1_1", "field2", "value2_1")
8         .setRefresh(true).execute().actionGet();
9
10    WrapperQueryBuilder wrapper
11        = new WrapperQueryBuilder("{ \"term\" : { \"field1\" : \"value1_1\" } }");
12    CountResponse countResponse = client().prepareCount()
13        .setQuery(wrapper).execute().actionGet();
14    assertHitCount(countResponse, 11);
15
16    BoolQueryBuilder bool = new BoolQueryBuilder();
17    bool.must(wrapper);
18    bool.must(new TermQueryBuilder(
19        "field2", "value2_1"));
20
21    countResponse = client().prepareCount().setQuery(wrapper).execute().actionGet();
22    assertHitCount(countResponse, 11);
23 }
24

```

Sample 3 srcDiff Modified

Modified

```

1 @Test
2 public void passQueryAsJSONStringTest() throws Exception {
3     assertAcked(prepareCreate("test").setSettings(SETTING_NUMBER_OF_SHARDS,
4         1));
5
6     client().prepareIndex("test", "type1", "1")
7         .setSource("field1", "value1_1", "field2", "value2_1")
8         .setRefresh(true).get();
9
10    WrapperQueryBuilder wrapper
11        = new WrapperQueryBuilder("{ \"term\" : { \"field1\" : \"value1_1\" } }");
12    assertHitCount(client().prepareCount()
13        .setQuery(wrapper).get(), 11);
14
15    BoolQueryBuilder bool = boolQuery().must(wrapper).must(new TermQueryBuilder(
16        "field2", "value2_1"));
17
18    assertHitCount(client().prepareCount().setQuery(bool).get(), 11);
19 }
20

```

Sample 3 GumTree Original

```

@Test
public void passQueryAsJSONStringTest() throws Exception {
    client().admin().indices().prepareCreate("test").setSettings(
        ImmutableSettings.settingsBuilder().put("index.number_of_shards", 1)).execute().actionGet();

    client().prepareIndex("test", "type1", "1")
        .setSource("field1", "value1_1", "field2", "value2_1")
        .setRefresh(true).execute().actionGet();

    WrapperQueryBuilder wrapper
        = new WrapperQueryBuilder("{ \"term\" : { \"field1\" : \"value1_1\" } }");
    CountResponse countResponse = client().prepareCount()
        .setQuery(wrapper).execute().actionGet();
    assertHitCount(countResponse, 11);

    BoolQueryBuilder bool = new BoolQueryBuilder();
    bool.must(wrapper);
    bool.must(new TermQueryBuilder(
        "field2", "value2_1"));

    countResponse = client().prepareCount().setQuery(wrapper).execute().actionGet();
    assertHitCount(countResponse, 11);
}

```

Sample 3 GumTree Modified

```

@Test
public void passQueryAsJSONStringTest() throws Exception {
    assertAcks(prepareCreate("test").setSettings(SETTING_NUMBER_OF_SHARDS,
        1));

    client().prepareIndex("test", "type1", "1")
        .setSource("field1", "value1_1", "field2", "value2_1")
        .setRefresh(true).get();

    WrapperQueryBuilder wrapper
        = new WrapperQueryBuilder("{ \"term\" : { \"field1\" : \"value1_1\" } }");
    assertHitCount(client().prepareCount()
        .setQuery(wrapper).get(), 11);

    BoolQueryBuilder bool = boolQuery().must(wrapper).must(new TermQueryBuilder(
        "field2", "value2_1"));
    assertHitCount(client().prepareCount().setQuery(bool).get(), 11);
}

```

Sample 4 Source Original

Original
<pre>public boolean isShutdown() { return shutdown; }</pre>

Sample 4 Source Modified

Modified
<pre>@Override public boolean isShutdown() { return false; }</pre>

Sample 4 GitHub Original

...	@@ -1 +1,4 @@
1	-public boolean isShutdown() { return shutdown; }

Sample 4 GitHub Modified

1	+@Override
2	+public boolean isShutdown() {
3	+ return false;
4	+}

Sample 4 Mergely Original

```
1 public boolean isShutdown() { return shutdown; }
```

Sample 4 Mergely Modified

```
1 @Override  
2 public boolean isShutdown() {  
3     return false;  
4 }  
5
```

Sample 4 srcDiff Original

Original	
1	public boolean isShutdown() {
1	return shutdown ;
1	}
2	

Sample 4 srcDiff Modified

```
Modified  
1 @Override  
2 public boolean isShutdown() {  
3     return false;  
4 }  
5
```

Sample 4 GumTree Original

```
public boolean isShutdown() { return shutdown; }
```

Sample 4 GumTree Modified

```
@Override  
public boolean isShutdown() {  
    return false;  
}
```

Sample 5 Source Original

Original
<pre> @Override public boolean containsAll(Collection<?> targets) { try { return range.containsAll((Iterable<? extends C>) targets); } catch (ClassCastException e) { return false; } } </pre>

Sample 5 Source Modified

Modified
<pre> @Override public boolean containsAll(Collection<?> targets) { return Collections2.containsAllImpl(this, targets); } </pre>

Sample 5 GitHub Original

...	@@ -1,7 +1,3 @@
1	@Override public boolean containsAll(Collection<?> targets) {
2	- try {
3	- return range.containsAll((Iterable<? extends C>) targets);
4	- } catch (ClassCastException e) {
5	- return false;
6	- }
7	}

Sample 5 GitHub Modified

```

1  @Override public boolean containsAll(Collection<?> targets) {
2  +  return Collections2.containsAllImpl(this, targets);
3  }

```

Sample 5 Mergely Original

```

1  @Override public boolean containsAll(Collection<?> targets) {
2  try {
3  return range.containsAll((Iterable<? extends C>) targets);
4  } catch (ClassCastException e) {
5  return false;
6  }
7  }
8

```

Sample 5 Mergely Modified

```

1  @Override public boolean containsAll(Collection<?> targets) {
2  return Collections2.containsAllImpl(this, targets);
3  }
4

```

Sample 5 srcDiff Original

Original

```

1 @Override public boolean containsAll(Collection<?> targets) {
2     try {
3         return range.containsAll((Iterable<? extends C>) targets);
4     } catch (ClassCastException e) {
5         return false;
6     }
7 }
8

```

Sample 5 srcDiff Modified

Modified

```

1 @Override public boolean containsAll(Collection<?> targets) {

2     return Collections2.containsAllImpl(this, targets);
3 }
4

```

Sample 5 GumTree Original

```

@Override public boolean containsAll(Collection<?> targets) {
    try {
        return range.containsAll((Iterable<? extends C>) targets);
    } catch (ClassCastException e) {
        return false;
    }
}

```

Sample 5 GumTree Modified

Up to date

```
@Override public boolean containsAll(Collection<?> targets) {  
    return Collections2.containsAllImpl(this, targets);  
}
```

Sample 6 Source Original

Original
<pre> public ReferenceEntry<K, V> getLiveEntry(Object key, int hash) { if (count != 0) { for (ReferenceEntry<K, V> e = getFirst(hash); e != null; e = e.getNext()) { if (e.getHash() != hash) { continue; } K entryKey = e.getKey(); if (entryKey == null) { continue; } if (keyEquivalence.equivalent(key, entryKey)) { if (expires() && isExpired(e)) { return null; } if (isInvalid(e)) { return null; } recordRead(e); return e; } } } return null; } </pre>

Sample 6 Source Modified

Modified
<pre>public ReferenceEntry<K, V> getLiveEntry(Object key, int hash) { if (count != 0) { for (ReferenceEntry<K, V> e = getFirst(hash); e != null; e = e.getNext()) { if (e.getHash() != hash) { continue; } K entryKey = e.getKey(); if (entryKey == null) { continue; } if (keyEquivalence.equivalent(key, entryKey)) { if (isLive(e)) { recordRead(e); return e; } return null; } } } return null; }</pre>

Sample 6 GitHub Original

...	@@ -1,27 +1,24 @@
1	public ReferenceEntry<K, V> getLiveEntry(Object key, int hash) {
2	if (count != 0) {
3	for (ReferenceEntry<K, V> e = getFirst(hash); e != null;
4	e = e.getNext()) {
5	if (e.getHash() != hash) {
6	continue;
7	}
8	
9	K entryKey = e.getKey();
10	if (entryKey == null) {
11	continue;
12	}
13	
14	if (keyEquivalence.equivalent(key, entryKey)) {
15	- if (expires() && isExpired(e)) {
16	- return null;
17	- }
18	- if (isInvalid(e)) {
19	- return null;
20	- }
21	- }
22	recordRead(e);
23	return e;
24	}
25	}
26	}
27	
28	return null;
29	}

Sample 6 GitHub Modified

1	public ReferenceEntry<K, V> getLiveEntry(Object key, int hash) {
2	if (count != 0) {
3	for (ReferenceEntry<K, V> e = getFirst(hash); e != null;
4	e = e.getNext()) {
5	if (e.getHash() != hash) {
6	continue;
7	}
8	
9	K entryKey = e.getKey();
10	if (entryKey == null) {
11	continue;
12	}
13	
14	if (keyEquivalence.equivalent(key, entryKey)) {
15	+ if (isLive(e)) {
16	recordRead(e);
17	return e;
18	}
19	+ return null;
20	+ }
21	+ }
22	}
23	}
24	
25	return null;
26	}

Sample 6 Mergely Original

```

1 public ReferenceEntry<K, V> getLiveEntry(Object key, int hash) {
2     if (count != 0) {
3         for (ReferenceEntry<K, V> e = getFirst(hash); e != null;
4             e = e.getNext()) {
5             if (e.getHash() != hash) {
6                 continue;
7             }
8
9             K entryKey = e.getKey();
10            if (entryKey == null) {
11                continue;
12            }
13
14            if (keyEquivalence.equivalent(key, entryKey)) {
15                if (expires() && isExpired(e)) {
16                    return null;
17                }
18                if (isInvalid(e)) {
19                    return null;
20                }
21
22                recordRead(e);
23                return e;
24            }
25        }
26    }
27
28    return null;
29 }
30

```


Sample 6 Mergely Modified

```
1 public ReferenceEntry<K, V> getLiveEntry(Object key, int hash) {
2     if (count != 0) {
3         for (ReferenceEntry<K, V> e = getFirst(hash); e != null;
4             e = e.getNext()) {
5             if (e.getHash() != hash) {
6                 continue;
7             }
8
9             K entryKey = e.getKey();
10            if (entryKey == null) {
11                continue;
12            }
13
14            if (keyEquivalence.equivalent(key, entryKey)) {
15                if (isLive(e)) {
16                    recordRead(e);
17                    return e;
18                }
19            }
20            return null;
21        }
22    }
23 }
24
25 return null;
26 }
27
```

Sample 6 srcDiff Original

Original

```

1 public ReferenceEntry<K, V> getLiveEntry(Object key, int hash) {
2     if (count != 0) {
3         for (ReferenceEntry<K, V> e = getFirst(hash); e != null;
4             e = e.getNext()) {
5             if (e.getHash() != hash) {
6                 continue;
7             }
8
9             K entryKey = e.getKey();
10            if (entryKey == null) {
11                continue;
12            }
13
14            if (keyEquivalence.equivalent(key, entryKey)) {
15                if (expires() && isExpired(e)) {
16                    return null;
17                }
18                if (isInvalid(e)) {
19                    return null;
20                }
21
22                recordRead(e);
23
24                return e;
25            }
26        }
27
28        return null;
29    }
30

```

Sample 6 srcDiff Modified

Modified

```

1 public ReferenceEntry<K, V> getLiveEntry(Object key, int hash) {
2     if (count != 0) {
3         for (ReferenceEntry<K, V> e = getFirst(hash); e != null;
4             e = e.getNext()) {
5             if (e.getHash() != hash) {
6                 continue;
7             }
8
9             K entryKey = e.getKey();
10            if (entryKey == null) {
11                continue;
12            }
13
14            if (keyEquivalence.equivalent(key, entryKey)) {
15
16                if (isLive(e)) {
17                    recordRead(e);
18                    return e;
19                }
20                return null;
21            }
22        }
23    }
24
25    return null;
26 }
27

```

Sample 6 GumTree Original

```

public ReferenceEntry<K, V> getLiveEntry(Object key, int hash) {
    if (count != 0) {
        for (ReferenceEntry<K, V> e = getFirst(hash); e != null;
            e = e.getNext()) {
            if (e.getHash() != hash) {
                continue;
            }

            K entryKey = e.getKey();
            if (entryKey == null) {
                continue;
            }

            if (keyEquivalence.equivalent(key, entryKey)) {
                if (expires() && isExpired(e)) {
                    return null;
                }
                if (isInvalid(e)) {
                    return null;
                }

                recordRead(e);
                return e;
            }
        }
    }

    return null;
}

```

Sample 6 GumTree Modified

```
public ReferenceEntry<K, V> getLiveEntry(Object key, int hash) {
    if (count != 0) {
        for (ReferenceEntry<K, V> e = getFirst(hash); e != null;
            e = e.getNext()) {
            if (e.getHash() != hash) {
                continue;
            }

            K entryKey = e.getKey();
            if (entryKey == null) {
                continue;
            }

            if (keyEquivalence.equivalent(key, entryKey)) {
                if (isLive(e)) {
                    recordRead(e);
                    return e;
                }

                return null;
            }
        }
    }

    return null;
}
```

Sample 7 Source Original

Original
<pre>private void convertFile(File file) { try { InplaceFileConverter fc = new InplaceFileConverter(lineConverter); byte[] ba = fc.readFile(file); fc.convert(file, ba); } catch (IOException exc) { addException(new ConversionException(exc.toString())); } }</pre>

Sample 7 Source Modified

Modified
<pre>private void convertFile(File file) { try { InplaceFileConverter fc = new InplaceFileConverter(ruleSet); fc.convert(file); } catch (IOException exc) { addException(new ConversionException(exc.toString())); } }</pre>

Sample 7 GitHub Original

```

... @@ -1,8 +1,7 @@
1   private void convertFile(File file) {
2       try {
3   -   InplaceFileConverter fc = new InplaceFileConverter(lineConverter);
4   -   byte[] ba = fc.readFile(file);
5   -   fc.convert(file, ba);
6       } catch (IOException exc) {
7           addException(new ConversionException(exc.toString()));
8       }
9   }

```

Sample 7 GitHub Modified

```

1   private void convertFile(File file) {
2       try {
3   +   InplaceFileConverter fc = new InplaceFileConverter(ruleSet);
4   +   fc.convert(file);
5
6       } catch (IOException exc) {
7           addException(new ConversionException(exc.toString()));
8       }
9   }

```

Sample 7 Mergely Original

```
1 private void convertFile(File file) {  
2     try {  
3         InplaceFileConverter fc = new InplaceFileConverter(lineConverter);  
4         byte[] ba = fc.readFile(file);  
5         fc.convert(file, ba);  
6     } catch (IOException exc) {  
7         addException(new ConversionException(exc.toString()));  
8     }  
9 }  
10
```

Sample 7 Mergely Modified

```
1 private void convertFile(File file) {  
2     try {  
3         InplaceFileConverter fc = new InplaceFileConverter(ruleSet);  
4         fc.convert(file);  
5     } catch (IOException exc) {  
6         addException(new ConversionException(exc.toString()));  
7     }  
8 }  
9
```


Sample 7 srcDiff Original**Original**

```
1 private void convertFile(File file) {
2     try {
3         InplaceFileConverter fc = new InplaceFileConverter(lineConverter);
4         byte[] ba = fc.readFile(file);
5         fc.convert(file, ba);
6     } catch (IOException exc) {
7         addException(new ConversionException(exc.toString()));
8     }
9 }
10
```

Sample 7 srcDiff Modified**Modified**

```
1 private void convertFile(File file) {
2     try {
3         InplaceFileConverter fc = new InplaceFileConverter(ruleSet);
4
5         fc.convert(file);
6     } catch (IOException exc) {
7         addException(new ConversionException(exc.toString()));
8     }
9 }
```

Sample 7 GumTree Original

```
private void convertFile(File file) {
    try {
        InplaceFileConverter fc = new InplaceFileConverter(lineConverter);
        byte[] ba = fc.readFile(file);
        fc.convert(file, ba);
    } catch (IOException exc) {
        addException(new ConversionException(exc.toString()));
    }
}
```

Sample 7 GumTree Modified

```
private void convertFile(File file) {
    try {
        InplaceFileConverter fc = new InplaceFileConverter(ruleSet);
        fc.convert(file);
    } catch (IOException exc) {
        addException(new ConversionException(exc.toString()));
    }
}
```

Sample 8 Source Original

Original	
<pre> public static void assertDuration(double currentDuration, long referenceDuraion, double referenceBIPS) throws AssertionError { double ajustedDuration = adjustExpectedDuration(referenceDuraion, referenceBIPS); if (currentDuration > ajustedDuration * SLACK_FACTOR) { throw new AssertionError(currentDuration + " exceeded expected " + ajustedDuration + " (adjusted), " + referenceDuraion + " (raw)"); } } </pre>	1

Sample 8 Source Modified

Modified	
<pre> public static void assertDuration(double currentDuration, long referenceDuration, double referenceBIPS) throws AssertionError { double ajustedDuration = adjustExpectedDuration(referenceDuration, referenceBIPS); if (currentDuration > ajustedDuration * SLACK_FACTOR) { throw new AssertionError("current duration "+ currentDuration + " exceeded expected " + ajustedDuration + " (adjusted reference), " + referenceDuration + " (raw reference)"); } } </pre>	

Sample 8 GitHub Original

```

... @@ -1,12 +1,12 @@
1   public static void assertDuration(double currentDuration,
2   -   long referenceDuraion, double referenceBIPS)
3       throws AssertionError {
4   -   double ajustedDuration = adjustExpectedDuration(referenceDuraion,
5       referenceBIPS);
6       if (currentDuration > ajustedDuration * SLACK_FACTOR) {
7   -   throw new AssertionError(currentDuration
8       + " exceeded expected "
9   -   + ajustedDuration + " (adjusted), "
10  -   + referenceDuraion + " (raw)");
11  }
12  }

```

Sample 8 GitHub Modified

```

1   public static void assertDuration(double currentDuration,
2   +   long referenceDuration, double referenceBIPS)
3       throws AssertionError {
4   +   double ajustedDuration = adjustExpectedDuration(referenceDuration,
5       referenceBIPS);
6       if (currentDuration > ajustedDuration * SLACK_FACTOR) {
7   +   throw new AssertionError("current duration " + currentDuration
8       + " exceeded expected "
9   +   + ajustedDuration + " (adjusted reference), "
10  +   + referenceDuration + " (raw reference)");
11  }
12  }

```

Sample 8 Mergely Original

```

1 public static void assertDuration(double currentDuration,
2   long referenceDuraion, double referenceBIPS)
3   throws AssertionError {
4   double ajustedDuration = adjustExpectedDuration(referenceDuraion,
5     referenceBIPS);
6   if (currentDuration > ajustedDuration * SLACK_FACTOR) {
7     throw new AssertionFailedError(currentDuration
8       + " exceeded expected "
9       + ajustedDuration + " {adjusted}, "
10      + referenceDuraion + " {raw})");
11   }
12 }
13

```

Sample 8 Mergely Modified

```

1 public static void assertDuration(double currentDuration,
2   long referenceDuration, double referenceBIPS)
3   throws AssertionError {
4   double ajustedDuration = adjustExpectedDuration(referenceDuration,
5     referenceBIPS);
6   if (currentDuration > ajustedDuration * SLACK_FACTOR) {
7     throw new AssertionFailedError("current duration "+ currentDuration
8       + " exceeded expected "
9       + ajustedDuration + " (adjusted reference), "
10      + referenceDuration + " (raw reference)");
11   }
12 }
13

```

Sample 9 srcDiff Original

Original

```

1 public static void assertDuration(double currentDuration,
2     long referenceDuraion, double referenceBIPS)
3     throws AssertionError {
4     double ajustedDuration = adjustExpectedDuration(referenceDuraion,
5         referenceBIPS);
6     if (currentDuration > ajustedDuration * SLACK_FACTOR) {
7         throw new AssertionError( currentDuration
8             + " exceeded expected "
9             + ajustedDuration + " (adjusted), "
10            + referenceDuraion + " (raw)");
11     }
12 }
13

```

Sample 8 srcDiff Modified

Modified

```

1 public static void assertDuration(double currentDuration,
2     long referenceDuration, double referenceBIPS)
3     throws AssertionError {
4     double ajustedDuration = adjustExpectedDuration(referenceDuration,
5         referenceBIPS);
6     if (currentDuration > ajustedDuration * SLACK_FACTOR) {
7         throw new AssertionError("current duration "+ currentDuration
8             + " exceeded expected "
9             + ajustedDuration + " (adjusted reference), "
10            + referenceDuration + " (raw reference)");
11     }
12 }
13

```

Sample 8 GumTree Original

```
public static void assertDuration(double currentDuration,
    long referenceDuraion, double referenceBIPS)
    throws AssertionError {
    double ajustedDuration = adjustExpectedDuration(referenceDuraion,
        referenceBIPS);
    if (currentDuration > ajustedDuration * SLACK_FACTOR) {
        throw new AssertionError(currentDuration
            + " exceeded expected "
            + ajustedDuration + " (adjusted), "
            + referenceDuraion + " (raw)");
    }
}
```

Sample 8 GumTree Modified

```
public static void assertDuration(double currentDuration,
    long referenceDuration, double referenceBIPS)
    throws AssertionError {
    double ajustedDuration = adjustExpectedDuration(referenceDuration,
        referenceBIPS);
    if (currentDuration > ajustedDuration * SLACK_FACTOR) {
        throw new AssertionError("current duration " + currentDuration
            + " exceeded expected "
            + ajustedDuration + " (adjusted reference), "
            + referenceDuration + " (raw reference)");
    }
}
```

Sample 9 Source Original

Original
<pre> public synchronized boolean remove(Marker markerToRemove) { if (children == null) { return false; } int size = children.size(); for (int i = 0; i < size; i++) { Marker m = (Marker) children.get(i); if (markerToRemove.equals(m)) { children.remove(i); return true; } } return false; } </pre>

Sample 9 Source Modified

Modified
<pre> public synchronized boolean remove(Marker referenceToRemove) { if (refereceList == null) { return false; } int size = refereceList.size(); for (int i = 0; i < size; i++) { Marker m = (Marker) refereceList.get(i); if (referenceToRemove.equals(m)) { refereceList.remove(i); return true; } } return false; } </pre>

Sample 9 GitHub Original

```
... @@ -1,16 +1,15 @@
1  -public synchronized boolean remove(Marker markerToRemove) {
2  -  if (children == null) {
3      return false;
4  }
5
6  -  int size = children.size();
7      for (int i = 0; i < size; i++) {
8  -  Marker m = (Marker) children.get(i);
9  -  if (markerToRemove.equals(m)) {
10 -  children.remove(i);
11      return true;
12  }
13  }
14 -
15      return false;
16  }
```

Sample 9 GitHub Modified

```
1  +public synchronized boolean remove(Marker referenceToRemove) {
2  +  if (refereceList == null) {
3      return false;
4  }
5
6  +  int size = refereceList.size();
7      for (int i = 0; i < size; i++) {
8  +      Marker m = (Marker) refereceList.get(i);
9  +      if (referenceToRemove.equals(m)) {
10 +         refereceList.remove(i);
11         return true;
12     }
13 }
14
14     return false;
15 }
```

Sample 9 Mergely Original

```
1 public synchronized boolean remove(Marker markerToRemove) {  
2     if (children == null) {  
3         return false;  
4     }  
5  
6     int size = children.size();  
7     for (int i = 0; i < size; i++) {  
8         Marker m = (Marker) children.get(i);  
9         if (markerToRemove.equals(m)) {  
10             children.remove(i);  
11             return true;  
12         }  
13     }  
14 —  
15     return false;  
16 }  
17
```

Sample 9 Mergely Modified

```
1 public synchronized boolean remove(Marker referenceToRemove) {  
2     if (refereceList == null) {  
3         return false;  
4     }  
5  
6     int size = refereceList.size();  
7     for (int i = 0; i < size; i++) {  
8         Marker m = (Marker) refereceList.get(i);  
9         if (referenceToRemove.equals(m)) {  
10             refereceList.remove(i);  
11             return true;  
12         }  
13     }  
14     return false;  
15 }  
16
```

Sample 9 srcDiff Original

Original

```
1 public synchronized boolean remove(Marker markerToRemove) {
2     if (children == null) {
3         return false;
4     }
5
6     int size = children.size();
7     for (int i = 0; i < size; i++) {
8         Marker m = (Marker) children.get(i);
9         if (markerToRemove.equals(m)) {
10             children.remove(i);
11             return true;
12         }
13     }
14
15     return false;
16 }
17
```

Sample 9 srcDiff Modified

Modified

```
1 public synchronized boolean remove(Marker referenceToRemove) {
2     if (refereceList == null) {
3         return false;
4     }
5
6     int size = refereceList.size();
7     for (int i = 0; i < size; i++) {
8         Marker m = (Marker) refereceList.get(i);
9         if (referenceToRemove.equals(m)) {
10             refereceList.remove(i);
11             return true;
12         }
13     }
14
15     return false;
16 }
```

Sample 9 GumTree Original

```
public synchronized boolean remove(Marker markerToRemove) {
    if (children == null) {
        return false;
    }

    int size = children.size();
    for (int i = 0; i < size; i++) {
        Marker m = (Marker) children.get(i);
        if (markerToRemove.equals(m)) {
            children.remove(i);
            return true;
        }
    }

    return false;
}
```

Sample 9 GumTree Modified

```
public synchronized boolean remove(Marker referenceToRemove) {
    if (referenceList == null) {
        return false;
    }

    int size = referenceList.size();
    for (int i = 0; i < size; i++) {
        Marker m = (Marker) referenceList.get(i);
        if (referenceToRemove.equals(m)) {
            referenceList.remove(i);
            return true;
        }
    }

    return false;
}
```

Sample 10 Source Original

Original
<pre> static class ExpirationSpec { @Nullable private final Long expireAfterAccessMillis; @Nullable private final Long expireAfterWriteMillis; private ExpirationSpec(Long expireAfterAccessMillis, Long expireAfterWriteMillis) { Preconditions.checkArgument(expireAfterAccessMillis == null expireAfterWriteMillis == null); this.expireAfterAccessMillis = expireAfterAccessMillis; this.expireAfterWriteMillis = expireAfterWriteMillis; } public static ExpirationSpec afterAccess(long afterAccess, TimeUnit unit) { return new ExpirationSpec(unit.toMillis(afterAccess), null); } public static ExpirationSpec afterWrite(long afterWrite, TimeUnit unit) { return new ExpirationSpec(null, unit.toMillis(afterWrite)); } @Override public int hashCode() { return Objects.hashCode(expireAfterAccessMillis, expireAfterWriteMillis); } @Override public boolean equals(Object o) { if (o instanceof ExpirationSpec) { ExpirationSpec that = (ExpirationSpec) o; return Objects.equal(this.expireAfterAccessMillis, that.expireAfterAccessMillis) && Objects.equal(this.expireAfterWriteMillis, that.expireAfterWriteMillis); } return false; } @Override public String toString() { return Objects.toStringHelper(this) .add("expireAfterAccessMillis", expireAfterAccessMillis) .add("expireAfterWriteMillis", expireAfterWriteMillis) .toString(); } } </pre>

Sample 10 Source Modified

Modified
<pre> static class DurationSpec { private final long duration; private final TimeUnit unit; private DurationSpec(long duration, TimeUnit unit) { this.duration = duration; this.unit = unit; } public static DurationSpec of(long duration, TimeUnit unit) { return new DurationSpec(duration, unit); } @Override public int hashCode() { return Objects.hashCode(duration, unit); } @Override public boolean equals(Object o) { if (o instanceof DurationSpec) { DurationSpec that = (DurationSpec) o; return unit.toNanos(duration) == that.unit.toNanos(that.duration); } return false; } @Override public String toString() { return Objects.toStringHelper(this) .add("duration", duration) .add("unit", unit) .toString(); } } </pre>

Sample 10 GitHub Original

```

... @@ -1,35 +1,26 @@
1  -static class ExpirationSpec {
2  - @Nullable
3  - private final Long expireAfterAccessMillis;
4  - @Nullable
5  - private final Long expireAfterWriteMillis;
6
7  - private ExpirationSpec(Long expireAfterAccessMillis, Long expireAfterWriteMillis) {
8  -     Preconditions.checkArgument(
9  -         expireAfterAccessMillis == null || expireAfterWriteMillis == null);
10 -     this.expireAfterAccessMillis = expireAfterAccessMillis;
11 -     this.expireAfterWriteMillis = expireAfterWriteMillis;
12 }
13
14 - public static ExpirationSpec afterAccess(long afterAccess, TimeUnit unit) {
15 -     return new ExpirationSpec(unit.toMillis(afterAccess), null);
16 - }
17 -
18 - public static ExpirationSpec afterWrite(long afterWrite, TimeUnit unit) {
19 -     return new ExpirationSpec(null, unit.toMillis(afterWrite));
20 }
21
22 @Override
23 public int hashCode() {
24 -     return Objects.hashCode(expireAfterAccessMillis, expireAfterWriteMillis);
25 }
26
27 @Override
28 public boolean equals(Object o) {
29 -     if (o instanceof ExpirationSpec) {
30 -         ExpirationSpec that = (ExpirationSpec) o;
31 -         return Objects.equal(this.expireAfterAccessMillis, that.expireAfterAccessMillis)
32 -             && Objects.equal(this.expireAfterWriteMillis, that.expireAfterWriteMillis);
33     }
34     return false;
35 }
36
37 @Override
38 public String toString() {
39     return Objects.toStringHelper(this)
40 -         .add("expireAfterAccessMillis", expireAfterAccessMillis)
41 -         .add("expireAfterWriteMillis", expireAfterWriteMillis)
42         .toString();
43     }
44 }

```

Sample 10 GitHub Modified

1	+static class DurationSpec {
2	+ private final long duration;
3	+ private final TimeUnit unit;
4	
5	+ private DurationSpec(long duration, TimeUnit unit) {
6	+ this.duration = duration;
7	+ this.unit = unit;
8	}
9	
10	+ public static DurationSpec of(long duration, TimeUnit unit) {
11	+ return new DurationSpec(duration, unit);
12	}
13	
14	@Override
15	public int hashCode() {
16	+ return Objects.hashCode(duration, unit);
17	}
18	
19	@Override
20	public boolean equals(Object o) {
21	+ if (o instanceof DurationSpec) {
22	+ DurationSpec that = (DurationSpec) o;
23	+ return unit.toNanos(duration) == that.unit.toNanos(that.duration);
24	}
25	return false;
26	}
27	
28	@Override
29	public String toString() {
30	return Objects.toStringHelper(this)
31	+ .add("duration", duration)
32	+ .add("unit", unit)
33	.toString();
34	}
35	}

Sample 10 Mergely Original

```

1 static class ExpirationSpec {
2     @Nullable
3     private final Long expireAfterAccessMillis;
4     @Nullable
5     private final Long expireAfterWriteMillis;
6
7     private ExpirationSpec(Long expireAfterAccessMillis, Long expireAfterWriteMillis) {
8         Preconditions.checkArgument(
9             expireAfterAccessMillis == null || expireAfterWriteMillis == null);
10        this.expireAfterAccessMillis = expireAfterAccessMillis;
11        this.expireAfterWriteMillis = expireAfterWriteMillis;
12    }
13
14    public static ExpirationSpec afterAccess(long afterAccess, TimeUnit unit) {
15        return new ExpirationSpec(unit.toMillis(afterAccess), null);
16    }
17
18    public static ExpirationSpec afterWrite(long afterWrite, TimeUnit unit) {
19        return new ExpirationSpec(null, unit.toMillis(afterWrite));
20    }
21
22    @Override
23    public int hashCode() {
24        return Objects.hashCode(expireAfterAccessMillis, expireAfterWriteMillis);
25    }
26
27    @Override
28    public boolean equals(Object o) {
29        if (o instanceof ExpirationSpec) {
30            ExpirationSpec that = (ExpirationSpec) o;
31            return Objects.equal(this.expireAfterAccessMillis, that.expireAfterAccessMillis)
32                && Objects.equal(this.expireAfterWriteMillis, that.expireAfterWriteMillis);
33        }
34        return false;
35    }
36
37    @Override
38    public String toString() {
39        return Objects.toStringHelper(this)
40            .add("expireAfterAccessMillis", expireAfterAccessMillis)
41            .add("expireAfterWriteMillis", expireAfterWriteMillis)
42            .toString();
43    }
44 }
45

```

Sample 10 Mergely Modified

```

1 static class DurationSpec {
2     private final long duration;
3     private final TimeUnit unit;
4
5     private DurationSpec(long duration, TimeUnit unit) {
6         this.duration = duration;
7         this.unit = unit;
8     }
9
10    public static DurationSpec of(long duration, TimeUnit unit) {
11        return new DurationSpec(duration, unit);
12    }
13
14    @Override
15    public int hashCode() {
16        return Objects.hashCode(duration, unit);
17    }
18
19    @Override
20    public boolean equals(Object o) {
21        if (o instanceof DurationSpec) {
22            DurationSpec that = (DurationSpec) o;
23            return unit.toNanos(duration) == that.unit.toNanos(that.duration);
24        }
25        return false;
26    }
27
28    @Override
29    public String toString() {
30        return Objects.toStringHelper(this)
31            .add("duration", duration)
32            .add("unit", unit)
33            .toString();
34    }
35 }
36

```

Sample 10 srcDiff Original

Original

```

1 static class ExpirationSpec {
2     @Nullable
3     private final Long expireAfterAccessMillis;
4     @Nullable
5     private final Long expireAfterWriteMillis;
6
7     private ExpirationSpec(Long expireAfterAccessMillis, Long expireAfterWriteMillis) {
8         Preconditions.checkArgument(
9             expireAfterAccessMillis == null || expireAfterWriteMillis == null);
10        this.expireAfterAccessMillis = expireAfterAccessMillis;
11        this.expireAfterWriteMillis = expireAfterWriteMillis;
12    }
13
14    public static ExpirationSpec afterAccess(long afterAccess, TimeUnit unit) {
15        return new ExpirationSpec(unit.toMillis(afterAccess), null);
16    }
17
18    public static ExpirationSpec afterWrite(long afterWrite, TimeUnit unit) {
19        return new ExpirationSpec(null, unit.toMillis(afterWrite));
20    }
21
22    @Override
23    public int hashCode() {
24        return Objects.hashCode(expireAfterAccessMillis, expireAfterWriteMillis);
25    }
26
27    @Override
28    public boolean equals(Object o) {
29        if (o instanceof ExpirationSpec) {
30            ExpirationSpec that = (ExpirationSpec) o;
31            return Objects.equal(this.expireAfterAccessMillis, that.expireAfterAccessMillis)
32                && Objects.equal(this.expireAfterWriteMillis, that.expireAfterWriteMillis);
33        }
34
35        return false;
36    }
37
38    @Override
39    public String toString() {
40        return Objects.toStringHelper(this)
41            .add("expireAfterAccessMillis", expireAfterAccessMillis)
42            .add("expireAfterWriteMillis", expireAfterWriteMillis)
43            .toString();
44    }
45 }

```

Sample 10 srcDiff Modified

```

                                     Modified
1 static class DurationSpec {

2     private final long duration;
3     private final TimeUnit unit;
4
5     private DurationSpec(long duration, TimeUnit unit) {
6         this.duration = duration;
7         this.unit = unit;
8     }
9
10    public static DurationSpec of(long duration, TimeUnit unit) {
11        return new DurationSpec(duration, unit);
12    }
13
14    @Override
15    public int hashCode() {
16        return Objects.hashCode(duration, unit);
17    }
18
19    @Override
20    public boolean equals(Object o) {

21        if (o instanceof DurationSpec) {
22            DurationSpec that = (DurationSpec) o;
23            return unit.toNanos(duration) == that.unit.toNanos(that.duration);
24        }
25        return false;
26    }
27
28    @Override
29    public String toString() {
30        return Objects.toStringHelper(this)
31            .add("duration", duration)
32            .add("unit", unit)
33            .toString();
34    }
35 }
36

```

Sample 10 GumTree Original

```

static class ExpirationSpec {
    @Nullable
    private final Long expireAfterAccessMillis;
    @Nullable
    private final Long expireAfterWriteMillis;

    private ExpirationSpec(Long expireAfterAccessMillis, Long expireAfterWriteMillis) {
        Preconditions.checkArgument(
            expireAfterAccessMillis == null || expireAfterWriteMillis == null);
        this.expireAfterAccessMillis = expireAfterAccessMillis;
        this.expireAfterWriteMillis = expireAfterWriteMillis;
    }

    public static ExpirationSpec afterAccess(long afterAccess, TimeUnit unit) {
        return new ExpirationSpec(unit.toMillis(afterAccess), null);
    }

    public static ExpirationSpec afterWrite(long afterWrite, TimeUnit unit) {
        return new ExpirationSpec(null, unit.toMillis(afterWrite));
    }

    @Override
    public int hashCode() {
        return Objects.hashCode(expireAfterAccessMillis, expireAfterWriteMillis);
    }

    @Override
    public boolean equals(Object o) {
        if (o instanceof ExpirationSpec) {
            ExpirationSpec that = (ExpirationSpec) o;
            return Objects.equal(this.expireAfterAccessMillis, that.expireAfterAccessMillis)
                && Objects.equal(this.expireAfterWriteMillis, that.expireAfterWriteMillis);
        }
        return false;
    }

    @Override
    public String toString() {
        return Objects.toStringHelper(this)
            .add("expireAfterAccessMillis", expireAfterAccessMillis)
            .add("expireAfterWriteMillis", expireAfterWriteMillis)
            .toString();
    }
}

```

SingleVariableDeclaration/SimpleName

Sample 10 GumTree Modified

```

static class DurationSpec {
    private final long duration;
    private final TimeUnit unit;

    private DurationSpec(long duration, TimeUnit unit) {
        this.duration = duration;
        this.unit = unit;
    }

    public static DurationSpec of(long duration, TimeUnit unit) {
        return new DurationSpec(duration, unit);
    }

    @Override
    public int hashCode() {
        return Objects.hashCode(duration, unit);
    }

    @Override
    public boolean equals(Object o) {
        if (o instanceof DurationSpec) {
            DurationSpec that = (DurationSpec) o;
            return unit.toNanos(duration) == that.unit.toNanos(that.duration);
        }
        return false;
    }

    @Override
    public String toString() {
        return Objects.toStringHelper(this)
            .add("duration", duration)
            .add("unit", unit)
            .toString();
    }
}

```

Sample 11 Source Original

Original
<pre> public void expandFolds(int foldLevel, boolean update) { if(buffer.getFoldHandler() instanceof IndentFoldHandler) foldLevel = (foldLevel - 1) * buffer.getIndentSize() + 1; showLineRange(0,buffer.getLineCount() - 1); int leastFolded = -1; int firstInvisible = 0; for(int i = 0; i < buffer.getLineCount(); i++) { if (leastFolded == -1 buffer.getFoldLevel(i) < leastFolded) { leastFolded = buffer.getFoldLevel(i); } if (buffer.getFoldLevel(i) < foldLevel buffer.getFoldLevel(i) == leastFolded) { if(firstInvisible != i) { hideLineRange(firstInvisible, i - 1); } firstInvisible = i + 1; } } if(firstInvisible != buffer.getLineCount()) hideLineRange(firstInvisible,buffer.getLineCount() - 1); notifyScreenLineChanges(); if(update && textArea.getDisplayManager() == this) { textArea.foldStructureChanged(); } } </pre>

Sample 11 Source Modified

Modified
<pre> public void expandFolds(int foldLevel, boolean update) { if(buffer.getFoldHandler() instanceof IndentFoldHandler) foldLevel = (foldLevel - 1) * buffer.getIndentSize() + 1; int lineCount = buffer.getLineCount(); int end = lineCount - 1; showLineRange(0,end); int leastFolded = -1; int firstInvisible = 0; for(int i = 0; i < lineCount; i++) { int level = buffer.getFoldLevel(i); if (leastFolded == -1 level < leastFolded) { leastFolded = level; } if (level < foldLevel level == leastFolded) { if(firstInvisible != i) { hideLineRange(firstInvisible, i - 1); } firstInvisible = i + 1; } } if(firstInvisible != lineCount) hideLineRange(firstInvisible,end); notifyScreenLineChanges(); if(update && textArea.getDisplayManager() == this) { textArea.foldStructureChanged(); } } </pre>

Sample 11 GitHub Original

...	@@ -1,22 +1,24 @@
1	public void expandFolds(int foldLevel, boolean update)
2	{
3	if(buffer.getFoldHandler() instanceof IndentFoldHandler)
4	foldLevel = (foldLevel - 1) * buffer.getIndentSize() + 1;
5	
6	- showLineRange(0,buffer.getLineCount() - 1);
7	
8	int leastFolded = -1;
9	int firstInvisible = 0;
10	
11	- for(int i = 0; i < buffer.getLineCount(); i++)
12	{
13	- if (leastFolded == -1 buffer.getFoldLevel(i) < leastFolded)
14	{
15	- leastFolded = buffer.getFoldLevel(i);
16	}
17	
18	- if (buffer.getFoldLevel(i) < foldLevel
19	- buffer.getFoldLevel(i) == leastFolded)
20	{
21	if(firstInvisible != i)
22	{
23	hideLineRange(firstInvisible,
24	i - 1);
25	}
26	firstInvisible = i + 1;
27	}
28	}
29	
30	- if(firstInvisible != buffer.getLineCount())
31	- hideLineRange(firstInvisible,buffer.getLineCount() - 1);
32	
33	notifyScreenLineChanges();
34	if(update && textArea.getDisplayManager() == this)
35	{
36	textArea.foldStructureChanged();
37	}
38	}

Sample 11 GitHub Modified

1	<code>public void expandFolds(int foldLevel, boolean update)</code>
2	<code>{</code>
3	<code> if(buffer.getFoldHandler() instanceof IndentFoldHandler)</code>
4	<code> foldLevel = (foldLevel - 1) * buffer.getIndentSize() + 1;</code>
5	
6	<code>+ int lineCount = buffer.getLineCount();</code>
7	<code>+ int end = lineCount - 1;</code>
8	<code>+ showLineRange(0,end);</code>
9	
10	<code> int leastFolded = -1;</code>
11	<code> int firstInvisible = 0;</code>
12	
13	<code>+ for(int i = 0; i < lineCount; i++)</code>
14	<code>{</code>
15	<code>+ int level = buffer.getFoldLevel(i);</code>
16	<code>+ if (leastFolded == -1 level < leastFolded)</code>
17	<code>{</code>
18	<code>+ leastFolded = level;</code>
19	<code>}</code>
20	
21	<code>+ if (level < foldLevel level == leastFolded)</code>
22	<code>{</code>
23	<code> if(firstInvisible != i)</code>
24	<code>{</code>
25	<code> hideLineRange(firstInvisible,</code>
26	<code> i - 1);</code>
27	<code>}</code>
28	<code> firstInvisible = i + 1;</code>
29	<code>}</code>
30	<code>}</code>
31	
32	<code>+ if(firstInvisible != lineCount)</code>
33	<code>+ hideLineRange(firstInvisible,end);</code>
34	
35	<code> notifyScreenLineChanges();</code>
36	<code> if(update && textArea.getDisplayManager() == this)</code>
37	<code>{</code>
38	<code> textArea.foldStructureChanged();</code>
39	<code>}</code>
40	<code>}</code>

Sample 11 Mergely Original

```

1 public void expandFolds(int foldLevel, boolean update)
2 {
3     if(buffer.getFoldHandler() instanceof IndentFoldHandler)
4         foldLevel = (foldLevel - 1) * buffer.getIndentSize() + 1;
5
6     showLineRange(0,buffer.getLineCount()-1);
7
8     int leastFolded = -1;
9     int firstInvisible = 0;
10
11     for(int i = 0; i < buffer.getLineCount(); i++)
12     {
13         if (leastFolded == -1 || buffer.getFoldLevel(i) < leastFolded)
14         {
15             leastFolded = buffer.getFoldLevel(i);
16         }
17
18         if (buffer.getFoldLevel(i) < foldLevel ||
19             buffer.getFoldLevel(i) == leastFolded)
20         {
21             if(firstInvisible != i)
22             {
23                 hideLineRange(firstInvisible,
24                     i - 1);
25             }
26             firstInvisible = i + 1;
27         }
28     }
29
30     if(firstInvisible != buffer.getLineCount();
31        hideLineRange(firstInvisible,buffer.getLineCount()-1);
32
33     notifyScreenLineChanges();
34     if(update && textArea.getDisplayManager() == this)
35     {
36         textArea.foldStructureChanged();
37     }
38 }
39

```

Sample 11 Mergely Modified

```

1 public void expandFolds(int foldLevel, boolean update)
2 {
3     if(buffer.getFoldHandler() instanceof IndentFoldHandler)
4         foldLevel = (foldLevel - 1) * buffer.getIndentSize() + 1;
5
6     int lineCount = buffer.getLineCount();
7     int end = lineCount - 1;
8     showLineRange(0,end);
9
10    int leastFolded = -1;
11    int firstInvisible = 0;
12
13    for(int i = 0; i < lineCount; i++)
14    {
15        int level = buffer.getFoldLevel(i);
16        if (leastFolded == -1 || level < leastFolded)
17        {
18            leastFolded = level;
19        }
20
21        if (level < foldLevel || level == leastFolded)
22        {
23            if(firstInvisible != i)
24            {
25                hideLineRange(firstInvisible,
26                    i - 1);
27            }
28            firstInvisible = i + 1;
29        }
30    }
31
32    if(firstInvisible != lineCount)
33        hideLineRange(firstInvisible,end);
34
35    notifyScreenLineChanges();
36    if(update && textArea.getDisplayManager() == this)
37    {
38        textArea.foldStructureChanged();
39    }
40 }
41

```

Sample 11 srcDiff Original

Original

```

1 public void expandFolds(int foldLevel, boolean update)
2 {
3     if(buffer.getFoldHandler() instanceof IndentFoldHandler)
4         foldLevel = (foldLevel - 1) * buffer.getIndentSize() + 1;
5
6     showLineRange(0, buffer.getLineCount() - 1);
7
8     int leastFolded = -1;
9     int firstInvisible = 0;
10
11     for(int i = 0; i < buffer.getLineCount(); i++)
12     {
13         if (leastFolded == -1 || buffer.getFoldLevel(i) < leastFolded)
14         {
15             leastFolded = buffer.getFoldLevel(i);
16         }
17
18         if (buffer.getFoldLevel(i) < foldLevel ||
19             buffer.getFoldLevel(i) == leastFolded)
20         {
21             if(firstInvisible != i)
22             {
23                 hideLineRange(firstInvisible,
24                     i - 1);
25             }
26             firstInvisible = i + 1;
27         }
28     }
29
30     if(firstInvisible != buffer.getLineCount())
31         hideLineRange(firstInvisible, buffer.getLineCount() - 1);
32
33     notifyScreenLineChanges();
34     if(update && textArea.getDisplayManager() == this)
35     {
36         textArea.foldStructureChanged();
37     }
38 }
39

```


Sample 11 srcDiff Modified

Modified

```

1 public void expandFolds(int foldLevel, boolean update)
2 {
3     if(buffer.getFoldHandler() instanceof IndentFoldHandler)
4         foldLevel = (foldLevel - 1) * buffer.getIndentSize() + 1;
5
6     int lineCount = buffer.getLineCount();
7     int end = lineCount - 1;
8     showLineRange(0, end);
9
10    int leastFolded = -1;
11    int firstInvisible = 0;
12
13    for(int i = 0; i < lineCount; i++)
14    {
15        int level = buffer.getFoldLevel(i);
16        if (leastFolded == -1 || level < leastFolded)
17        {
18            leastFolded = level;
19        }
20
21        if (level < foldLevel ||
21        level == leastFolded)
22        {
23            if(firstInvisible != i)
24            {
25                hideLineRange(firstInvisible,
26                    i - 1);
27            }
28            firstInvisible = i + 1;
29        }
30    }
31
32    if(firstInvisible != lineCount)
33        hideLineRange(firstInvisible, end);
34
35    notifyScreenLineChanges();
36    if(update && textArea.getDisplayManager() == this)
37    {
38        textArea.foldStructureChanged();
39    }
40 }
41

```

Sample 11 GumTree Original

```

public void expandFolds(int foldLevel, boolean update)
{
    if(buffer.getFoldHandler() instanceof IndentFoldHandler)
        foldLevel = (foldLevel - 1) * buffer.getIndentSize() + 1;

    showLineRange(0,buffer.getLineCount() - 1);

    int leastFolded = -1;
    int firstInvisible = 0;

    for(int i = 0; i < buffer.getLineCount(); i++)
    {
        if (leastFolded == -1 || buffer.getFoldLevel(i) < leastFolded)
        {
            leastFolded = buffer.getFoldLevel(i);
        }

        if (buffer.getFoldLevel(i) < foldLevel ||
            buffer.getFoldLevel(i) == leastFolded)
        {
            if(firstInvisible != i)
            {
                hideLineRange(firstInvisible,
                    i - 1);
            }
            firstInvisible = i + 1;
        }
    }

    if(firstInvisible != buffer.getLineCount())
        hideLineRange(firstInvisible,buffer.getLineCount() - 1);

    notifyScreenLineChanges();
    if(update && textArea.getDisplayManager() == this)
    {
        textArea.foldStructureChanged();
    }
}

```

Sample 11 GumTree Modified

```

public void expandFolds(int foldLevel, boolean update)
{
    if(buffer.getFoldHandler() instanceof IndentFoldHandler)
        foldLevel = (foldLevel - 1) * buffer.getIndentSize() + 1;

    int lineCount = buffer.getLineCount();
    int end = lineCount - 1;
    showLineRange(0,end);

    int leastFolded = -1;
    int firstInvisible = 0;

    for(int i = 0; i < lineCount; i++)
    {
        int level = buffer.getFoldLevel(i);
        if (leastFolded == -1 || level < leastFolded)
        {
            leastFolded = level;
        }

        if (level < foldLevel || level == leastFolded)
        {
            if(firstInvisible != i)
            {
                hideLineRange(firstInvisible,
                    i - 1);
            }
            firstInvisible = i + 1;
        }
    }

    if(firstInvisible != lineCount)
        hideLineRange(firstInvisible,end);

    notifyScreenLineChanges();
    if(update && textArea.getDisplayManager() == this)
    {
        textArea.foldStructureChanged();
    }
}

```

Sample 12 Source Original

Original
<pre> void physDown(int amount, int screenAmount) { skew = 0; if(!isLineVisible(physicalLine)) { int lastVisibleLine = getLastVisibleLine(); if(physicalLine > lastVisibleLine) physicalLine = lastVisibleLine; else { int nextPhysicalLine = getNextVisibleLine(physicalLine); amount -= (nextPhysicalLine - physicalLine); scrollLine += getScreenLineCount(physicalLine); physicalLine = nextPhysicalLine; } } for(;;) { int nextPhysicalLine = getNextVisibleLine(physicalLine); if(nextPhysicalLine == -1) break; else if(nextPhysicalLine > physicalLine + amount) break; else { scrollLine += getScreenLineCount(physicalLine); amount -= (nextPhysicalLine - physicalLine); physicalLine = nextPhysicalLine; } } if(screenAmount != 0) { if(screenAmount < 0) scrollUp(-screenAmount); else scrollDown(screenAmount); } } </pre>

Sample 12 Source Modified

Modified
<pre> void physDown(int amount, int screenAmount) { skew = 0; if(!isLineVisible(physicalLine)) { int lastVisibleLine = getLastVisibleLine(); if(physicalLine > lastVisibleLine) physicalLine = lastVisibleLine; else { int nextPhysicalLine = getNextVisibleLine(physicalLine); amount -= (nextPhysicalLine - physicalLine); scrollLine += getScreenLineCount(physicalLine); physicalLine = nextPhysicalLine; } } for(;;) { int nextPhysicalLine = getNextVisibleLine(physicalLine); if(nextPhysicalLine == -1) break; else if(nextPhysicalLine > physicalLine + amount) break; else { scrollLine += getScreenLineCount(physicalLine); amount -= (nextPhysicalLine - physicalLine); physicalLine = nextPhysicalLine; } } if(screenAmount < 0) scrollUp(-screenAmount); else if(screenAmount > 0) scrollDown(screenAmount); } </pre>

Sample 12 GitHub Original

```

... @@ -1,42 +1,39 @@
1  void physDown(int amount, int screenAmount)
2  {
3      skew = 0;
4
5      if(!isLineVisible(physicalLine))
6      {
7          int lastVisibleLine = getLastVisibleLine();
8          if(physicalLine > lastVisibleLine)
9              physicalLine = lastVisibleLine;
10         else
11         {
12             int nextPhysicalLine = getNextVisibleLine(physicalLine);
13             amount -= (nextPhysicalLine - physicalLine);
14             scrollLine += getScreenLineCount(physicalLine);
15             physicalLine = nextPhysicalLine;
16         }
17     }
18
19     for(;;)
20     {
21         int nextPhysicalLine = getNextVisibleLine(
22             physicalLine);
23         if(nextPhysicalLine == -1)
24             break;
25         else if(nextPhysicalLine > physicalLine + amount)
26             break;
27         else
28         {
29             scrollLine += getScreenLineCount(physicalLine);
30             amount -= (nextPhysicalLine - physicalLine);
31             physicalLine = nextPhysicalLine;
32         }
33     }
34
35 - if(screenAmount != 0)
36 - {
37     if(screenAmount < 0)
38         scrollUp(-screenAmount);
39 - else
40     scrollDown(screenAmount);
41 }
42 -}

```

Sample 12 GitHub Modified

```

1  void physDown(int amount, int screenAmount)
2  {
3      skew = 0;
4
5      if(!isLineVisible(physicalLine))
6      {
7          int lastVisibleLine = getLastVisibleLine();
8          if(physicalLine > lastVisibleLine)
9              physicalLine = lastVisibleLine;
10         else
11         {
12             int nextPhysicalLine = getNextVisibleLine(physicalLine);
13             amount -= (nextPhysicalLine - physicalLine);
14             scrollLine += getScreenLineCount(physicalLine);
15             physicalLine = nextPhysicalLine;
16         }
17     }
18
19     for(;;)
20     {
21         int nextPhysicalLine = getNextVisibleLine(
22             physicalLine);
23         if(nextPhysicalLine == -1)
24             break;
25         else if(nextPhysicalLine > physicalLine + amount)
26             break;
27         else
28         {
29             scrollLine += getScreenLineCount(physicalLine);
30             amount -= (nextPhysicalLine - physicalLine);
31             physicalLine = nextPhysicalLine;
32         }
33     }
34
35     if(screenAmount < 0)
36         scrollUp(-screenAmount);
37 + else if(screenAmount > 0)
38         scrollDown(screenAmount);
39 }

```

Sample 12 Mergely Original

```

1 void physDown(int amount, int screenAmount)
2 {
3     skew = 0;
4
5     if(!isLineVisible(physicalLine))
6     {
7         int lastVisibleLine = getLastVisibleLine();
8         if(physicalLine > lastVisibleLine)
9             physicalLine = lastVisibleLine;
10        else
11        {
12            int nextPhysicalLine = getNextVisibleLine(physicalLine);
13            amount -= (nextPhysicalLine - physicalLine);
14            scrollLine += getScreenLineCount(physicalLine);
15            physicalLine = nextPhysicalLine;
16        }
17    }
18
19    for(;;)
20    {
21        int nextPhysicalLine = getNextVisibleLine(
22            physicalLine);
23        if(nextPhysicalLine == -1)
24            break;
25        else if(nextPhysicalLine > physicalLine + amount)
26            break;
27        else
28        {
29            scrollLine += getScreenLineCount(physicalLine);
30            amount -= (nextPhysicalLine - physicalLine);
31            physicalLine = nextPhysicalLine;
32        }
33    }
34
35 if(screenAmount != 0)
36 {
37     if(screenAmount < 0)
38         scrollUp(-screenAmount);
39 else
40     scrollDown(screenAmount);
41 }
42 }
43

```


Sample 12 Mergely Modified

```

1 void physDown(int amount, int screenAmount)
2 {
3     skew = 0;
4
5     if(!isLineVisible(physicalLine))
6     {
7         int lastVisibleLine = getLastVisibleLine();
8         if(physicalLine > lastVisibleLine)
9             physicalLine = lastVisibleLine;
10        else
11        {
12            int nextPhysicalLine = getNextVisibleLine(physicalLine);
13            amount -= (nextPhysicalLine - physicalLine);
14            scrollLine += getScreenLineCount(physicalLine);
15            physicalLine = nextPhysicalLine;
16        }
17    }
18
19    for(;;)
20    {
21        int nextPhysicalLine = getNextVisibleLine(
22            physicalLine);
23        if(nextPhysicalLine == -1)
24            break;
25        else if(nextPhysicalLine > physicalLine + amount)
26            break;
27        else
28        {
29            scrollLine += getScreenLineCount(physicalLine);
30            amount -= (nextPhysicalLine - physicalLine);
31            physicalLine = nextPhysicalLine;
32        }
33    }
34
35    if(screenAmount < 0)
36        scrollUp(-screenAmount);
37    else if(screenAmount > 0)
38        scrollDown(screenAmount);
39 }
40

```

Sample 12 srcDiff Original

Original

```

1 void physDown(int amount, int screenAmount)
2 {
3     skew = 0;
4
5     if(!isLineVisible(physicalLine))
6     {
7         int lastVisibleLine = getLastVisibleLine();
8         if(physicalLine > lastVisibleLine)
9             physicalLine = lastVisibleLine;
10        else
11        {
12            int nextPhysicalLine = getNextVisibleLine(physicalLine);
13            amount -= (nextPhysicalLine - physicalLine);
14            scrollLine += getScreenLineCount(physicalLine);
15            physicalLine = nextPhysicalLine;
16        }
17    }
18
19    for(;;)
20    {
21        int nextPhysicalLine = getNextVisibleLine(
22            physicalLine);
23        if(nextPhysicalLine == -1)
24            break;
25        else if(nextPhysicalLine > physicalLine + amount)
26            break;
27        else
28        {
29            scrollLine += getScreenLineCount(physicalLine);
30            amount -= (nextPhysicalLine - physicalLine);
31            physicalLine = nextPhysicalLine;
32        }
33    }
34
35    if(screenAmount != 0)
36    {
37        if(screenAmount < 0)
38            scrollUp(-screenAmount);
39        else
40            scrollDown(screenAmount);
41    }
42 }
43

```

Sample 12 srcDiff Modified

Modified

```

1 void physDown(int amount, int screenAmount)
2 {
3     skew = 0;
4
5     if(!isLineVisible(physicalLine))
6     {
7         int lastVisibleLine = getLastVisibleLine();
8         if(physicalLine > lastVisibleLine)
9             physicalLine = lastVisibleLine;
10        else
11        {
12            int nextPhysicalLine = getNextVisibleLine(physicalLine);
13            amount -= (nextPhysicalLine - physicalLine);
14            scrollLine += getScreenLineCount(physicalLine);
15            physicalLine = nextPhysicalLine;
16        }
17    }
18
19    for(;;)
20    {
21        int nextPhysicalLine = getNextVisibleLine(
22            physicalLine);
23        if(nextPhysicalLine == -1)
24            break;
25        else if(nextPhysicalLine > physicalLine + amount)
26            break;
27        else
28        {
29            scrollLine += getScreenLineCount(physicalLine);
30            amount -= (nextPhysicalLine - physicalLine);
31            physicalLine = nextPhysicalLine;
32        }
33    }
34
35    if(screenAmount < 0)
36        scrollUp(-screenAmount);
37    else if(screenAmount > 0)
38        scrollDown(screenAmount);
39 }
40

```

Sample 12 GumTree Original

```

void physDown(int amount, int screenAmount)
{
    skew = 0;

    if(!isLineVisible(physicalLine))
    {
        int lastVisibleLine = getLastVisibleLine();
        if(physicalLine > lastVisibleLine)
            physicalLine = lastVisibleLine;
        else
        {
            int nextPhysicalLine = getNextVisibleLine(physicalLine);
            amount -= (nextPhysicalLine - physicalLine);
            scrollLine += getScreenLineCount(physicalLine);
            physicalLine = nextPhysicalLine;
        }
    }

    for(;;)
    {
        int nextPhysicalLine = getNextVisibleLine(
            physicalLine);
        if(nextPhysicalLine == -1)
            break;
        else if(nextPhysicalLine > physicalLine + amount)
            break;
        else
        {
            scrollLine += getScreenLineCount(physicalLine);
            amount -= (nextPhysicalLine - physicalLine);
            physicalLine = nextPhysicalLine;
        }
    }

    if(screenAmount != 0)
    {
        if(screenAmount < 0)
            scrollUp(-screenAmount);
        else
            scrollDown(screenAmount);
    }
}

```

Sample 12 GumTree Modified

```

void physDown(int amount, int screenAmount)
{
    skew = 0;

    if(!isLineVisible(physicalLine))
    {
        int lastVisibleLine = getLastVisibleLine();
        if(physicalLine > lastVisibleLine)
            physicalLine = lastVisibleLine;
        else
        {
            int nextPhysicalLine = getNextVisibleLine(physicalLine);
            amount -= (nextPhysicalLine - physicalLine);
            scrollLine += getScreenLineCount(physicalLine);
            physicalLine = nextPhysicalLine;
        }
    }

    for(;;)
    {
        int nextPhysicalLine = getNextVisibleLine(
            physicalLine);
        if(nextPhysicalLine == -1)
            break;
        else if(nextPhysicalLine > physicalLine + amount)
            break;
        else
        {
            scrollLine += getScreenLineCount(physicalLine);
            amount -= (nextPhysicalLine - physicalLine);
            physicalLine = nextPhysicalLine;
        }
    }

    if(screenAmount < 0)
        scrollUp(-screenAmount);
    else if(screenAmount > 0)
        scrollDown(screenAmount);
}

```

Sample 13 Source Original

Original
<pre> @Beta @Nullable @CheckForNull public static Long tryParse(String string) { if (checkNotNull(string).isEmpty()) { return null; } boolean negative = string.charAt(0) == '-'; int index = negative ? 1 : 0; if (index == string.length()) { return null; } int digit = string.charAt(index++) - '0'; if (digit < 0 digit > 9) { return null; } long accum = -digit; while (index < string.length()) { digit = string.charAt(index++) - '0'; if (digit < 0 digit > 9 accum < Long.MIN_VALUE / 10) { return null; } accum *= 10; if (accum < Long.MIN_VALUE + digit) { return null; } accum -= digit; } if (negative) { return accum; } else if (accum == Long.MIN_VALUE) { return null; } else { return -accum; } } </pre>

Sample 13 Source Modified

Modified
<pre> @Beta @Nullable @CheckForNull public static Long tryParse(String string, int radix) { if (checkNotNull(string).isEmpty()) { return null; } if (radix < Character.MIN_RADIX radix > Character.MAX_RADIX) { throw new IllegalArgumentException("radix must be between MIN_RADIX and MAX_RADIX but was " + radix); } boolean negative = string.charAt(0) == '-'; int index = negative ? 1 : 0; if (index == string.length()) { return null; } int digit = digit(string.charAt(index++)); if (digit < 0 digit >= radix) { return null; } long accum = -digit; long cap = Long.MIN_VALUE / radix; while (index < string.length()) { digit = digit(string.charAt(index++)); if (digit < 0 digit >= radix accum < cap) { return null; } accum *= radix; if (accum < Long.MIN_VALUE + digit) { return null; } accum -= digit; } if (negative) { return accum; } else if (accum == Long.MIN_VALUE) { return null; } else { return -accum; } } </pre>

Sample 13 GitHub Original

```

... @@ -1,26 +1,33 @@
1  @Beta
2  @Nullable
3  @CheckForNull
4  -public static Long tryParse(String string) {
5      if (checkNotNull(string).isEmpty()) {
6          return null;
7      }

8      boolean negative = string.charAt(0) == '-';
9      int index = negative ? 1 : 0;
10     if (index == string.length()) {
11         return null;
12     }
13     - int digit = string.charAt(index++) - '0';
14     - if (digit < 0 || digit > 9) {
15         return null;
16     }
17     long accum = -digit;

18     while (index < string.length()) {
19     - digit = string.charAt(index++) - '0';
20     - if (digit < 0 || digit > 9 || accum < Long.MIN_VALUE / 10) {
21         return null;
22     }
23     - accum *= 10;
24     if (accum < Long.MIN_VALUE + digit) {
25         return null;
26     }
27     accum -= digit;
28     }
29
30     if (negative) {
31         return accum;
32     } else if (accum == Long.MIN_VALUE) {
33         return null;
34     } else {
35         return -accum;
36     }
37 }

```


Sample 13 GitHub Modified

```

1  @Beta
2  @Nullable
3  @CheckForNull
4  +public static Long tryParse(String string, int radix) {
5      if (checkNotNull(string).isEmpty()) {
6          return null;
7      }
8  + if (radix < Character.MIN_RADIX || radix > Character.MAX_RADIX) {
9  +     throw new IllegalArgumentException(
10 +         "radix must be between MIN_RADIX and MAX_RADIX but was " + radix);
11 + }
12     boolean negative = string.charAt(0) == '-';
13     int index = negative ? 1 : 0;
14     if (index == string.length()) {
15         return null;
16     }
17 + int digit = digit(string.charAt(index++));
18 + if (digit < 0 || digit >= radix) {
19     return null;
20 }
21     long accum = -digit;
22 +
23 + long cap = Long.MIN_VALUE / radix;
24 +
25     while (index < string.length()) {
26 +     digit = digit(string.charAt(index++));
27 +     if (digit < 0 || digit >= radix || accum < cap) {
28         return null;
29     }
30 +     accum *= radix;
31     if (accum < Long.MIN_VALUE + digit) {
32         return null;
33     }
34     accum -= digit;
35 }
36
37     if (negative) {
38         return accum;
39     } else if (accum == Long.MIN_VALUE) {
40         return null;
41     } else {
42         return -accum;
43     }
44 }

```

Sample 13 Mergely Original

```

1  @Beta
2  @Nullable
3  @CheckForNull
4  public static Long tryParse(String string) {
5      if (checkNotNull(string).isEmpty()) {
6          return null;
7      }
8      boolean negative = string.charAt(0) == '-';
9      int index = negative ? 1 : 0;
10     if (index == string.length()) {
11         return null;
12     }
13     int digit = string.charAt(index++) - '0';
14     if (digit < 0 || digit > 9) {
15         return null;
16     }
17     long accum = -digit;
18     while (index < string.length()) {
19         digit = string.charAt(index++) - '0';
20         if (digit < 0 || digit > 9 || accum < Long.MIN_VALUE / 10) {
21             return null;
22         }
23         accum *= 10;
24         if (accum < Long.MIN_VALUE + digit) {
25             return null;
26         }
27         accum -= digit;
28     }
29
30     if (negative) {
31         return accum;
32     } else if (accum == Long.MIN_VALUE) {
33         return null;
34     } else {
35         return -accum;
36     }
37 }
38

```

Sample 13 Mergely Modified

```

1  @Beta
2  @Nullable
3  @CheckForNull
4  public static Long tryParse(String string, int radix) {
5      if (checkNotNull(string).isEmpty()) {
6          return null;
7      }
8      if (radix < Character.MIN_RADIX || radix > Character.MAX_RADIX) {
9          throw new IllegalArgumentException(
10             "radix must be between MIN_RADIX and MAX_RADIX but was " + radix);
11      }
12      boolean negative = string.charAt(0) == '-';
13      int index = negative ? 1 : 0;
14      if (index == string.length()) {
15          return null;
16      }
17      int digit = digit(string.charAt(index++));
18      if (digit < 0 || digit >= radix) {
19          return null;
20      }
21      long accum = -digit;
22
23      long cap = Long.MIN_VALUE / radix;
24
25      while (index < string.length()) {
26          digit = digit(string.charAt(index++));
27          if (digit < 0 || digit >= radix || accum < cap) {
28              return null;
29          }
30          accum *= radix;
31          if (accum < Long.MIN_VALUE + digit) {
32              return null;
33          }
34          accum -= digit;
35      }
36
37      if (negative) {
38          return accum;
39      } else if (accum == Long.MIN_VALUE) {
40          return null;
41      } else {
42          return -accum;
43      }
44  }
45

```

Sample 13 srcDiff Original

Original

```

1 @Beta
2 @Nullable
3 @CheckForNull
4 public static Long tryParse(String string) {
5     if (checkNotNull(string).isEmpty()) {
6         return null;
7     }

8     boolean negative = string.charAt(0) == '-';
9     int index = negative ? 1 : 0;
10    if (index == string.length()) {
11        return null;
12    }
13    int digit = string.charAt(index++) - '0';
14    if (digit < 0 || digit > 9) {
15        return null;
16    }
17    long accum = -digit;

18    while (index < string.length()) {
19        digit = string.charAt(index++) - '0';
20        if (digit < 0 || digit > 9 || accum < Long.MIN_VALUE / 10) {
21            return null;
22        }
23        accum *= 10;
24        if (accum < Long.MIN_VALUE + digit) {
25            return null;
26        }
27        accum -= digit;
28    }
29
30    if (negative) {
31        return accum;
32    } else if (accum == Long.MIN_VALUE) {
33        return null;
34    } else {
35        return -accum;
36    }
37 }
38

```

Sample 13 srcDiff Modified

Modified

```

1 @Beta
2 @Nullable
3 @CheckForNull
4 public static Long tryParse(String string, int radix) {
5     if (checkNotNull(string).isEmpty()) {
6         return null;
7     }
8     if (radix < Character.MIN_RADIX || radix > Character.MAX_RADIX) {
9         throw new IllegalArgumentException(
10             "radix must be between MIN_RADIX and MAX_RADIX but was " + radix);
11     }
12     boolean negative = string.charAt(0) == '-';
13     int index = negative ? 1 : 0;
14     if (index == string.length()) {
15         return null;
16     }
17     int digit = digit(string.charAt(index++));
18     if (digit < 0 || digit >= radix) {
19         return null;
20     }
21     long accum = -digit;
22
23     long cap = Long.MIN_VALUE / radix;
24
25     while (index < string.length()) {
26         digit = digit(string.charAt(index++));
27         if (digit < 0 || digit >= radix || accum < cap) {
28             return null;
29         }
30         accum *= radix;
31         if (accum < Long.MIN_VALUE + digit) {
32             return null;
33         }
34         accum -= digit;
35     }
36
37     if (negative) {
38         return accum;
39     } else if (accum == Long.MIN_VALUE) {
40         return null;
41     } else {
42         return -accum;
43     }
44 }
45

```

Sample 13 GumTree Original

```

@Beta
@Nullable
@CheckForNull
public static Long tryParse(String string) {
    if (checkNotNull(string).isEmpty()) {
        return null;
    }
    boolean negative = string.charAt(0) == '-';
    int index = negative ? 1 : 0;
    if (index == string.length()) {
        return null;
    }
    int digit = string.charAt(index++) - '0';
    if (digit < 0 || digit > 9) {
        return null;
    }
    long accum = -digit;
    while (index < string.length()) {
        digit = string.charAt(index++) - '0';
        if (digit < 0 || digit > 9 || accum < Long.MIN_VALUE / 10) {
            return null;
        }
        accum *= 10;
        if (accum < Long.MIN_VALUE + digit) {
            return null;
        }
        accum -= digit;
    }

    if (negative) {
        return accum;
    } else if (accum == Long.MIN_VALUE) {
        return null;
    } else {
        return -accum;
    }
}

```

Sample 13 GumTree Modified

```

@Beta
@Nullable
@CheckForNull
public static Long tryParse(String string, int radix) {
    if (checkNotNull(string).isEmpty()) {
        return null;
    }
    if (radix < Character.MIN_RADIX || radix > Character.MAX_RADIX) {
        throw new IllegalArgumentException(
            "radix must be between MIN_RADIX and MAX_RADIX but was " + radix);
    }

    boolean negative = string.charAt(0) == '-';
    int index = negative ? 1 : 0;
    if (index == string.length()) {
        return null;
    }
    int digit = digit(string.charAt(index++));
    if (digit < 0 || digit >= radix) {
        return null;
    }
    long accum = -digit;

    long cap = Long.MIN_VALUE / radix;

    while (index < string.length()) {
        digit = digit(string.charAt(index++));
        if (digit < 0 || digit >= radix || accum < cap) {
            return null;
        }
        accum *= radix;
        if (accum < Long.MIN_VALUE + digit) {
            return null;
        }
        accum -= digit;
    }

    if (negative) {
        return accum;
    } else if (accum == Long.MIN_VALUE) {
        return null;
    } else {
        return -accum;
    }
}

```

Sample 14 Source Original

Original
<pre>public ImmutableSet<ClassInfo> getClasses(Package pkg) { return getClasses(pkg.getName()); }</pre>

Sample 14 Source Modified

Modified
<pre>public ImmutableSet<ClassInfo> getClassesRecursive(String packageName) { checkNotNull(packageName); String packagePrefix = packageName + '.'; ImmutableSet.Builder<ClassInfo> builder = ImmutableSet.builder(); for (ClassInfo classInfo : classes) { if (classInfo.getName().startsWith(packagePrefix)) { builder.add(classInfo); } } return builder.build(); }</pre>

Sample 14 GitHub Original

```
... @@ -1,3 +1,11 @@  
1 -public ImmutableSet<ClassInfo> getClasses(Package pkg) {  
2 -    return getClasses(pkg.getName());  
  
3 }
```

Sample 14 GitHub Modified

```
1 +public ImmutableSet<ClassInfo> getClassesRecursive(String packageName) {  
2 +    checkNotNull(packageName);  
3 +    String packagePrefix = packageName + '.';  
4 +    ImmutableSet.Builder<ClassInfo> builder = ImmutableSet.builder();  
5 +    for (ClassInfo classInfo : classes) {  
6 +        if (classInfo.getName().startsWith(packagePrefix)) {  
7 +            builder.add(classInfo);  
8 +        }  
9 +    }  
10 +    return builder.build();  
11 }
```

Sample 14 Mergely Original

```
1 public ImmutableSet<ClassInfo> getClasses(Package pkg) {  
2     return getClassess(pkg.getName());  
3 }  
4
```

Sample 14 Merely Modified

```
1 public ImmutableSet<ClassInfo> getClassessRecursive(String packageName) {  
2     checkNotNull(packageName);  
3     String packagePrefix = packageName + '.';  
4     ImmutableSet.Builder<ClassInfo> builder = ImmutableSet.builder();  
5     for (ClassInfo classInfo : classes) {  
6         if (classInfo.getName().startsWith(packagePrefix)) {  
7             builder.add(classInfo);  
8         }  
9     }  
10    return builder.build();  
11 }  
12
```

Sample 14 srcDiff Original

Original

```
1 public ImmutableSet<ClassInfo> getClasses(Package pkg) {  
2     return getClasses(pkg.getName());  
3 }
```

3
4

Sample 14 srcDiff Modified

Modified

```
1 public ImmutableSet<ClassInfo> getClassesRecursive(String packageName) {  
2     checkNotNull(packageName);  
3     String packagePrefix = packageName + '.';  
4     ImmutableSet.Builder<ClassInfo> builder = ImmutableSet.builder();  
5     for (ClassInfo classInfo : classes) {  
6         if (classInfo.getName().startsWith(packagePrefix)) {  
7             builder.add(classInfo);  
8         }  
9     }  
10    return builder.build();  
11 }  
12
```

Sample 14 GumTree Original

```
public ImmutableSet<ClassInfo> getClasses(Package pkg) {  
    return getClasses(pkg.getName());  
}
```

Sample 14 GumTree Modified

```
public ImmutableSet<ClassInfo> getClassesRecursive(String packageName) {  
    checkNotNull(packageName);  
    String packagePrefix = packageName + '.';  
    ImmutableSet.Builder<ClassInfo> builder = ImmutableSet.builder();  
    for (ClassInfo classInfo : classes) {  
        if (classInfo.getName().startsWith(packagePrefix)) {  
            builder.add(classInfo);  
        }  
    }  
    return builder.build();  
}
```

REFERENCES

- [Abid, Dragan, Collard, Maletic 2015] Abid, N. J., Dragan, N., Collard, M. L., and Maletic, J. I., (2015), "Using Stereotypes in the Automatic Generation of Natural Language Summaries for C++ Methods", in *Proceedings of IEEE International Conference on Software Maintenance and Evolution (ICSME'15)*, pp. 561-565.
- [ANTLR 2014] ANTLR, (2014), "The ANTLR Translator generator", Web page, Date Accessed: June 27, 2016, <http://www.antlr2.org/>.
- [Apiwattanapong, Orso, Harold 2004] Apiwattanapong, T., Orso, A., and Harold, M. J., (2004), "A Differencing Algorithm for Object-Oriented Programs", in *19th IEEE International Conference on Automated Software Engineering (ASE '04)*. Linz, Austria: IEEE.
- [Apiwattanapong, Orso, Harold 2007] Apiwattanapong, T., Orso, A., and Harold, M. J., (2007), "JDiff: A Differencing Technique and Tool for Object-oriented Programs", *Automated Software Engineering*, vol. 14, no. 1, pp. 3-36.
- [Asklund 1994] Asklund, U., (1994), "Identifying Conflicts During Structural Merge", in *Proceedings of Nordic Workshop Programming Environment Research*, pp. 231-242.
- [Badros 2000] Badros, G. J., (2000), "JavaML: A Markup Language for Java Source Code", in *Proceedings of 9th International World Wide Web Conference (WWW9)*, Asterdam, The Netherlands, May 13-15.
- [Baxter, Pidgeon, Mehlich 2004] Baxter, I. D., Pidgeon, C., and Mehlich, M., (2004), "DMS: Program Transformations for Practical Scalable Software Evolution", in *Proceedings of 26th International Conference on Software Engineering (ICSE04)*, Edinburgh, Scotland, UK, May 23 -28, pp. 625-634.
- [Bellcore 1988] Bellcore, (1988), "dontcallmedom/spiff", github.com/dontcallmedom/spiff.
- [Berzins 1986] Berzins, V., (1986), "On Merging Software Extensions", *Acta Informatica*, vol. 23, no. 6, pp. 607-619.
- [Berzins 1994] Berzins, V., (1994), "Software Merge: Semantics of Combining Changes to Programs", *ACM Transactions on Programming Languages and Systems*, vol. 16, no. 6, pp. 1875-1903.
- [Bille 2005] Bille, P., (2005), "A Survey on Tree Edit Distance and Related Problems", *Theoretical Computer Science*, vol. 337, no. 1, pp. 217-239.
- [Binkley, Horwitz, Reps 1995] Binkley, D., Horwitz, S., and Reps, T., (1995), "Program Integration for Languages with Procedure Calls", *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 4, no. 1, pp. 3-35.
- [Boshernitsan, Graham 2000] Boshernitsan, M. and Graham, S. L., (2000), "Designing an XML-Based Exchange Format for Harmonia", in *Proceedings of Seventh Working Conference on Reverse Engineering (WCRE'00)*, Brisbane, Australia, November 23-25, pp. 287-289.
- [Calligra] Calligra, Date Accessed: June 27, 2016, <https://github.com/KDE/calligra.git>.
- [Canfora, Cerulo, Penta 2009] Canfora, G., Cerulo, L., and Penta, M. D., (2009), "Ldiff: An enhanced line differencing tool", in *Proceedings of 31st International Conference on Software Engineering (ICSE'09)*, pp. 595-598.
- [Chacon, Straub 2014] Chacon, S. and Straub, B., (2014), *Pro Git*, 2nd ed., Apress.
- [Chawathe, Rajaraman, Garcia-Molina, Widom 1996] Chawathe, S. S., Rajaraman, A., Garcia-Molina, H., and Widom, J., (1996), "Change Detection in Hierarchically Structured

- Information", in Proceedings of ACM SIGMOD International Conference on Management of Data, pp. 493-504.
- [Cohen 2010] Cohen, B., (2010), "Patient Diff Advantages", Date Accessed: April 25, 2017, <http://bramcohen.livejournal.com/73318.html>.
- [Collard, Decker, Maletic 2011] Collard, M. L., Decker, M., and Maletic, J. I., (2011), "Lightweight Transformation and Fact Extraction with the srcML Toolkit", in Proceedings of 11th IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM'11), Williamsburg, VA, USA, Sept 25-26, pp. 10 pages.
- [Collard, Decker, Maletic 2013] Collard, M. L., Decker, M. J., and Maletic, J. I., (2013), "srcML: An Infrastructure for the Exploration, Analysis, and Manipulation of Source Code: A Tool Demonstration", in Proceedings, pp. 516-519.
- [Cordy 2003] Cordy, J. R., (2003), "Comprehending Reality-Practical Barriers to Industrial Adoption of Software Maintenance Automation", in Proceedings of 11th IEEE International Workshop on Program Comprehension (IWPC'03), Portland, OR, USA, pp. 196-205.
- [Cortes-Coy, Linares-Vasquez, Aponte, Poshyvanyk 2014] Cortes-Coy, L. F., Linares-Vasquez, M., Aponte, J., and Poshyvanyk, D., (2014), "On Automatically Generating Commit Messages via Summarization of Source Code Changes", in Proceedings of 14th IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM'14), 28-29 Sept. 2014, pp. 275-284.
- [Dawes, Abrahams, Rivera 1998-2007] Dawes, B., Abrahams, D., and Rivera, R., (1998-2007), "boost C++ Libraries", Date Accessed: May 22, 2017, <http://www.boost.org/>.
- [Decker 2012] Decker, M. J., (2012), *Structural Analysis of Source-code Changes in Large Software Through srcDiff and DiffPath*, The University of Akron, Akron, Ohio.
- [Decker, Collard, Volkert, Maletic 2017] Decker, M. J., Collard, M. L., Volkert, L. G., and Maletic, J. I., (2017), "srcDiff: A Syntactic Differencing Approach to Improve the Understandability of Deltas", *Journal of Software: Evolution and Process*.
- [DeltaXML 2000-2016] DeltaXML, (2000-2016), "Diff, Compare, & Merge XML | DeltaXML", Date Accessed: March 24, 2016, <http://www.deltaxml.com/>.
- [Deutskens, de Ruyter, Wetzels, Oosterveld 2004] Deutskens, E., de Ruyter, K., Wetzels, M., and Oosterveld, P., (2004), "Response Rate and Response Quality of Internet-Based Surveys: An Experimental Study", *Marketing Letters*, vol. 15, no. 1, 2004/02/01, pp. 21-36.
- [Dias 2015] Dias, M., (2015), *Supporting Software Integration Activities with First-Class Code Changes*, Laboratoire d'Informatique Fondamentale de Lille.
- [Dictionary.com 2017] Dictionary.com, L., (2017), "Dictionary.com | Meanings and Definitions of Words at Dictionary.com", Date Accessed: July 14, 2017, <http://www.dictionary.com/>.
- [Dotzler, Philippsen 2016] Dotzler, G. and Philippsen, M., (2016), "Move-optimized source code tree differencing", in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*. Singapore, Singapore: ACM, pp. 660-671.
- [Dragan 2010] Dragan, N., (2010), *The Emergent Laws of Method and Class Stereotypes in Object Oriented Software*, Kent State University, Kent, Ohio USA, Ph.D. Dissertation Thesis.
- [Dragan, Collard, Hammad, Maletic 2011] Dragan, N., Collard, M. L., Hammad, M., and Maletic, J. I., (2011), "Using Stereotypes to Help Characterize Commits", in Proceedings of 27th IEEE International Conference on Software Maintenance (ICSM'11), pp. 520-523.

- [Dragan, Collard, Maletic 2006] Dragan, N., Collard, M. L., and Maletic, J. I., (2006), "Reverse engineering method stereotypes", in Proceedings of 22nd IEEE International Conference on Software Maintenance (ICSM'06), Philadelphia, Pennsylvania, USA, pp. 24-34.
- [Dragan, Collard, Maletic 2010] Dragan, N., Collard, M. L., and Maletic, J. I., (2010), "Automatic Identification of Class Stereotypes", in Proceedings of IEEE International Conference on Software Maintenance (ICSM'10), pp. 1-10.
- [Duley, Spandikow, Kim 2012] Duley, A., Spandikow, C., and Kim, M., (2012), "Vdiff: a program differencing algorithm for Verilog hardware description language", *Automated Software Engineering*, vol. 19, no. 4, pp. 459-490.
- [Falleri et al. 2014] Falleri, J.-R. m., Morandat, F. a., Blanc, X., Martinez, M., and Montperrus, M., (2014), "Fine-grained and accurate source code differencing", in *29th ACM/IEEE International Conference on Automated software engineering (ASE'14)*. Vasteras, Sweden: ACM, pp. 313-324.
- [Fluri, Gall 2006] Fluri, B. and Gall, H. C., (2006), "Classifying Change Types for Qualifying Change Couplings", in Proceedings of 14th IEEE International Conference on Program Comprehension (ICPC'06), 0-0 0, pp. 35-45.
- [Fluri, Wursch, Pinzger, Gall 2007] Fluri, B., Wursch, M., Pinzger, M., and Gall, H. C., (2007), "Change Distilling: Tree Differencing for Fine-Grained Source Code Change Extraction", *IEEE Transactions on Software Engineering*, vol. 33, no. 11, pp. 725-743.
- [Foundation 2011] Foundation, A. S., (2011), "Apache™ Subversion®", Date Accessed: April 25, 2017, <https://subversion.apache.org/>.
- [Foundation 2017] Foundation, T. E., (2017), "eclipse", Date Accessed: April 26, 2017, <https://eclipse.org/>.
- [Fowler 1999] Fowler, M., (1999), *Refactoring: Improving the Design of Existing Code*, Addison-Wesley.
- [Fraser 2012] Fraser, N., (2012), "google-diff-match-patch - Diff, Match and Patch libraries for Plain Text - Google Project Hosting", code.google.com/p/google-diff-match-patch/.
- [Free Software Foundation 2010] Free Software Foundation, I., (2010), "Diffutils - GNU Project - Free Software Foundation", Date Accessed: April, 23, 2017, <https://www.gnu.org/software/diffutils/>.
- [Galesic, Bosnjak 2009] Galesic, M. and Bosnjak, M., (2009), "Effects of questionnaire length on participation and indicators of response quality in a web survey", *Public opinion quarterly*, vol. 73, no. 2, pp. 349-360.
- [GitHub 2016] GitHub, (2016), "GitHub", Date Accessed: March 25, 2016, <https://github.com/>.
- [Good, Hardin 2012] Good, P. I. and Hardin, J. W., (2012), *Common errors in statistics (and how to avoid them)*, John Wiley & Sons.
- [Grass 1992] Grass, J. E., (1992), "Cdiff: A Syntax Directed Diff for C++ Programs", in *The Advanced Computing Systems Professional and Technical Association (USENIX) Conference*. Portland, Oregon, pp. 181-193.
- [Hashimoto, Mori 2008] Hashimoto, M. and Mori, A., (2008), "Diff/TS: A Tool for Fine-Grained Structural Change Analysis", in Proceedings of 15th Working Conference on Reverse Engineering (WCRE'08) 15-18 Oct. 2008, pp. 279-288.
- [Hirschberg 1975] Hirschberg, D. S., (1975), "A Linear Space Algorithm for Computing Maximal Common Subsequences", *Communications of the ACM*, vol. 18, no. 6, pp. 341-343.
- [Hirschberg 1977] Hirschberg, D. S., (1977), "Algorithms for the Longest Common Subsequence Problem", *Journal of the ACM*, vol. 24, no. 4, pp. 664-675.

- [Holt, Winter, Schürr 2000] Holt, R. C., Winter, A., and Schürr, A., (2000), "GXL: Toward a Standard Exchange Format", in Proceedings of 7th Working Conference on Reverse Engineering (WCRE '00), Brisbane, Queensland, Australia, November, 23 - 25, pp. 162-171.
- [Horwitz, Prins, Reps 1989] Horwitz, S., Prins, J., and Reps, T., (1989), "Integrating Noninterfering Versions of Programs", *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 11, no. 3, pp. 345-387.
- [Hunt, Tichy 2002] Hunt, J. J. and Tichy, W. F., (2002), "Extensible Language-Aware Merging", in Proceedings of IEEE International Conference on Software Maintenance (ICSM'02), Montreal, Canada, October 3-6, pp. 511-520.
- [Hunt, Vo, Tichy 1998] Hunt, J. J., Vo, K.-P., and Tichy, W. F., (1998), "Delta Algorithms: an Empirical Analysis", *ACM Transactions on Software Engineering Methodologies*, vol. 7, no. 2, pp. 192-214.
- [Hunt, Szymanski 1977] Hunt, J. W. and Szymanski, T. G., (1977), "A Fast Algorithm for Computing Longest Common Subsequences", *Communications of the ACM*, vol. 20, no. 5, May, pp. 350 - 353.
- [IBM 1998] IBM, (1998), "XML TreeDiff", Date Accessed: May 16, <http://alphaworks.ibm.com/tech/xmltreediff>.
- [Inc.] Inc., F. S. F., "Git", Date Accessed: April 25, 2017, <https://git-scm.com/>.
- [INRIA] INRIA, "XyDiff", <http://www-rocq.inria.fr/~cobena/cdrom/www/xydiff/eng.htm>.
- [Jackson, Ladd 1994] Jackson, D. and Ladd, D. A., (1994), "Semantic Diff: A Tool for Summarizing the Effects of Modifications", in Proceedings of IEEE International Conference on Software Maintenance (ICSM'94), Victoria, British Columbia, Canada, September 19-23, pp. 243-252.
- [Kagdi 2003] Kagdi, H., (2003), *Using An Island Grammar Approach for Lightweight Parsing : A C++ To SrcML Translator*, Kent State University, Kent, Masters Thesis Thesis.
- [Keel, Snyder 2013] Keel, J. and Snyder, J., (2013), "Kompere", Date Accessed: April 26, 2017, <https://www.kde.org/applications/development/kompare/>.
- [Kientzle 2003] Kientzle, T., (2003), "libarchive - C library and command-line tools for reading and writing tar, cpio, zip, ISO, and other archive formats @ GitHub", Date Accessed: April 24, 2016, <http://libarchive.org/>.
- [Kitware] Kitware, "CMake", Date Accessed: May 22, 2017, <https://cmake.org/>.
- [Kitware 2002-2012] Kitware, (2002-2012), "<GCC_XML description=\"XML output for GCC\"/>", Date Accessed: April 26, 2017, <http://gccxml.github.io/HTML/Index.html>.
- [Kitware 2013-2015] Kitware, (2013-2015), "C-family Abstract Syntax Tree XML Output", <https://github.com/CastXML/CastXML>.
- [Klassa 2011] Klassa, J., (2011), "TkDiff", Date Accessed: April 26, 2017, <http://tkdiff.sourceforge.net/>.
- [Levenshtein 1966] Levenshtein, V. I., (1966), "Binary codes capable of correcting deletions, insertions and reversals", in Proceedings of Soviet physics doklady, pp. 707.
- [Loh, Kim 2010] Loh, A. and Kim, M., (2010), "LSdiff: a program differencing tool to identify systematic structural differences", in *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering*, vol. 2. Cape Town, South Africa: ACM, pp. 263-266.
- [Ltd 2016] Ltd, S. H. P., (2016), "Sublime Text", Date Accessed: May 22, 2017, <https://www.sublimetext.com/>.

- [Mackall 2017] Mackall, M., (2017), "Mercurial SCM", Date Accessed: April 25, 2017, <https://www.mercurial-scm.org/>.
- [Maletic, Collard 2004] Maletic, J. I. and Collard, M. L., (2004), "Supporting Source Code Difference Analysis", in Proceedings of IEEE International Conference on Software Maintenance (ICSM'04), Chicago, Illinois, September 11-17, pp. 210-219.
- [Mammas, Kontogiannis 2000] Mammas, E. and Kontogiannis, C., (2000), "Towards Portable Source Code Representations using XML", in Proceedings of 7th Working Conference on Reverse Engineering (WCRE '00), Brisbane, Queensland, Australia, November, 23 - 25, pp. 172-182.
- [Masek, Paterson 1980] Masek, W. J. and Paterson, M. S., (1980), "A Faster Algorithm Computing String Edit Distances", *Journal of Computer and System Sciences*, vol. 20, no. 1, pp. 18-31.
- [mdb 2015] mdb, (2015), "CVS - Concurrent Versions System", Date Accessed: April 25, 2017, <http://www.nongnu.org/cvs/>.
- [Mens 1999] Mens, T., (1999), "Conditional Graph Rewriting as a Domain-Independent Formalism for Software Evolution", in Proceedings of International Workshop on Applications of Graph Transformations with Industrial Relevance, pp. 127-143.
- [Mens 2002] Mens, T., (2002), "A State-of-the-Art Survey on Software Merging", *IEEE Transactions on Software Engineering*, vol. 28, no. 5, May, pp. 449 - 462.
- [Microsoft 2017] Microsoft, (2017), "Team Foundation Server", Date Accessed: April 25, 2017, <https://www.visualstudio.com/tfs/>.
- [Miller, Myers 1985] Miller, W. and Myers, E. W., (1985), "A File Comparison Program", *Software: Practice and Experience*, vol. 15, no. 11, pp. 1025-1040.
- [Moreno et al. 2013] Moreno, L., Aponte, J., Sridhara, G., Marcus, A., Pollock, L., and Vijay-Shanker, K., (2013), "Automatic generation of natural language summaries for Java classes", in Proceedings of 21st IEEE International Conference on Program Comprehension (ICPC'13), 20-21 May 2013, pp. 23-32.
- [Myers 1986] Myers, E. W., (1986), "An O(ND) difference algorithm and its variations", *Algorithmica*, vol. 1, pp. 16.
- [Nguyen et al. 2012] Nguyen, H. A., Nguyen, T. T., Pham, N. H., Al-Kofahi, J., and Nguyen, T. N., (2012), "Clone Management for Evolving Software", *IEEE Transactions on Software Engineering*, vol. 38, no. 5, pp. 1008-1026.
- [Omori, Maruyama 2008] Omori, T. and Maruyama, K., (2008), "A change-aware development environment by recording editing operations of source code", in *International Working Conference on Mining Software Repositories*. Leipzig, Germany: ACM, pp. 31-34.
- [Pan, Kim, Whitehead 2009] Pan, K., Kim, S., and Whitehead, E. J., Jr., (2009), "Toward an understanding of bug fix patterns", *Empirical Software Engineering*, vol. 14, no. 3, 2009/06/01, pp. 286-315.
- [Pawlik, Augsten 2011] Pawlik, M. and Augsten, N., (2011), "RTED: a robust algorithm for the tree edit distance", *Proceedings of the Very Large Database (VLDB) Endowment*, vol. 5, no. 4, pp. 334-345.
- [Peabody 2016] Peabody, J., (2016), "Mergely - Online diff, merge documents", Date Accessed: March 25, 2016, <http://www.mergely.com/>.
- [Perforce 2017] Perforce, (2017), "Perforce", Date Accessed: April 25, 2017, <https://www.perforce.com/>.

- [Perry, Siy, Votta 2001] Perry, D. E., Siy, H. P., and Votta, L. G., (2001), "Parallel changes in large-scale software development: an observational case study", *ACM Transactions on Software Engineering Methodology*, vol. 10, no. 3, pp. 308-337.
- [Prus 2002-2004] Prus, V., (2002-2004), "Chapter 30. Boost.Program_options", Date Accessed: May 22, 2017, http://www.boost.org/doc/libs/1_64_0/doc/html/program_options.html.
- [Raghavan, Rohana, Podgurski, Augustine 2004] Raghavan, S., Rohana, R., Podgurski, A., and Augustine, V., (2004), "Dex: A Semantic-Graph Differencing Tool for Studying Changes in Large Code Bases", in *Proceedings of 20th IEEE International Conference on Software Maintenance (ICSM'04)*, Chicago, Illinois, September 11 - 14, pp. 188-197.
- [Rho, Wu 1998] Rho, J. and Wu, C., (1998), "An Efficient Version Model of Software Diagrams", in *Proceedings of Fifth Asia Pacific Software Engineering Conference*, pp. 236-243.
- [Robbes, Lanza 2008] Robbes, R. and Lanza, M., (2008), "SpyWare: a change-aware development toolset", in *30th International Conference on Software Engineering (ICSE'08)*, Leipzig, Germany: ACM, pp. 847-850.
- [SAX 2001] SAX, (2001), "Simple API for XML (SAX)", SAX, Web page, Date Accessed: April 24, 2016, <http://www.saxproject.org/>.
- [Semantic-Designs 1995-2012] Semantic-Designs, (1995-2012), "Smart Differencer Tool", Date Accessed: December 2, 2014, <http://www.semdesigns.com/Products/SmartDifferencer/>.
- [Tichy 1988] Tichy, W. F., (1988), "Tools for Software Configuration Management", *SCM*, vol. 30, pp. 1-20.
- [Van De Vanter 2002] Van De Vanter, M. L., (2002), "The Documentary Structure of Source Code", *Information and Software Technology*, vol. 44, no. 13, October 1, pp. 767-782.
- [van den Brand, Sellink, Verhoef 1998] van den Brand, M., Sellink, A., and Verhoef, C., (1998), "Current Parsing Techniques in Software Renovation Considered Harmful", in *Proceedings of 6th International Workshop on Program Comprehension (IWPC'98)*, Ischia, Italy, June 24-26, pp. 108 - 117.
- [Veillard 2004a] Veillard, D., (2004a), "Libxml2 XmlTextReader Interface tutorial", Date Accessed: April 24, <http://www.xmlsoft.org/xmlreader.html>.
- [Veillard 2004b] Veillard, D., (2004b), "The XML C parser and toolkit of Gnome", Date Accessed: April 24, <http://www.xmlsoft.org/>.
- [W3C 1997] W3C, (1997), "Document Object Model (DOM)", W3C, Web page, Date Accessed: April 24, 2017, <https://www.w3.org/DOM/>.
- [W3C 1999a] W3C, (1999a), "XML Path Language (XPath) Version 1.0", W3C, Web page, Date Accessed: April 24, 2017, <https://www.w3.org/TR/xpath/>.
- [W3C 1999b] W3C, (1999b), "XSL Transformations (XSLT) Version 1.0", W3C, Web page, Date Accessed: April 24, 2017, <http://www.w3.org/TR/xslt>.
- [W3C 2000] W3C, (2000), "W3C XML Query (XQuery)", W3C, Web page, Date Accessed: April 24, 2017, <https://www.w3.org/XML/Query/>.
- [W3C 2008] W3C, (2008), "Extensible Markup Language (XML) 1.0 (Fifth Edition)", W3C, Web page, Date Accessed: April 24, 2017, <https://www.w3.org/TR/REC-xml/>.
- [Wagner, Fischer 1974] Wagner, R. A. and Fischer, M. J., (1974), "The String-to-String Correction Problem", *Journal of the ACM* vol. 21, no. 1, pp. 168-173.

- [Wang, DeWitt, Cai 2003] Wang, Y., DeWitt, D. J., and Cai, J.-Y., (2003), "X-Diff: An Effective Change Detection Algorithm for XML Documents", in Proceedings of 19th International Conference on Data Engineering (ICDE'03), Bangalore, India, pp. 519-530.
- [Westfechtel 1991] Westfechtel, B., (1991), "Structure-oriented Merging of Revisions of Software Documents", in Proceedings of 3rd International Workshop on Software Configuration Management, pp. 68-79.
- [Wikipedia® 2017] Wikipedia®, (2017), "Comparison of file comparison tools", Date Accessed: April 26, 2017,
https://en.wikipedia.org/wiki/Comparison_of_file_comparison_tools.
- [Willadsen 2011-2012] Willadsen, K., (2011-2012), "Meld", Date Accessed: March 24, 2016,
<http://meldmerge.org/>.
- [Wuu 1994] Wu, Y., (1994), "How to Merge program Texts", *Journal of Systems and Software*, vol. 27, no. 2, pp. 129-135.
- [Yang, Horwitz, Reps 1992] Yang, W., Horwitz, S., and Reps, T., (1992), "A Program Integration Algorithm that Accommodates Semantics-Preserving Transformations", *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 1, no. 3, pp. 310-354.
- [Zhang, Shasha 1989] Zhang, K. and Shasha, D., (1989), "Simple Fast Algorithms for the Editing Distance Between Trees and Related Problems", *SIAM Journal on Computing*, vol. 18, no. 6, pp. 1245-1262.
- [Zhenchang, Eleni 2005] Zhenchang, X. and Eleni, S., (2005), "UMLDiff: an algorithm for object-oriented design differencing", in Proceedings of 20th IEEE/ACM International Conference on Automated Software Engineering (ASE'05), Long Beach, CA, USA, pp. 54-65.